Computer Architecture

Lecture 10a: Memory Controllers II

Dr. Mohammad Sadrosadati
Prof. Onur Mutlu

ETH Zürich

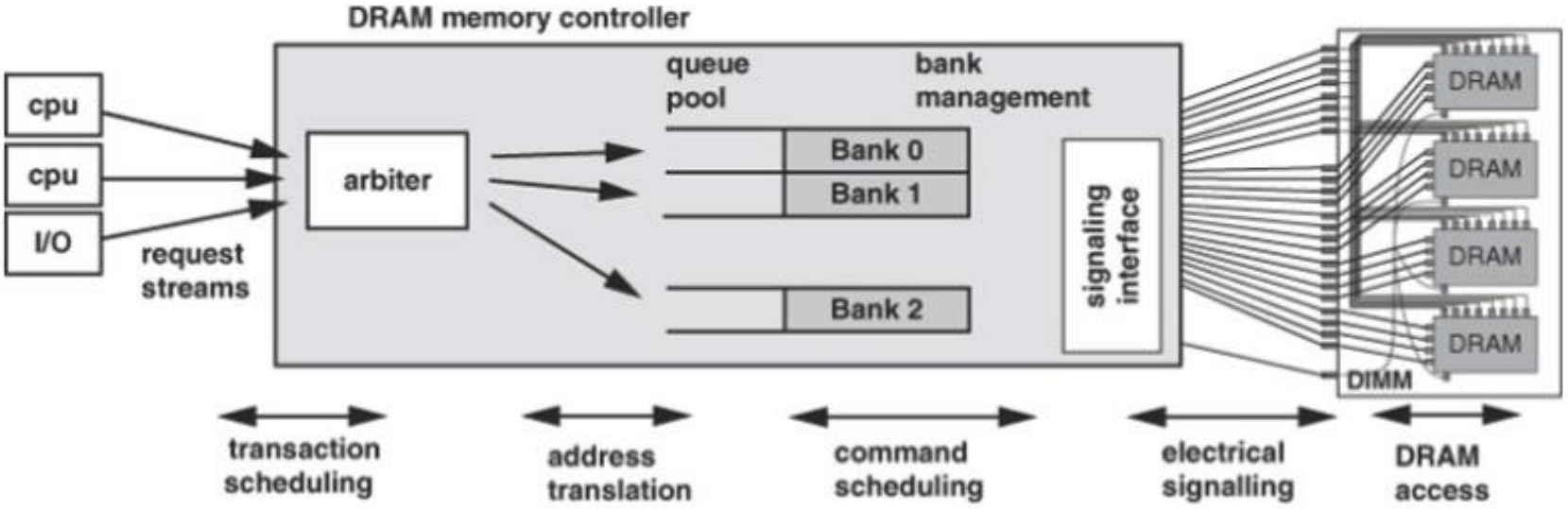
Fall 2025

24 October 2025

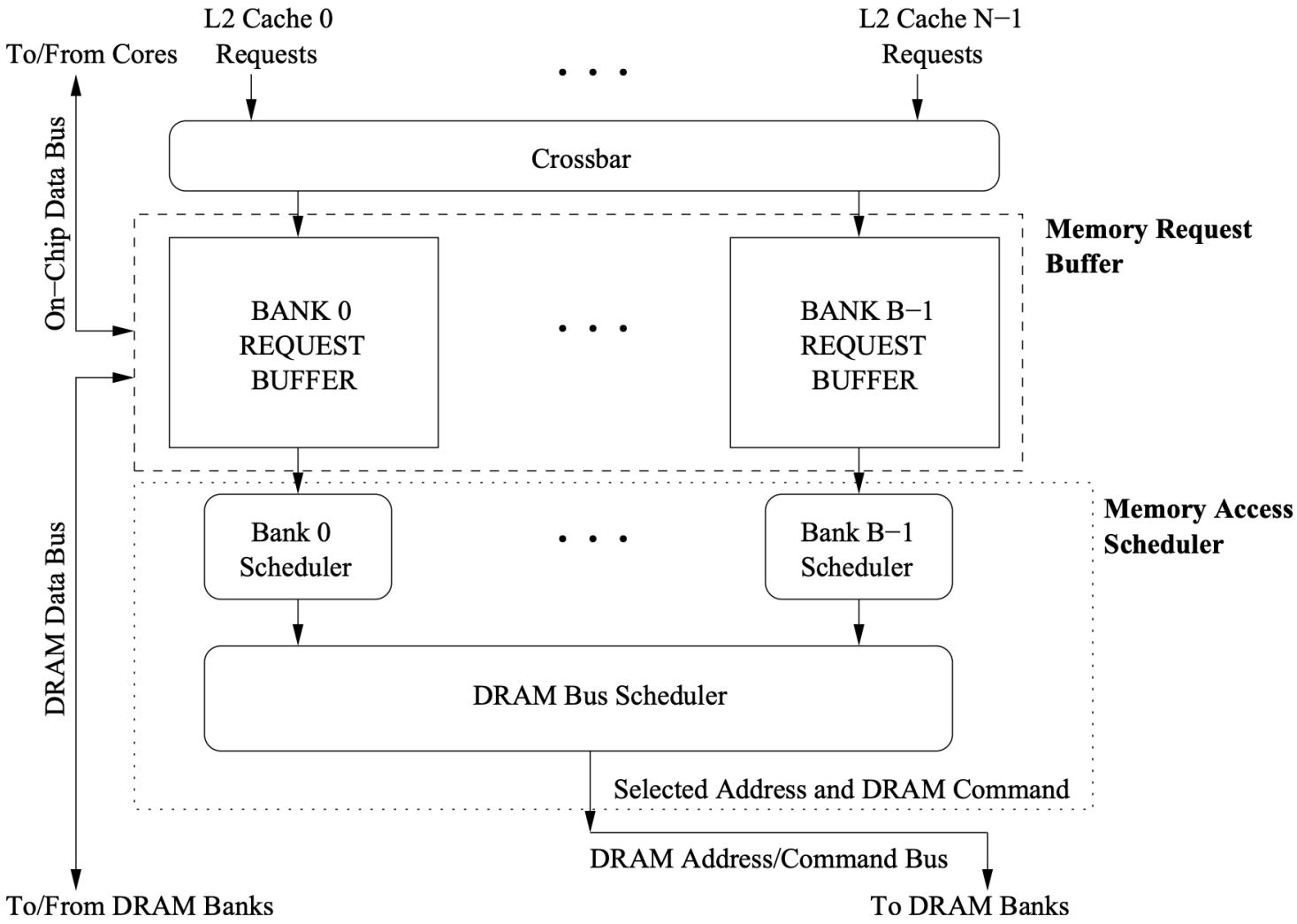
Recall: DRAM Controller: Functions

- Ensure correct operation of DRAM (refresh and timing)
- Service DRAM requests while obeying timing constraints of DRAM chips
  - Constraints: resource conflicts (bank, bus, channel), minimum write-to-read delays
  - Translate requests to DRAM command sequences
- Buffer and schedule requests for high performance + QoS
  - Reordering, row-buffer, bank, rank, bus management
- Manage power consumption and thermals in DRAM
  - Turn on/off DRAM chips, manage power modes

A Modern DRAM Controller (I)



A Modern DRAM Controller



Mutlu+, "Stall-Time Fair Memory Scheduling," MICRO 2007.

DRAM Scheduling Policies (I)

- FCFS (first come first served)
  - Oldest request first

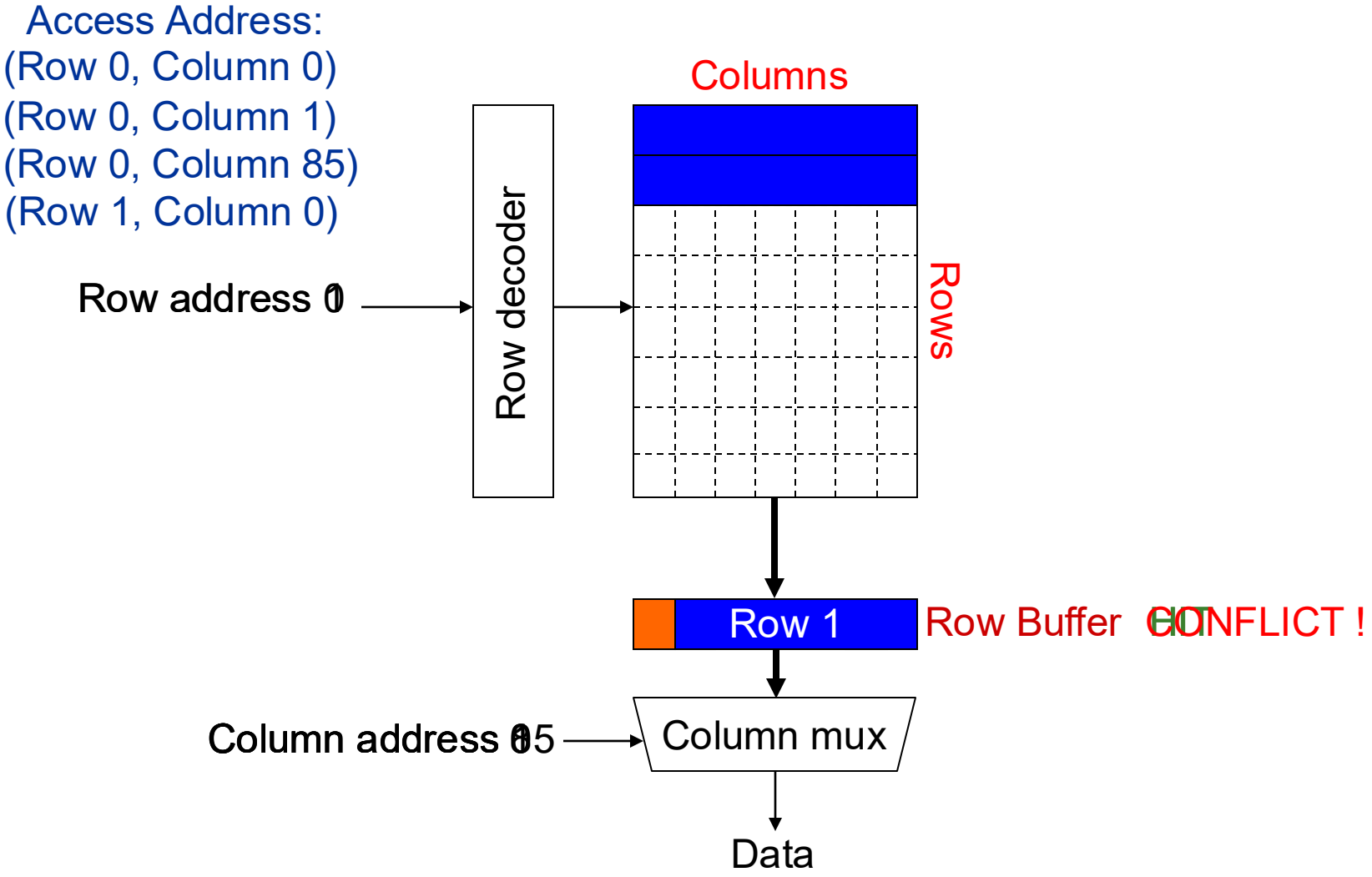
- FR-FCFS (first ready, first come first served)

  1. Row-hit first
  2. Oldest first

  Goal: Maximize row buffer hit rate → maximize DRAM throughput

  - Actually, scheduling is done at the command level
    - Column commands (read/write) prioritized over row commands (activate/precharge)
    - Within each group, older commands prioritized over younger ones

Review: DRAM Bank Operation



DRAM Scheduling Policies (II)

- A scheduling policy is a request prioritization order

- Prioritization can be based on
  - Request age
  - Row buffer hit/miss status
  - Request type (prefetch, read, write)
  - Requestor type (load miss or store miss)
  - Request criticality
    - Oldest miss in the core?
    - How many instructions in core are dependent on it?
    - Will it stall the processor?
  - Interference caused to other cores
  - ...

Row Buffer Management Policies

- Open row
  - Keep the row open after an access

  + Next access might need the same row → row hit

  -- Next access might need a different row → row conflict, wasted energy

- Closed row
  - Close the row after an access (if no other requests already in the request buffer need the same row)

  + Next access might need a different row → avoid a row conflict

  -- Next access might need the same row → extra activate latency

- Adaptive policies
  - Predict whether or not the next access to the bank will be to the same row and act accordingly

Open vs. Closed Row Policies

Policy	First access	Next access	Commands needed for next access
Open row	Row 0	Row 0 (row hit)	Read
Open row	Row 0	Row 1 (row conflict)	Precharge + Activate Row 1 + Read
Closed row	Row 0	Row 0 – access in request buffer (row hit)	Read
Closed row	Row 0	Row 0 – access not in request buffer (row closed)	Activate Row 0 + Read + Precharge
Closed row	Row 0	Row 1 (row closed)	Activate Row 1 + Read + Precharge

DRAM Power Management

- DRAM chips have power modes
- Idea: When not accessing a chip power it down

- Power states
  - Active (highest power)
  - All banks idle
  - Power-down
  - Self-refresh (lowest power)

- Tradeoff: State transitions incur latency during which the chip cannot be accessed

Difficulty of DRAM Control

Why Are DRAM Controllers Difficult to Design?

- Need to obey DRAM timing constraints for correctness
  - There are many (50+) timing constraints in DRAM
  - tWTR: Minimum number of cycles to wait before issuing a read command after a write command is issued
  - tRC: Minimum number of cycles between the issuing of two consecutive activate commands to the same bank
  - …
- Need to keep track of many resources to prevent conflicts
  - Channels, banks, ranks, data bus, address bus, row buffers
- Need to handle DRAM refresh
- Need to manage power consumption
- Need to optimize performance & QoS (in the presence of constraints)
  - Reordering is not simple
  - Fairness and QoS needs complicates the scheduling problem

Many DRAM Timing Constraints

Latency	Symbol	DRAM cycles	Latency	Symbol	DRAM cycles
Precharge	^tRP	11	Activate to read/write	^tRCD	11
Read column address strobe	CL	11	Write column address strobe	CWL	8
Additive	AL	0	Activate to activate	^tRC	39
Activate to precharge	^tRAS	28	Read to precharge	^tRTP	6
Burst length	^tBL	4	Column address strobe to column address strobe	^tCCD	4
Activate to activate (different bank)	^tRRD	6	Four activate windows	^tFAW	24
Write to read	^tWTR	6	Write recovery	^tWR	12

Table 4. DDR3 1600 DRAM timing specifications

- From Lee et al., "DRAM-Aware Last-Level Cache Writeback: Reducing Write-Caused Interference in Memory Systems," HPS Technical Report, April 2010.

More on DRAM Operation

- Kim et al., "A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM," ISCA 2012.
- Lee et al., "Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture," HPCA 2013.

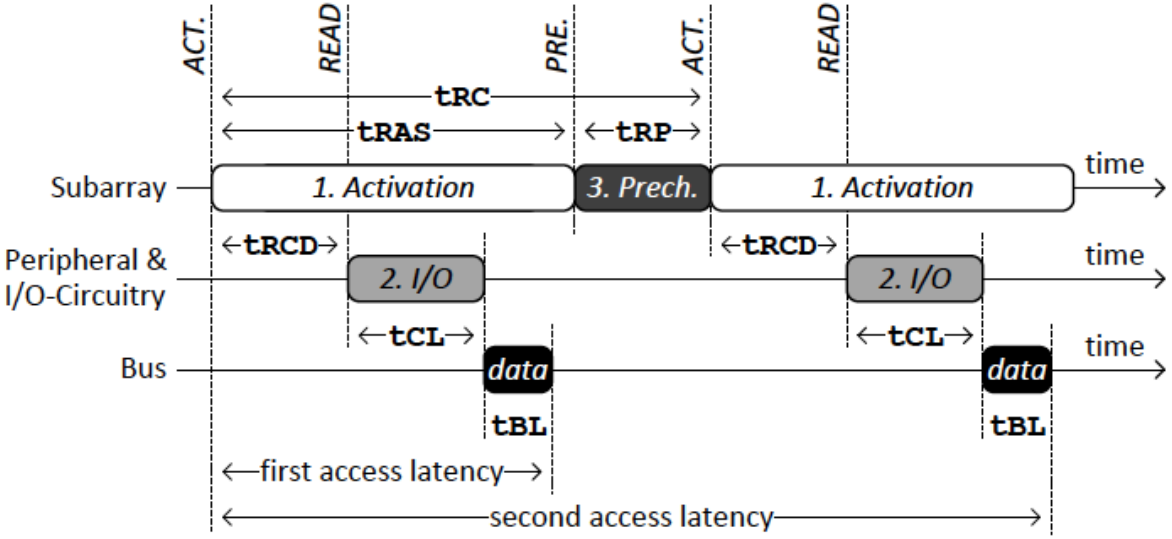
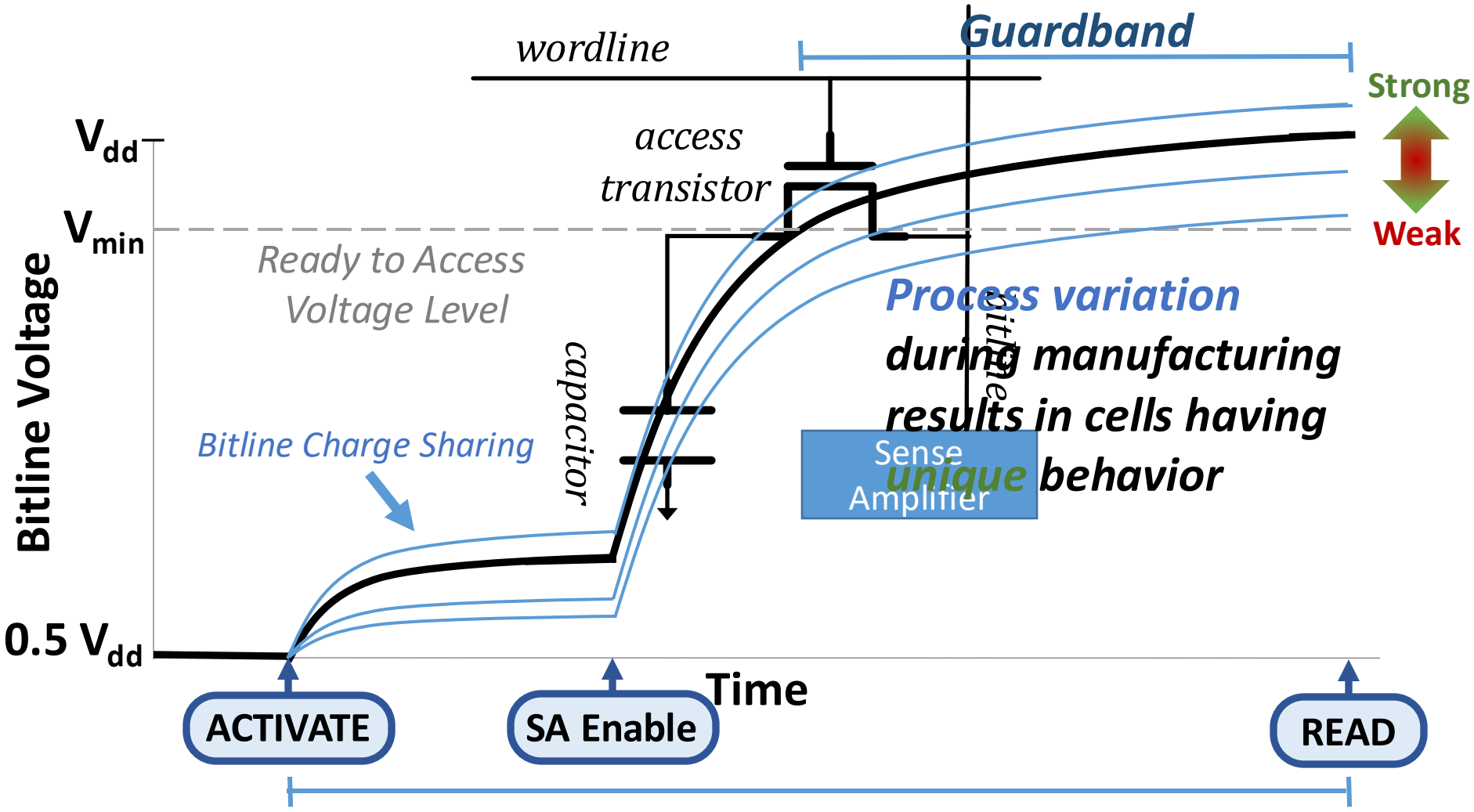


Figure 5. Three Phases of DRAM Access

Table 2. Timing Constraints (DDR3-1066) [43]

Phase	Commands	Name	Value
1	ACT → READ ACT → WRITE	tRCD	15ns
	ACT → PRE	tRAS	37.5ns
2	READ → *data* WRITE → *data*	tCL tCWL	15ns 11.25ns
	data burst	tBL	7.5ns
3	PRE → ACT	tRP	15ns
1 & 3	ACT → ACT	tRC (tRAS+tRP)	52.5ns

Why Timing Constraints?



Why So Many Timing Constraints? (I)

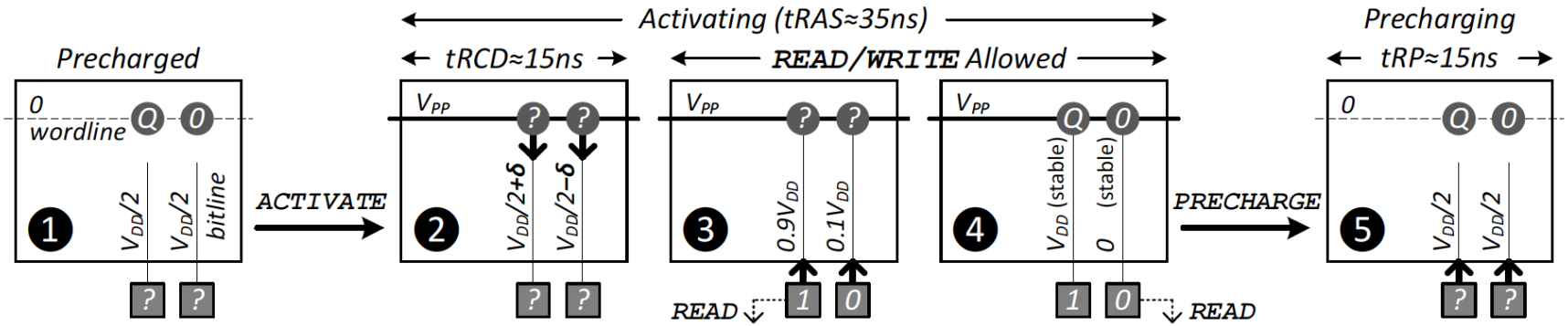


Figure 4. DRAM bank operation: Steps involved in serving a memory request [17] ($V_{PP} > V_{DD}$)

Category	RowCmd↔RowCmd			RowCmd↔ColCmd			ColCmd↔ColCmd			ColCmd→DATA	
Name	*tRC*	*tRAS*	*tRP*	*tRCD*	*tRTP*	*tWR**	*tCCD*	*tRTW*†	*tWTR**	*CL*	*CWL*
Commands	A→A	A→P	P→A	A→R/W	R→P	W*→P	R(W)→R(W)	R→W	W*→R	R→DATA	W→DATA
Scope	Bank	Bank	Bank	Bank	Bank	Bank	Channel	Rank	Rank	Bank	Bank
Value (ns)	**~50**	~35	13-15	13-15	~7.5	**15**	5-7.5	11-15	~7.5	13-15	10-15

A: ACTIVATE– P: PRECHARGE– R: READ– W: WRITE

∗ Goes into effect after the last write *data*, not from the WRITE command

† Not explicitly specified by the JEDEC DDR3 standard [18]. Defined as a function of other timing constraints.

Table 1. Summary of DDR3-SDRAM timing constraints (derived from Micron's 2Gb DDR3-SDRAM datasheet [33])

Kim et al., "A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM," ISCA 2012.

Why So Many Timing Constraints? (II)

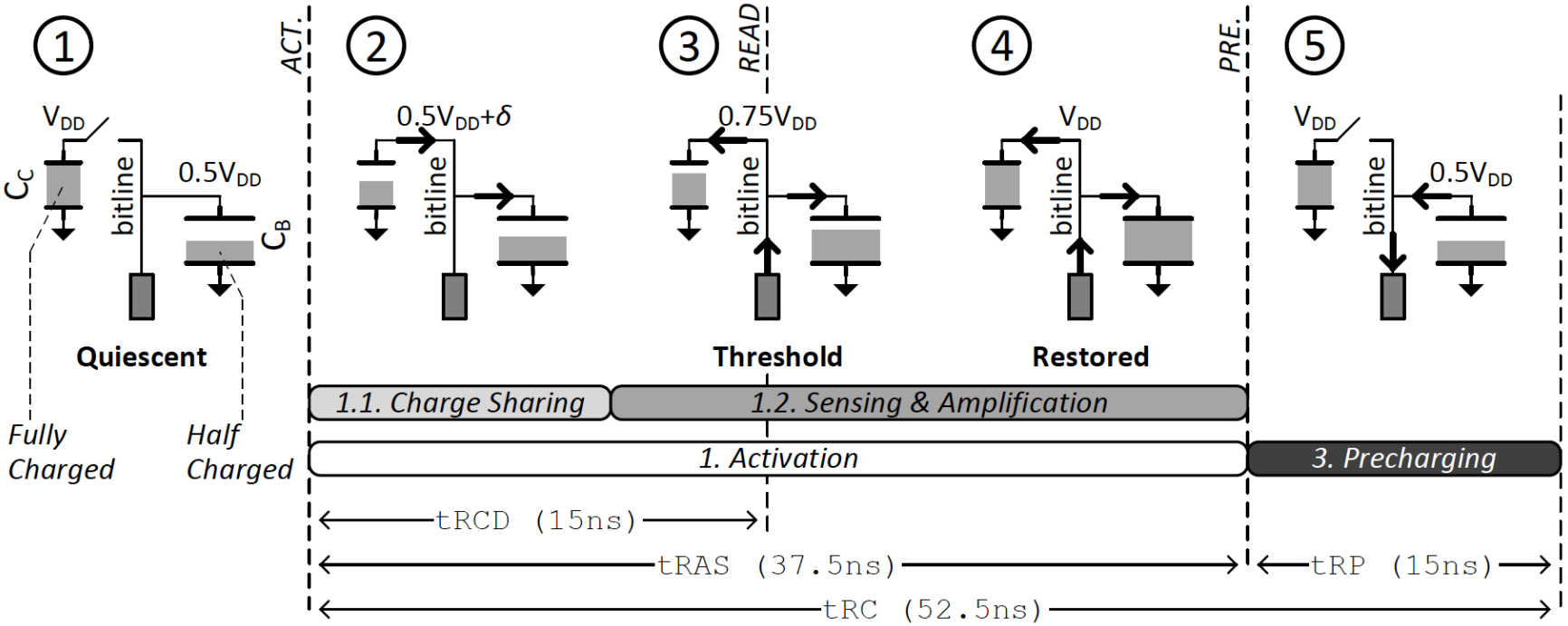


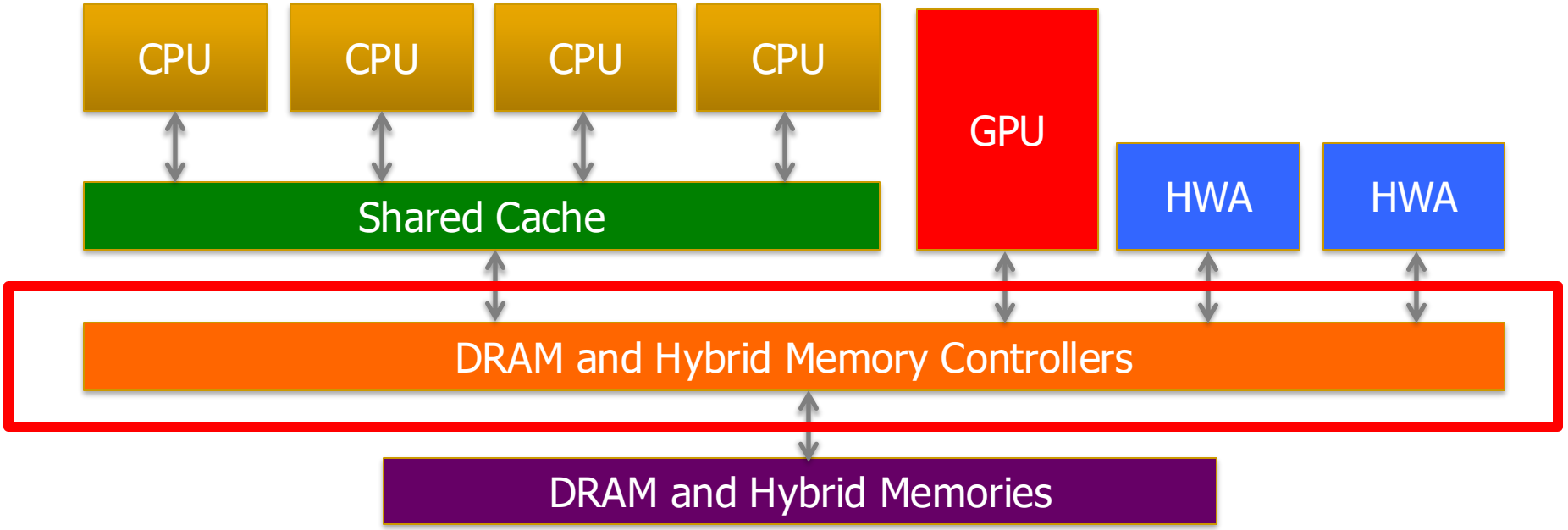
Figure 6. Charge Flow Between the Cell Capacitor (C_C), Bitline Parasitic Capacitor (C_B), and the Sense-Amplifier ($C_B \approx 3.5C_C$ [39])

Lee et al., "Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture," HPCA 2013.

Table 2. Timing Constraints (DDR3-1066) [43]

Phase	Commands	Name	Value
1	ACT → READ ACT → WRITE	tRCD	15ns
	ACT → PRE	tRAS	37.5ns
2	READ → *data* WRITE → *data*	tCL tCWL	15ns 11.25ns
	data burst	tBL	7.5ns
3	PRE → ACT	tRP	15ns
1 & 3	ACT → ACT	tRC (tRAS+tRP)	52.5ns

DRAM Controller Design Is Becoming More Difficult



- Heterogeneous agents: CPUs, GPUs, and HWAs
- Main memory interference between CPUs, GPUs, HWAs
- Many timing constraints for various memory types
- Many goals at the same time: performance, fairness, QoS, energy efficiency, …

Self-Optimizing Memory Controllers

Self-Optimizing Memory Controllers

- Engin Ipek, Onur Mutlu, José F. Martínez, and Rich Caruana, **"Self Optimizing Memory Controllers: A Reinforcement Learning Approach"**
  *Proceedings of the 35th International Symposium on Computer Architecture (**ISCA**)*, pages 39-50, Beijing, China, June 2008.
  Selected to the ISCA-50 25-Year Retrospective Issue covering 1996-2020 in 2023 (Retrospective (pdf) Full Issue).

Self-Optimizing Memory Controllers: A Reinforcement Learning Approach

Engin İpek[1,2] **Onur Mutlu**[2] **José F. Martínez**[1] **Rich Caruana**[1]

[1]Cornell University, Ithaca, NY 14850 USA
[2] Microsoft Research, Redmond, WA 98052 USA

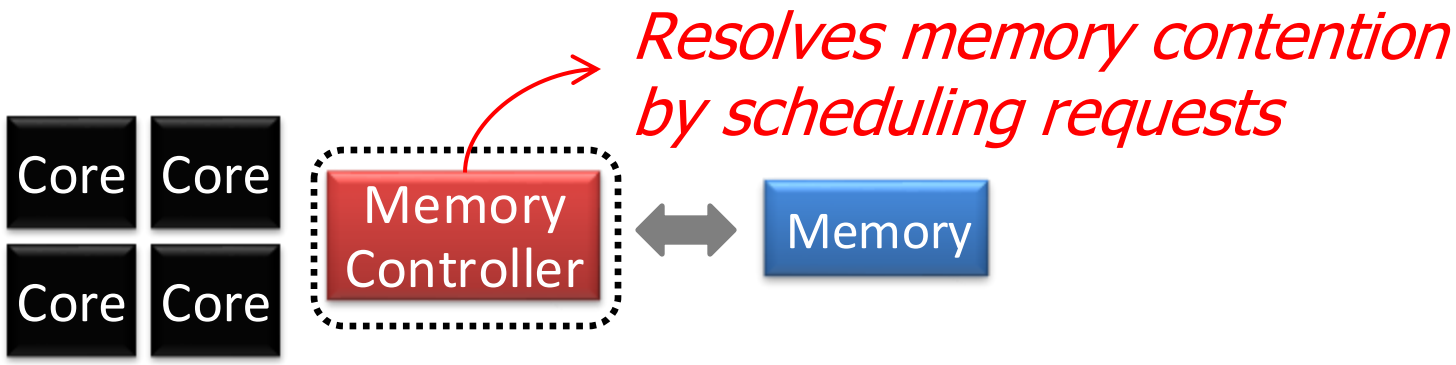
Reality and Dream

- Reality: It is difficult to design a policy that maximizes performance, QoS, energy-efficiency, …
  - Too many things to think about
  - Continuously changing workload and system behavior



- Dream: Wouldn't it be nice if the DRAM controller automatically found a good scheduling policy on its own?

Memory Controller: Performance Function



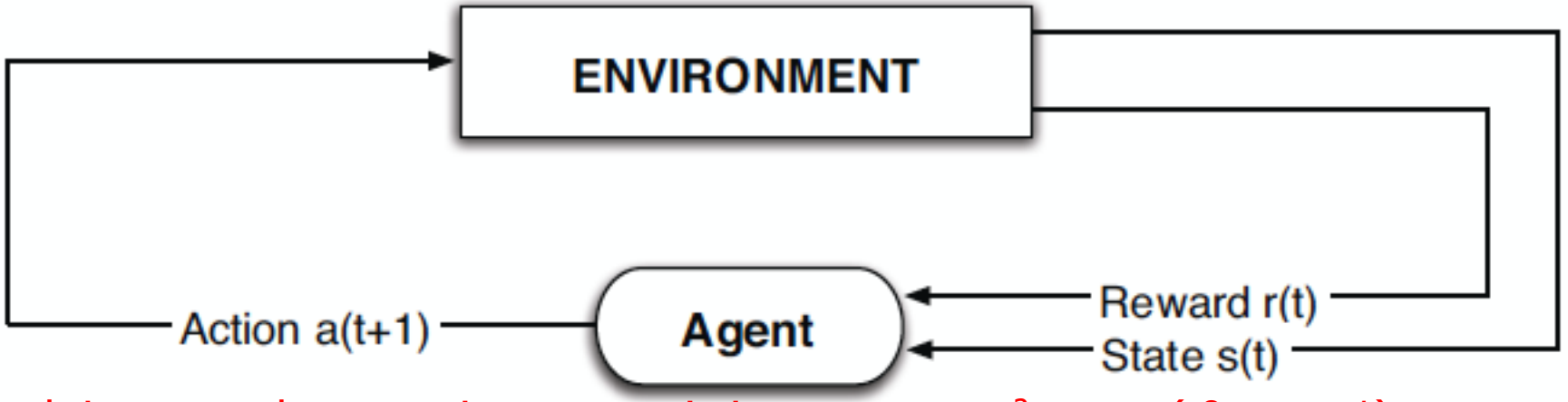
How to schedule requests to maximize system performance?

Self-Optimizing DRAM Controllers

- Problem: DRAM controllers are difficult to design
  - It is difficult for human designers to design a policy that can adapt itself very well to different workloads and different system conditions
- Idea: A memory controller that adapts its scheduling policy to workload behavior and system conditions using machine learning.
- Observation: Reinforcement learning maps nicely to memory control.
- Design: Memory controller is a reinforcement learning agent
  - It dynamically and continuously learns and employs the best scheduling policy to maximize long-term performance.

Ipek+, "Self Optimizing Memory Controllers: A Reinforcement Learning Approach," ISCA 2008.

Self-Optimizing DRAM Controllers

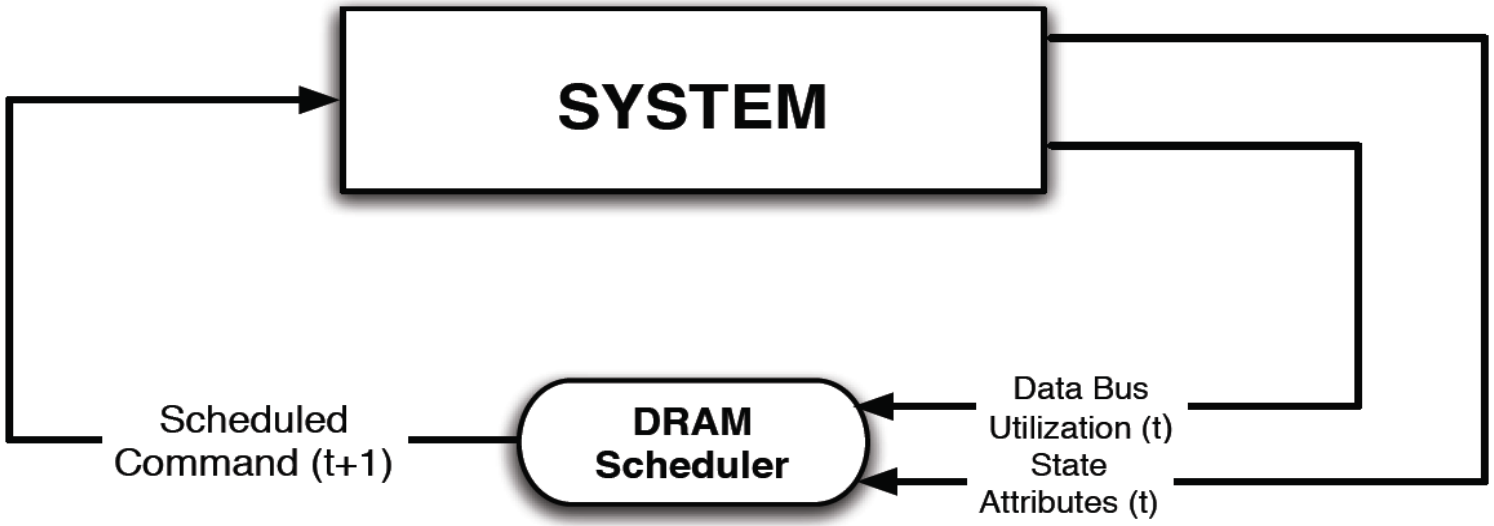


Goal: Learn to choose actions to maximize $r_0 + \gamma r_1 + \gamma^2 r_2 + \ldots$ ($0 \leq \gamma < 1$)

Figure 2: (a) Intelligent agent based on reinforcement learning principles;

Self-Optimizing DRAM Controllers

- Dynamically adapt the memory scheduling policy via interaction with the system at runtime
  - Associate system states and actions (commands) with long term reward values: each action at a given state leads to a learned reward
  - Schedule command with highest estimated long-term reward value in each state
  - Continuously update reward values for <state, action> pairs based on feedback from system



Self-Optimizing DRAM Controllers

- Engin Ipek, Onur Mutlu, José F. Martínez, and Rich Caruana, **"Self Optimizing Memory Controllers: A Reinforcement Learning Approach"**
  *Proceedings of the 35th International Symposium on Computer Architecture (**ISCA**)*, pages 39-50, Beijing, China, June 2008.

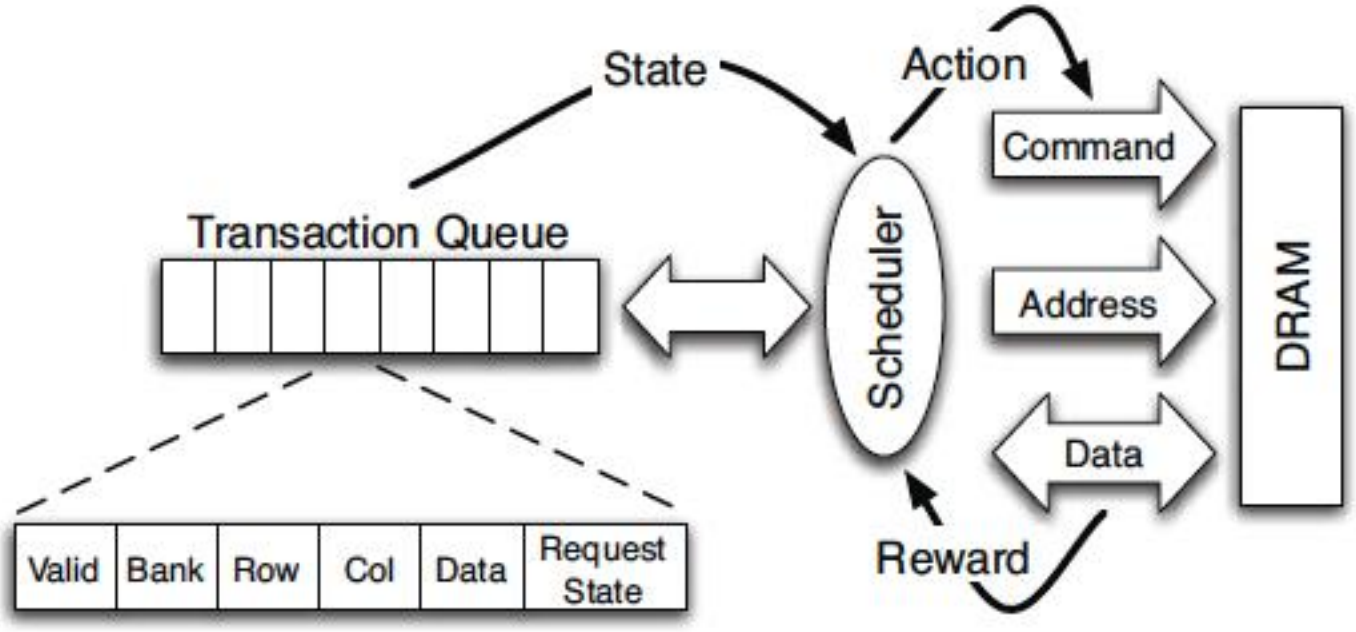


Figure 4: High-level overview of an RL-based scheduler.

States, Actions, Rewards

- Reward function
  - +1 for scheduling Read and Write commands
  - 0 at all other times

  Goal is to maximize long-term data bus utilization
- State attributes
  - Number of reads, writes, and load misses in transaction queue
  - Number of pending writes and ROB heads waiting for referenced row
  - Request's relative ROB order
- Actions
  - Activate
  - Write
  - Read - load miss
  - Read - store miss
  - Precharge - pending
  - Precharge - preemptive
  - NOP

Performance Results

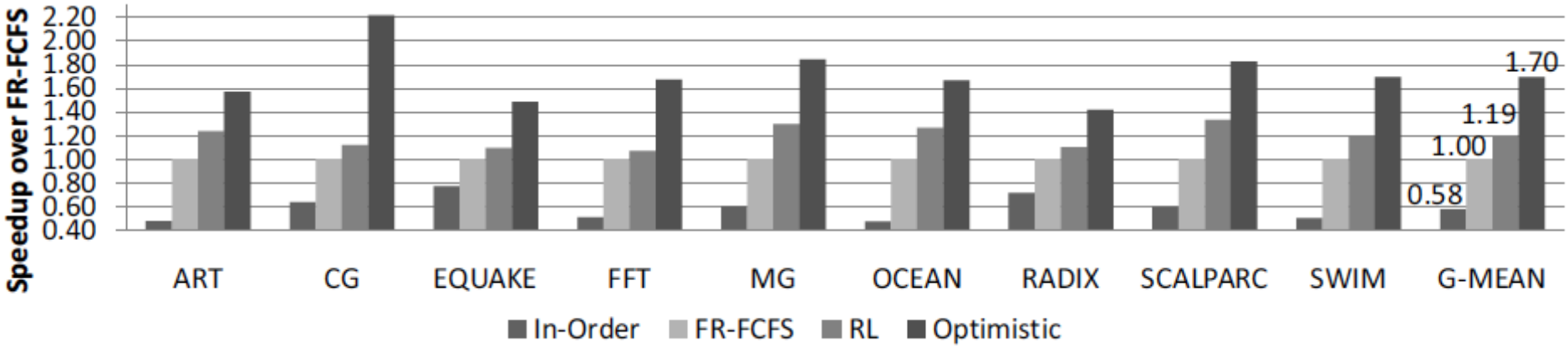


Figure 7: Performance comparison of in-order, FR-FCFS, RL-based, and optimistic memory controllers

Large, robust performance improvements over many human-designed policies

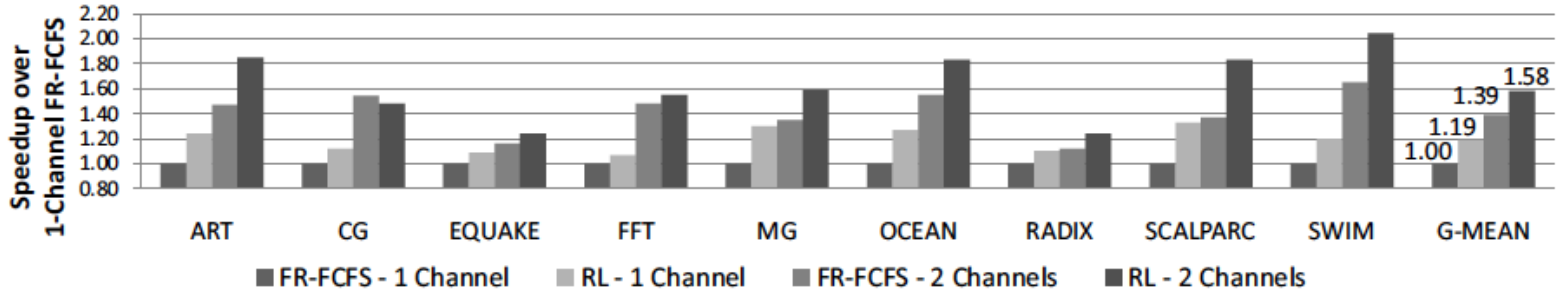


Figure 15: Performance comparison of FR-FCFS and RL-based memory controllers on systems with 6.4GB/s and 12.8GB/s peak DRAM bandwidth

Self Optimizing DRAM Controllers

+ Continuous learning in the presence of changing environment

+ Reduced designer burden in finding a good scheduling policy.
Designer specifies:

1) What system variables might be useful
2) What target to optimize, but not how to optimize it

-- How to specify different objectives? (e.g., fairness, QoS, …)

-- Hardware complexity?

-- Design **mindset** and flow

Self-Optimizing Memory Controllers

- Engin Ipek, Onur Mutlu, José F. Martínez, and Rich Caruana, **"Self Optimizing Memory Controllers: A Reinforcement Learning Approach"**
  *Proceedings of the 35th International Symposium on Computer Architecture (**ISCA**)*, pages 39-50, Beijing, China, June 2008.
  Selected to the ISCA-50 25-Year Retrospective Issue covering 1996-2020 in 2023 (Retrospective (pdf) Full Issue).

Self-Optimizing Memory Controllers: A Reinforcement Learning Approach

Engin İpek[1,2] **Onur Mutlu**[2] **José F. Martínez**[1] **Rich Caruana**[1]

[1]Cornell University, Ithaca, NY 14850 USA
[2] Microsoft Research, Redmond, WA 98052 USA

Retrospective After 15 Years [ISCA 2023]

RETROSPECTIVE: Self-optimizing Memory Controllers: A Reinforcement Learning Approach

José F. Martínez
Cornell University
Ithaca, NY USA
martinez@cornell.edu

Engin Ipek
Qualcomm
San Diego, CA
eipek@qti.qualcomm.com

Onur Mutlu
ETH Zurich
Zurich, Switzerland
onur.mutlu@inf.ethz.ch

Rich Caruana
Microsoft Research
Redmond, WA USA
rcaruana@microsoft.com

I. The Path to Reinforcement Learning

Around the time we began working on this idea, the computer architecture community was not particularly familiar with machine learning (ML), and solutions to both offline (i.e., design- or compile-time) and online (i.e., run-time) problems were largely based on ad hoc heuristics. Some proposals did employ ML offline, including genetic algorithms for automatic derivation of branch predictor designs [O-14] or artificial neural networks (ANNs) for static branch prediction [O-7].[1] But when it came to online heuristics, ML-based solutions were virtually inexistent, with a notable exception in Jiménez and Lin's perceptron-based branch predictor design [O-19]. A perceptron is essentially a very simple ANN, and thus a form *supervised learning*, a.k.a. "learning with a teacher." Supervised learning matches the branch prediction problem very well, as the outcome of every branch can be used to train the perceptron on what its prediction should have been for that branch. Jiménez and Lin's work was elegant and showed that the perception could outperform the best-known ad hoc heuristics for that problem.

At Cornell University, Engin Ipek had taken Rich Caruana's course in ML and had become enamored with the technology and its possibilities. With collaborators Sally McKee, Bronis de Supinski, and Martin Schulz, Engin and Rich had successfully devised an offline methodology to dramatically cut down the exploration of a microprocessor's design space, using ANNs to pick what configurations to simulate [5]. José Martínez was impressed by this work; however, he was more excited about the potential of inserting an ML engine right into the hardware that could beat the best-known online ad hoc heuristics for key architectural mechanisms, as Jiménez and Lin had done for branch prediction.

Engin and José had been working together on the Core Fusion project [4] and wrestled with a problem that looked nothing like branch prediction: instruction steering. Instruction steering is a problem typical of clustered microarchitectures in general, and Core Fusion in particular: given a group of consecutively fetched instructions, which ones should be sent for execution to what core? Unlike branch prediction, steering decisions don't have an obvious "correct answer," even a posteriori, so supervised learning did not seem like a good fit. It is, in fact, a fairly complex problem because 1) each decision needs to balance multiple constraints (e.g., try to steer producers and consumers to the same core but, at the same time, try to balance the load across cores), and 2) the optimality of a decision depends in part on future unknowns (e.g., will an instruction have many data dependencies with others yet to be fetched, and if so, will we come to regret a decision based solely on current knowledge?). A number of key architecture problems share these two characteristics; instruction scheduling in superscalar processors is another example.

Engin, José, and Rich began to look for ML mechanisms that would be a good fit for instruction steering and scheduling, and soon enough they found promise in reinforcement learning (RL) [O-42]. An RL agent has an a priori understanding of how nominally "good" an action can be. This a priori knowledge is often hardwired into the RL agent (e.g., based on a computer architect's intuition). For example, in instruction scheduling, filling an issue slot is nominally "good," while leaving it unassigned is "bad"—at least in principle. However, when making a decision, RL looks at the system's current state and picks among a set of available actions not based on the perceived immediate reward, but on the estimated long-term effect of taking such an action. This was, fundamentally, an exciting twist: don't be preoccupied with what action seems "best" at each point in time; instead, keep your eyes on the ultimate prize—in our case, minimizing execution time. As the system evolves, based in part on the actions already taken, an RL agent has the capacity to progressively refine its ability to estimate the long-term effect of a particular action in a state similar enough to one visited before. This is called "learning through interaction."

RL based on Q values, in particular, was particularly attractive because it seemed readily implementable in silicon. A Q-value matrix is essentially a multidimensional table that is indexed periodically to 1) read the current best estimation of the long-term effect of taking a particular action in a particular system state, and 2) update the Q-value matrix using newly acquired hindsight as the system evolves. This sounded a lot like something that could be folded into good ol' RAM!

II. A Solution in Search of a Problem

Designing an RL agent for instruction steering and scheduling turned out to be easier said than done, however. A particularly tough challenge was the fact that these operations were on the critical path of a microprocessor's datapath, and designing an RL mechanism that would access a Q-value matrix multiple times, once per possible action, before deciding which action to perform, was likely to take a handful of precious CPU cycles. José asked Engin: where else in a computer system could one find a scheduling problem that, on the one hand, had real impact on execution time, but on the other hand, had built-in slack to insert something as sophisticated as an RL agent? A few days later, Engin came up with something that seemed to have great potential: a memory controller.

A primary function of memory controllers is to schedule memory requests in a way that minimizes execution time. The memory controller had long moved on-chip and was therefore clocked at CPU speed. DDR2 DRAM, on the other hand, was clocked at a fraction of that speed. This meant that, for each memory scheduling decision, an RL-based memory controller would have multiple CPU cycles to ponder options. The "golden standard" of memory scheduling was arguably FR-FCFS by Rixner et al. [O-37], and it was a perfect example of a great design that was heavily reliant on expert intuition and without the capacity to improve at run-time. The stage was set, then: The goal would be to beat FR-FCFS by taking a fundamentally different approach based on RL.

[1] Citations with an O- prefix refer to those in the original paper.

III. Designing the RL-based Memory Scheduler

Engin, José, and Rich (well, mostly Engin) began to dissect the problem into the standard RL components: actions, immediate rewards, and state attributes. Actions were dictated by what a memory controller could do (e.g., issue a read command or activate a row), so there wasn't much to decide there. For immediate rewards, our first try turned out to be quite good: +1 if an action was actually reading or writing data, 0 otherwise. (Janani Mukundan and José would conclude later that better immediate reward values could be obtained offline using genetic algorithms [8].) State attributes were tricky: it was important to sense the ones that best informed the RL agent, and it couldn't be too many, as the number had a direct bearing on the size and effectiveness of the Q-value matrix—a problem known in the ML community as the "curse of dimensionality," which in our case would materialize in the form of more silicon area and longer access times. Engin decided to take a systematic approach, using linear feature selection to narrow down a list of more than 200 attributes that could intuitively be connected to memory scheduling decisions. Importantly, he included attributes that were observable on chip but not necessarily in the memory controller itself (e.g., relative ROB order of memory requests). In fact, among the final six attributes, two of them would be of this kind, so the final design included a description of how one would bus this information along with a memory request.

With the basic design out of the way, we still had to find a way to fit the scheduler's decision-making process into the number of CPU cycles that constituted a single memory cycle for DDR2-800 DRAM and, of course, evaluate the design in a detailed simulator. Around that time, José visited Microsoft Research and met Onur Mutlu, who had a long record of publishing on architectures for memory subsystems and specifically on memory controller design. Shortly afterward, Engin joined Microsoft Research for a co-op under Onur, and it was during that time that the paper really came together. Already convinced that memory controllers need to become more intelligent and having implemented perceptron-based branch predictors himself, Onur bought into the idea immediately, and the whole team spent several months nailing down the design, implementation, detailed experiments, comparisons, and results analyses.

IV. The Reviewers Chime In

The paper was first submitted to MICRO in 2007. Reviewers rejected it, in the process posing several probing questions that would eventually make the final evaluation much stronger. Two really intriguing ones were: 1) Could the baseline FR-FCFS design have benefited from factoring in some of the additional attributes that feature selection found? 2) Could an offline RL-based design (i.e., one where the Q-value matrix is frozen after offline training through simulation and then installed in the final design) match your results? We carried out additional experiments to try these scenarios and were able to show that an online RL approach was still significantly superior to those two. The paper was highly reviewed and accepted to ISCA 2008.

V. The Aftermath

Shortly after publishing the paper, RL pioneer Andrew Barto (UMass-Amherst) and co-author of the seminal book on RL [O-42] with Richard Sutton (U. of Alberta) reached out to us, as he had come across the work and thought was fascinating. The first edition of Sutton & Barto was a frequent source of inspiration and knowledge during our work, so we were incredibly flattered by this! Eventually, as Andrew and Richard were preparing the second edition of their book, Andrew reached out again to tell us that they had decided to include our work as one of eight case studies of successful uses of RL alongside projects like Google's AlphaGo and Watson's Jeopardy!

The design inspired follow-up work by us and many others: at the time of this writing, this paper has been cited over six hundred times. We believe this is in part because the problem was important and the ML-based approach inspiring, and in part because ML over the last ten years has become a major source of interest in computer architecture and beyond. As for the particular implementation itself, it is probably fair to say that it did not age well, as DRAM standards continued to crank up the clock while CPU frequency mostly stagnated, which essentially squeezed the clock margin that the original design enjoyed [3], [10]. However, we believe the fundamentals of the RL-based design can be applied and extended to the faster memory controllers of today and the future, and to other computer architecture problems as well [1], [2], [6], [7], [9], [11].

References

[1] Rahul Bera, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning. In *Intl. Symp. on Microarchitecture (MICRO)*, 2021

[2] Zhuo Chen and Diana Marculescu. Distributed reinforcement learning for power limited many-core system performance optimization. In *Design, Automation & Test in Europe Conf. (DATE)*, 2015

[3] Saugata Ghose, Hyodong Lee, and José F. Martínez. Improving memory scheduling via processor-side load criticality information. In *Intl. Symp. on Computer Architecture (ISCA)*, June 2013

[4] Engin Ipek, Meyrem Kırman, Nevin Kırman, and José F. Martínez. Core Fusion: Accommodating software diversity in chip multiprocessors. In *Intl. Symp. on Computer Architecture (ISCA)*, 2007

[5] Engin Ipek, Sally A. McKee, Rich Caruana, Bronis R. de Supinski, and Martin Schulz. Efficiently exploring architectural design spaces Via predictive modeling. In Intl. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2006

[6] Wonkyung Kang and Sungjoo Yoo. Dynamic management of key states for reinforcement learning-assisted garbage collection to reduce long tail latency in SSD. In *Design Automation Conference (DAC)*, 2018

[7] Sheng-Chun Kao, Geonhwa Jeong, and Tushar Krishna. ConfuciuX: Autonomous hardware resource assign- ment for DNN accelerators using reinforcement learning. In *Intl. Symp. on Microarchitecture (MICRO)*, 2020

[8] Janani Mukundan and José F. Martínez. MORSE: Multi-objective reconfigurable self-optimizing memory scheduler. In *Intl. Symp. on High-Performance Computer Architecture (HPCA)*, 2012

[9] Leeor Peled, Shie Mannor, Uri Weiser, and Yoav Etsion. Semantic locality and context-based prefetching using reinforcement learning. In *Intl. Symp. on Computer Architecture (ISCA)*, 2015

[10] Lavanya Subramanian, Donghyuk Lee, Vivek Seshadri, Harsha Rastogi, and Onur Mutlu. BLISS: Balancing Performance, Fairness and Complexity in Memory Access Scheduling. *IEEE Trans. on Parallel and Distributed Systems (TPDS)*, Vol. 27, No. 10, 2016

[11] Hao Zheng and Ahmed Louri. An energy-efficient network-on-chip design using reinforcement learning. In *Design Automation Conference (DAC)*, 2019

https://bpb-us-w2.wpmucdn.com/sites.coecis.cornell.edu/dist/7/587/files/2023/06/Retrospective__RL.pdf

More on Self-Optimizing DRAM Controllers (II)

- Janani Mukundan and José F. Martinez
  "MORSE: Multi-Objective Reconfigurable Self-Optimizing Memory Scheduler"
  *Proceedings of the 18th International Symposium on High Performance Computer Architecture (**HPCA**)*, New Orleans, Louisiana, February 2012.

MORSE: Multi-objective Reconfigurable Self-optimizing Memory Scheduler

Janani Mukundan **José F. Martínez**

Computer Systems Laboratory
Cornell University
Ithaca, NY, 14850 USA

http://m3.csl.cornell.edu/

Challenge and Opportunity for Future

Self-Optimizing
(Data-Driven)
Computing Architectures

System Architecture Design Today

- Human-driven
  - Humans design the policies (how to do things)
- Many (too) simple, short-sighted policies all over the system
- No automatic data-driven policy learning
- (Almost) no learning: cannot take lessons from past actions

Can we design fundamentally intelligent architectures?

An Intelligent Architecture

- Data-driven
  - Machine learns the "best" policies (how to do things)
- Sophisticated, workload-driven, changing, far-sighted policies
- Automatic data-driven policy learning
- All controllers are intelligent data-driven agents

We need to rethink design (of all controllers)

Self-Optimizing Memory Controllers

- Engin Ipek, Onur Mutlu, José F. Martínez, and Rich Caruana, **"Self Optimizing Memory Controllers: A Reinforcement Learning Approach"**
  *Proceedings of the 35th International Symposium on Computer Architecture (**ISCA**)*, pages 39-50, Beijing, China, June 2008.
  Selected to the ISCA-50 25-Year Retrospective Issue covering 1996-2020 in 2023 (Retrospective (pdf) Full Issue).

Self-Optimizing Memory Controllers: A Reinforcement Learning Approach

Engin İpek[1,2] **Onur Mutlu**[2] **José F. Martínez**[1] **Rich Caruana**[1]

[1]Cornell University, Ithaca, NY 14850 USA
[2] Microsoft Research, Redmond, WA 98052 USA

Self-Optimizing Memory Prefetchers

Rahul Bera, Konstantinos Kanellopoulos, Anant Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu,
"Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning"
Proceedings of the 54th International Symposium on Microarchitecture (***MICRO***), Virtual, October 2021.
[Slides (pptx) (pdf)]
[Short Talk Slides (pptx) (pdf)]
[Lightning Talk Slides (pptx) (pdf)]
[Talk Video (20 minutes)]
[Lightning Talk Video (1.5 minutes)]
[Pythia Source Code (Officially Artifact Evaluated with All Badges)]
[arXiv version]
Officially artifact evaluated as available, reusable and reproducible.



Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning

Rahul Bera[1] Konstantinos Kanellopoulos[1] Anant V. Nori[2] Taha Shahroodi[3,1]
Sreenivas Subramoney[2] Onur Mutlu[1]
[1]ETH Zürich [2]Processor Architecture Research Labs, Intel Labs [3]TU Delft

https://arxiv.org/pdf/2109.12021.pdf

Learning-Based Off-Chip Load Predictors

- Rahul Bera, Konstantinos Kanellopoulos, Shankar Balachandran, David Novo, Ataberk Olgun, Mohammad Sadrosadati, and Onur Mutlu,
  "Hermes: Accelerating Long-Latency Load Requests via Perceptron-Based Off-Chip Load Prediction"
  Proceedings of the 55th International Symposium on Microarchitecture (***MICRO***), Chicago, IL, USA, October 2022.
  [Slides (pptx) (pdf)]
  [Longer Lecture Slides (pptx) (pdf)]
  [Talk Video (12 minutes)]
  [Lecture Video (25 minutes)]
  [arXiv version]
  [Source Code (Officially Artifact Evaluated with All Badges)]
  Officially artifact evaluated as available, reusable and reproducible.
  Best paper award at MICRO 2022.



Hermes: Accelerating Long-Latency Load Requests via Perceptron-Based Off-Chip Load Prediction

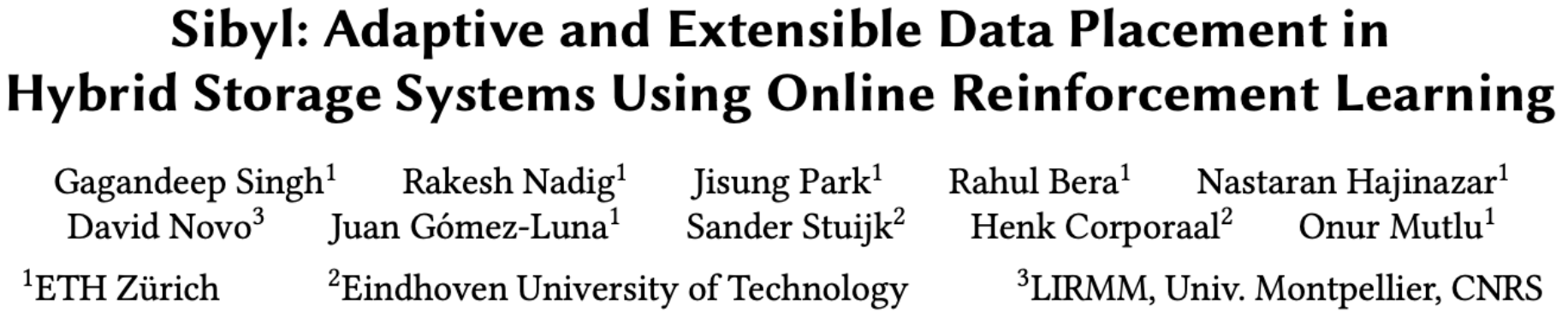
Rahul Bera[1] Konstantinos Kanellopoulos[1] Shankar Balachandran[2] David Novo[3]
Ataberk Olgun[1] Mohammad Sadrosadati[1] Onur Mutlu[1]

[1]ETH Zürich [2]Intel Processor Architecture Research Lab [3]LIRMM, Univ. Montpellier, CNRS

https://arxiv.org/pdf/2209.00188.pdf

Self-Optimizing Hybrid SSD Controllers

Gagandeep Singh, Rakesh Nadig, Jisung Park, Rahul Bera, Nastaran Hajinazar, David Novo, Juan Gomez-Luna, Sander Stuijk, Henk Corporaal, and Onur Mutlu,
"Sibyl: Adaptive and Extensible Data Placement in Hybrid Storage Systems Using Online Reinforcement Learning"
*Proceedings of the 49th International Symposium on Computer Architecture (**ISCA**)*, New York, June 2022.
[Slides (pptx) (pdf)]
[arXiv version]
[Sibyl Source Code]
[Talk Video (16 minutes)]



Sibyl: Adaptive and Extensible Data Placement in Hybrid Storage Systems Using Online Reinforcement Learning

Gagandeep Singh[1] Rakesh Nadig[1] Jisung Park[1] Rahul Bera[1] Nastaran Hajinazar[1]
David Novo[3] Juan Gómez-Luna[1] Sander Stuijk[2] Henk Corporaal[2] Onur Mutlu[1]

[1]ETH Zürich [2]Eindhoven University of Technology [3]LIRMM, Univ. Montpellier, CNRS

https://arxiv.org/pdf/2205.07394.pdf

Architectures for Intelligent Machines

Data-centric

Data-driven

Data-aware

Key Problems with Today's Architectures

- Architectures are terrible at dealing with data
  - Designed to mainly store and move data vs. to compute
  - They are processor-centric as opposed to **data-centric**

- Architectures are terrible at taking advantage of vast amounts of data (and metadata) available to them
  - Designed to make simple decisions, ignoring lots of data
  - They make human-driven decisions vs. **data-driven** decisions

- Architectures are terrible at knowing and exploiting different properties of application data
  - Designed to treat all data as the same
  - They make component-aware decisions vs. **data-aware**

Fundamentally Better Architectures

Data-centric

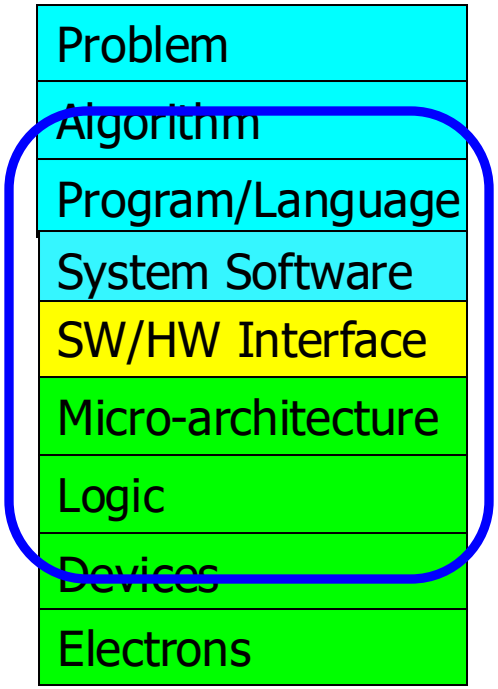
Data-driven

Data-aware



Source: http://spectrum.ieee.org/image/MjYzMzAyMg.jpeg

We Need to Think Across the Entire Stack



We can get there step by step

A Blueprint for Fundamentally Better Architectures

- Onur Mutlu,
  "Intelligent Architectures for Intelligent Computing Systems"
  *Invited Paper in Proceedings of the Design, Automation, and Test in Europe Conference (**DATE**)*, Virtual, February 2021.
  [Slides (pptx) (pdf)]
  [IEDM Tutorial Slides (pptx) (pdf)]
  [Short DATE Talk Video (11 minutes)]
  [Longer IEDM Tutorial Video (1 hr 51 minutes)]

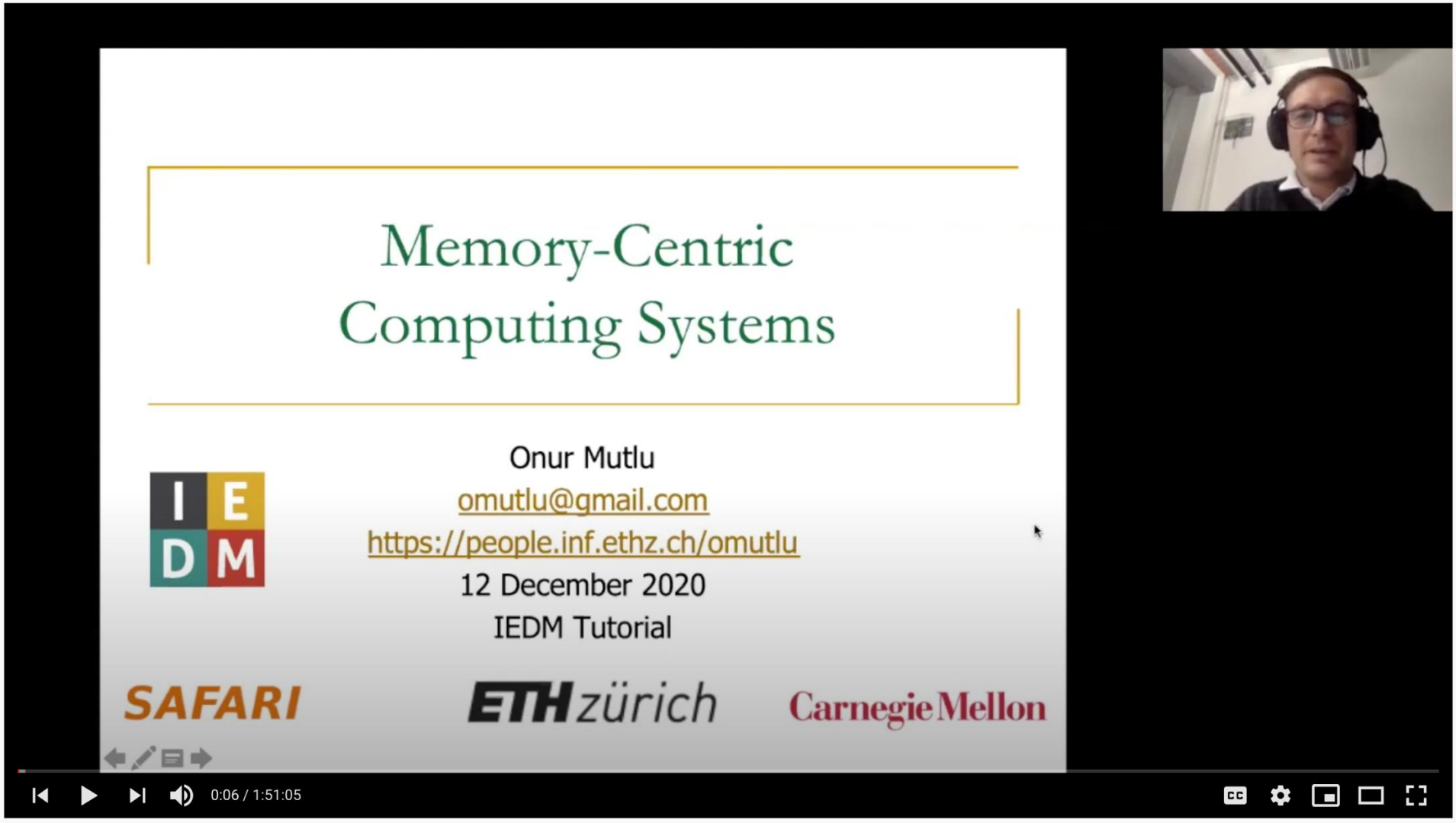

Intelligent Architectures for Intelligent Computing Systems

Onur Mutlu
ETH Zurich
omutlu@gmail.com

A Tutorial on Fundamentally Better Architectures

- Onur Mutlu,
  "Memory-Centric Computing Systems"
  Invited Tutorial at *66th International Electron Devices Meeting (**IEDM**)*, Virtual, 12 December 2020.
  [Slides (pptx) (pdf)]
  [Executive Summary Slides (pptx) (pdf)]
  [Tutorial Video (1 hour 51 minutes)]
  [Executive Summary Video (2 minutes)]
  [Abstract and Bio]
  [Related Keynote Paper from VLSI-DAT 2020]
  [Related Review Paper on Processing in Memory]

https://www.youtube.com/watch?v=H3sEaINPBOE



IEDM 2020 Tutorial: Memory-Centric Computing Systems, Onur Mutlu, 12 December 2020

1,641 views • Dec 23, 2020

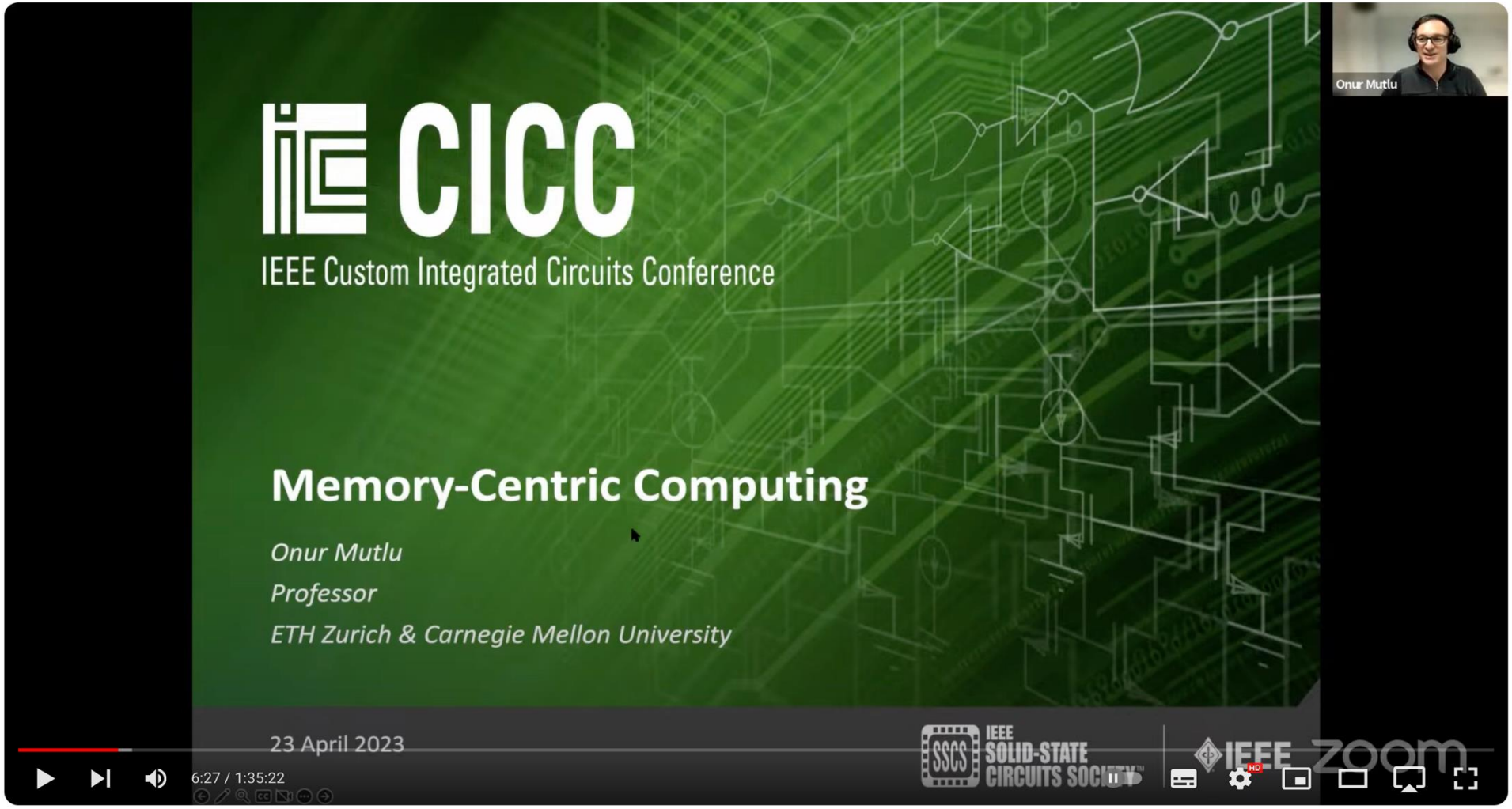
48 0 SHARE SAVE

Onur Mutlu Lectures
13.9K subscribers

https://www.youtube.com/watch?v=H3sEaINPBOE

ANALYTICS EDIT VIDEO

https://www.youtube.com/onurmutlulectures



IEEE CICC Educational Session Talk on Memory-Centric Computing (Prof. Onur Mutlu)

Onur Mutlu Lectures
36.6K subscribers

Subscribe

24 Share Clip Save

526 views Streamed live on 23 Apr 2023
IEEE CICC Educational Session Talk on Memory-Centric Computing
Presenter: Professor Onur Mutlu (https://people.inf.ethz.ch/omutlu/)
Date: April 23, 2023

https://www.youtube.com/watch?v=4x9nujJtqjM

https://www.youtube.com/onurmutlulectures

Recent Thoughts on Processing in Memory



In 2018, HiPEAC spoke to Onur Mutlu (ETH Zurich) about the role of memory in computing systems ['"It's the memory, stupid": A conversation with Onur Mutlu', *HiPEACinfo* 55, 2018, pp. 13-15] just after Onur's course at the HiPEAC summer school, ACACES. Six years later, fresh from a new edition of ACACES, we caught up with Onur to find out what's changed in the field of processing in memory, how disruptive hardware concepts get taken up, promising directions in computer architecture, and more.

How much has changed in this field over the last few years?

A lot, and in a very positive direction. When I first taught at ACACES in 2013, I described our initial ideas on processing in memory (PIM), including RowClone, a simple and effective mechanism for in-memory bulk data copy and initialization. However, at that time, we did not even have our first works published – some had been submitted but were rejected as being unrealistic or niche.

By the time I returned to teach at ACACES in 2018, we had a relatively large body of published work on the topic, so my course included much more material on processing in memory. Our works generated a lot of excitement, but there were still no industrial products or prototypes and there was still large scepticism, even among academics. For example, our Tesseract paper, published at ISCA 2015, was rejected twice before being accepted, while our Ambit work, published at MICRO 2017, was rejected four times before finally being accepted. Both works led to significant follow-on research after publication and have had a lasting influence; in fact, the Tesseract paper was selected for inclusion in the ***ISCA@50 25-Year Retrospective: 1996-2020***.

Six years later, in 2024, we have a completely different landscape. Processing in memory was an even bigger part of my ACACES 2024 course. My lectures covered various aspects of the field, from the near-term 'processing-near-memory' approaches, which place conventional logic elements near memory structures, to the longer-term 'processing-using-memory' approaches, which fundamentally exploit the analogue operational principles of memory structures for logic computation.

As for industry, with Google we have demonstrated the benefits of processing-near-memory for mobile workloads and machine-learning accelerators, while with NVIDIA we showed the benefits of PIM on graphics processing unit (GPU) workloads. There is now at least one commercial product (from a European company, UPMEM) that places a general-purpose programmable multithreaded processor next to each bank in a dynamic random-access memory (DRAM) chip. We experimentally evaluated the benefits and trade-offs of such a PIM product in both my ACACES course and our research papers. Meanwhile, the best paper at HPCA 2024 was based on the UPMEM chip.

One thing is clear: having such hardware has enabled significant progress in the research community by enabling software development, benchmarking, and much better understanding.

There are prototype DRAM chips from multiple large companies with similar near-bank processing capabilities specialized for accelerating machine-learning workloads, e.g. the Samsung FIMDRAM, SK Hynix AiM, and Alibaba's prototype. There are also other prototypes that perform processing near DRAM chips, such as the AxDIMM module from Samsung and Meta. Many startups are also trying to create systems that perform some sort of PIM.

Furthermore, in work published in 2024, we were able to show that commodity DRAM chips, without any modification to the chip itself, just by modifying the memory controller, are able to execute bulk-bitwise operations in a reasonably robust manner. These fascinating findings demonstrate the fundamental computation capability of DRAM, even when DRAM chips are not designed for this purpose.

What does it take for a disruptive hardware concept to be taken up?

Demonstrating results showing that PIM hardware provides large improvements in both performance and energy efficiency helps industry to pay attention. If workloads where the benefits are high are critical workloads to users, the attention level increases.

Disruptive hardware should address a serious need in industry, and the risk involved in changing paradigm should be compensated by significant benefits. For PIM, this is increasingly the case: data access and movement are causing an increasingly worse bottleneck for essentially all metrics we care about (performance, energy, scalability, sustainability, robustness), applications are increasingly bottlenecked by data movement and access, and we are extremely power and energy constrained in our systems (see cutting-edge machine-learning / artificial-intelligence (ML/AI) workloads and systems, for example). At the same time, memory technology is not getting better.

Yet it is not enough to introduce disruptive hardware, even if it is outstanding; we also need to address barriers to adoption. A large software stack needs to be built to make sure the hardware is usable, efficient, and effective – and that is where most of the work needs to be done to make a disruptive hardware concept successful. We therefore focus a lot on software programming, tools, methodologies and compilers in our research. We have multiple works in this area that make life easier for the programmer, by enabling them to decide what to execute where (e.g. PIM-enabled instructions and DAMOV), compile easily into PIM hardware (e.g. SIMDRAM and MIMDRAM), and program using high-level frameworks that enable better productivity and better automated optimizations (e.g. SimplePIM and Dappa).

What are the main memory technologies to keep an eye on?

It is unclear when we will find a new technology that could replace DRAM for main memory and NAND flash for storage – bear in mind that it took at least three decades for NAND flash memory to become widespread.

In my view, DRAM and flash memory will continue to be very strong in the general-purpose domain, despite all the scaling challenges that are plaguing them today (e.g. RowHammer and RowPress) – challenges that would be better handled with more system-memory co-design. For example, industry is finally moving to having intelligence (i.e. logic) in DRAM chips and memory controllers to avoid RowHammer and RowPress bitflips. This is a step in the right direction, but we should also be rethinking the rigid interfaces we have to memory chips today, which give them little breathing room to perform management or computation functions (see, for example, our upcoming work on self-managing DRAM).

For future DRAM, emerging 3D and ferroelectric technologies are promising. While true 3D DRAM may take some time, 3D stacking of DRAM and logic, with a high-quality (i.e. logic process) logic layer, will happen in the shorter term. This will enable much better processing-near-memory.

What do you think are promising directions in computer architecture more generally?

I think we should freely explore creative ideas that have high potential to enable fundamentally better – i.e. more efficient, higher performance, robust, and sustainable – computing systems. Interdisciplinary research to enable systems for bioinformatics, algorithm-hardware co-design, and research that optimizes workloads (e.g. graph analytics, AI, genomics) across the stack, all the way from algorithms to devices, is very valuable. These directions require a more holistic way of thinking about the computing stack and co-designing across the transformation hierarchy, which I call the 'broader view of computing architecture'. The HiPEAC Vision touches on this nicely by calling for 'global co-design'.

One major direction is designing algorithms and systems for biological sequence analysis and information processing (e.g. for genomics). This is going to be even more important in the future as fast, efficient analysis of data is critical for many medical, public health, and personalized-medicine use cases.

https://www.hipeac.net/magazine/7169.pdf

Recent Thoughts on Processing in Memory

Examples from my group include in-storage computing accelerators for genome filtering and in-storage computing for metagenomics. In-storage computing and specialized accelerators placed near flash chips greatly improve the performance and efficiency of many genome analytics workloads by reducing the data movement bottleneck from the storage system and specializing the computation to the primitives needed by genome-analysis workloads.

Processing in memory is another promising direction to accelerate such data-intensive workloads with algorithm-architecture co-design and co-optimization. We are also examining an exciting new paradigm for genome analysis, raw-signal analysis, that operates directly on electrical signals generated by modern nanopore sequencing devices, without requiring the translation of such signals to the genomic alphabet – a very costly process in modern systems.

Another exciting direction is architectural controllers, such as memory controllers, prefetchers, memory and thread-management mechanisms, that are designed such that their policies are not dictated by humans but are data driven, based on observed data in the field via machine-learning techniques. A data-driven architecture enables the machine itself to learn the (best) policies for managing itself and executing programs. Prime examples are reinforcement-learning-based, self-optimizing memory controllers and prefetchers.

We believe an intelligent architecture will consist of a collection of such intelligent controllers that perform automatic data-driven online policy learning, including learning how to best coordinate with each other to make decisions that benefit the overall system. Such machines learn the best policies over time and thus become better as they learn, adapting, evolving, and executing far-sighted policies.

To enable such a machine, we need to revisit the design of all controllers (e.g. caching, prefetching, storage, memory, interconnect) and turn them into data-driven agents. Some example works in this area apply data-driven design principles to memory controllers, prefetchers, memory hierarchy management policies, and hybrid storage systems.

A third direction that is critically important is building fundamentally robust (i.e. safe, reliable, and secure) systems, particularly since computing infrastructure is increasingly used in all scenarios that affect human life today. Robustness should become a goal from the beginning of the design to the end of the lifetime of a computing system. We have a long way to go to achieve this goal and technology scaling does not help us: denser



chips have more robustness problems (e.g. the RowHammer and RowPress phenomena in modern DRAM chips used in essentially all computers), which can be exploited for security attacks or which can manifest themselves as safety problems (think bitflips in self-driving cars, planes and spacecraft).

A final direction I would highlight is architecture-technology co-optimization. Rethinking how modern architectures should be designed for emerging technologies, like tightly integrated packaging and interconnection techniques (e.g. 3D stacking of logic and DRAM, monolithic 3D stacking, 3D DRAM / NVM, promising and unconventional emerging memory technologies) is an exciting and important direction.

What about other considerations, such as sustainability?

As our systems are not designed to be energy efficient and sustainable from the get-go, we are recklessly building huge data centres, mainly to try to satisfy the a particular type of growing ML models, that waste the world's most important energy and power resources. Fundamentally rethinking how we can make the hardware extremely efficient and sustainable is necessary.

With any fundamental rethinking, we should explore various options, including the extremes. A good extreme option for researchers to explore is to take a clean-slate approach and ask how one would design everything in a computing system to be energy efficient, sustainable and high performance. I believe processing everywhere, including in memory and storage, is the right choice if one thinks this way.

A longer version of this interview and all research papers referenced are available on the HiPEAC website:
bit.ly/Onur_Mutlu_interview_2024

Videos of Onur Mutlu's ACACES24 course are available on the HiPEAC YouTube channel: bit.ly/ACACES24_playlist

https://www.hipeac.net/magazine/7169.pdf

Self-Managing DRAM

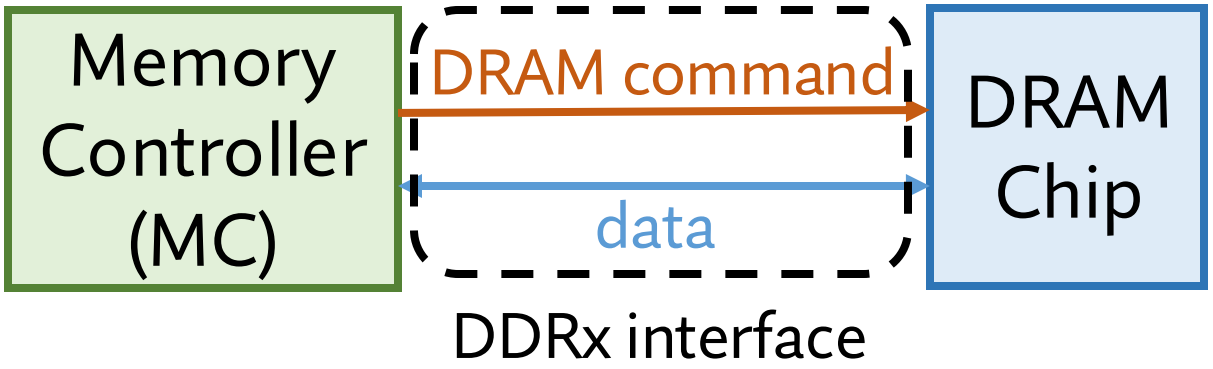
Self Managing DRAM

- Hasan Hassan, Ataberk Olgun, A. Giray Yaglikci, Haocong Luo, and Onur Mutlu,
  "Self-Managing DRAM: A Low-Cost Framework for Enabling Autonomous and Efficient DRAM Maintenance Operations"
  *Proceedings of the 57th International Symposium on Microarchitecture (**MICRO**)*, Austin, TX, USA, November 2024.

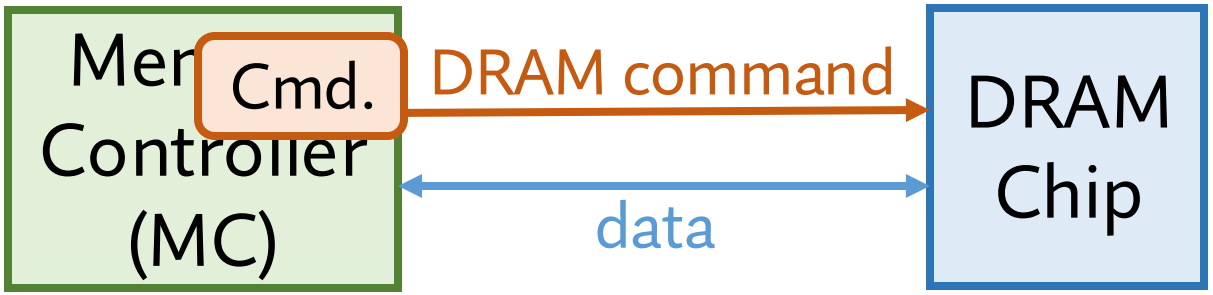
Self-Managing DRAM: A Low-Cost Framework for Enabling Autonomous and Efficient DRAM Maintenance Operations

Hasan Hassan† Ataberk Olgun† A. Giray Yağlıkçı Haocong Luo Onur Mutlu

DRAM Interface Status Quo



DRAM Interface is Rigid



orchestrates
all DRAM operations

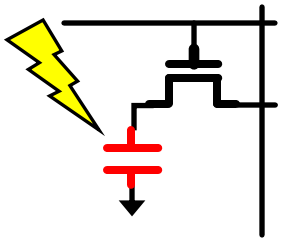
executes
all DRAM commands

- by issuing *DRAM commands*

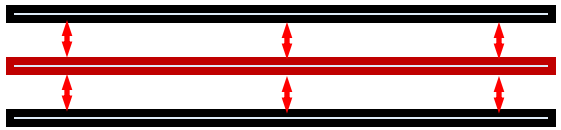
DRAM interface is completely controlled by one side

DRAM Maintenance Mechanisms

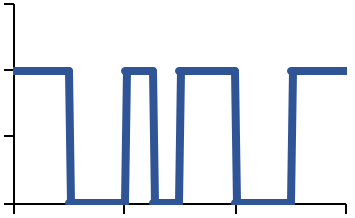
Data Retention



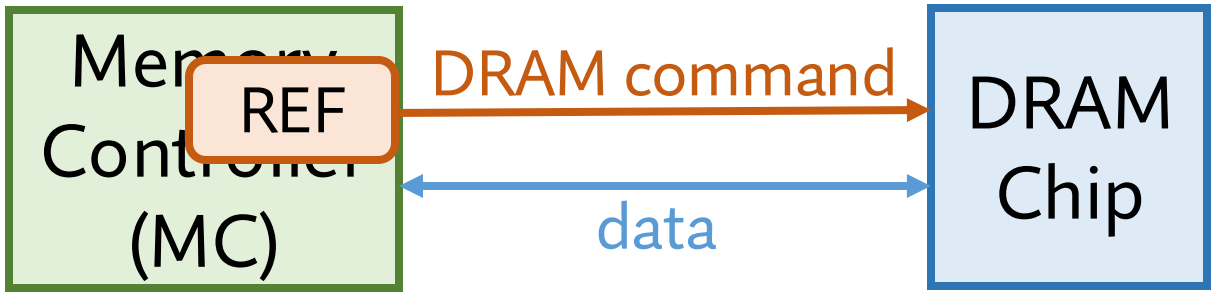
Read Disturbance (e.g., RowHammer)



Variable Retention Time



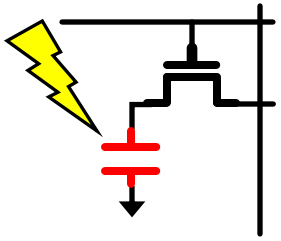
- DRAM failure modes necessitate maintenance mechanisms
- Perform operations to maintain DRAM data integrity
  - A prominent example is periodic refresh



New Maintenance Mechanisms are Needed

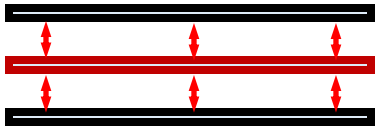
- Density scaling increases memory error rates

Data Retention



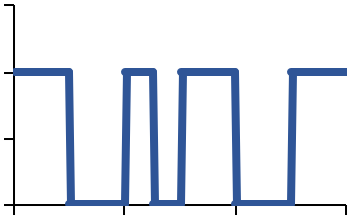
shrinking capacitance worsening leakage

Read Disturbance (e.g., RowHammer)



increasing interference

Variable Retention Time

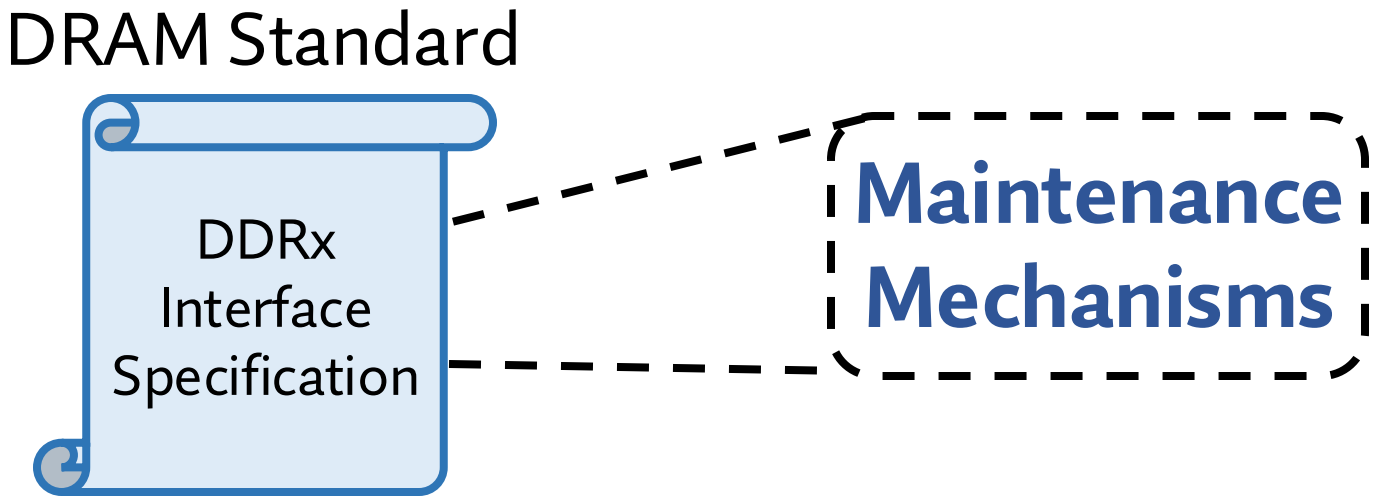


shrinking capacitance worsening leakage

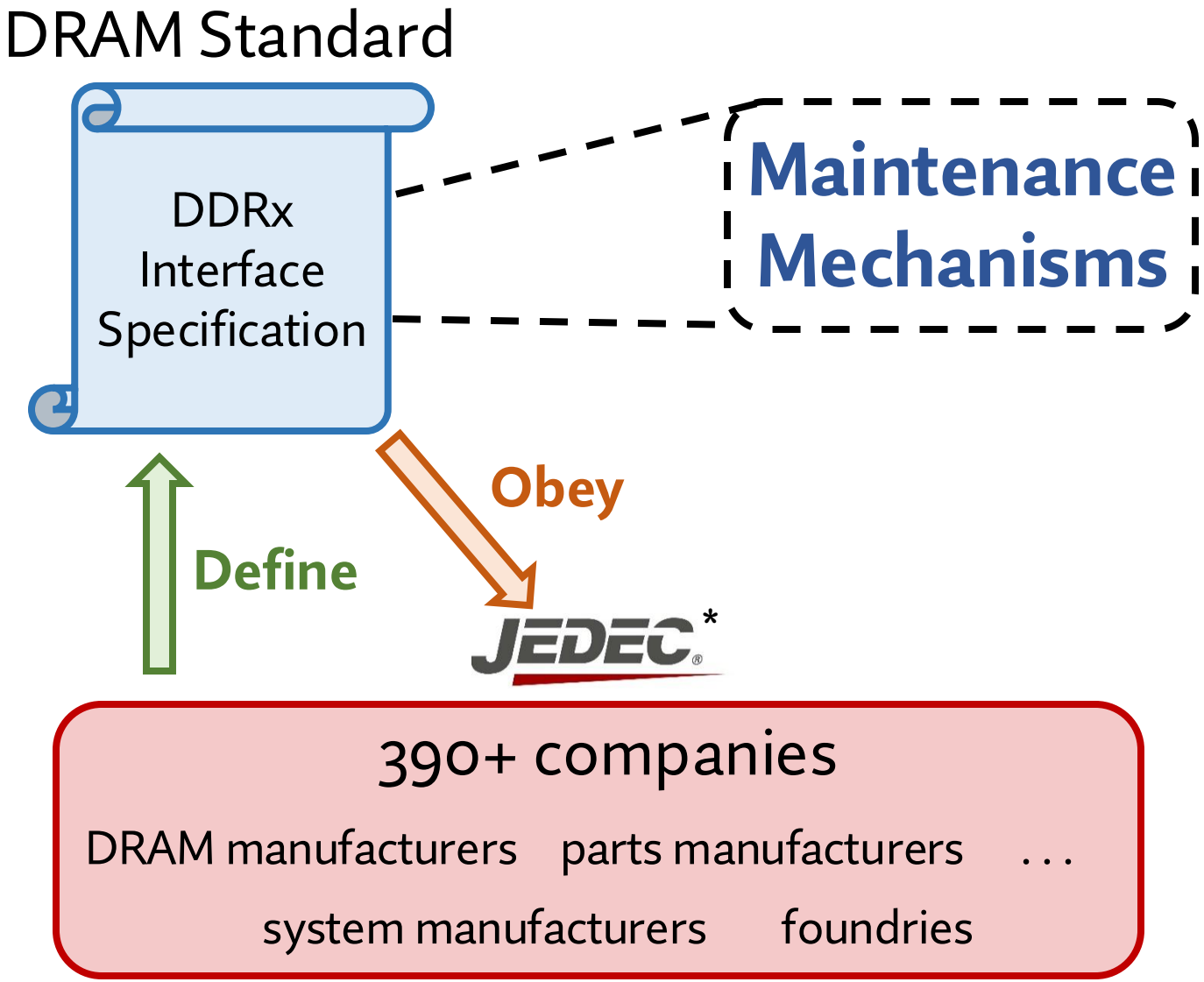
Continued DRAM process scaling necessitates new efficient maintenance mechanisms

[Patel+, MICRO'21]

DRAM Standard Interface Specification

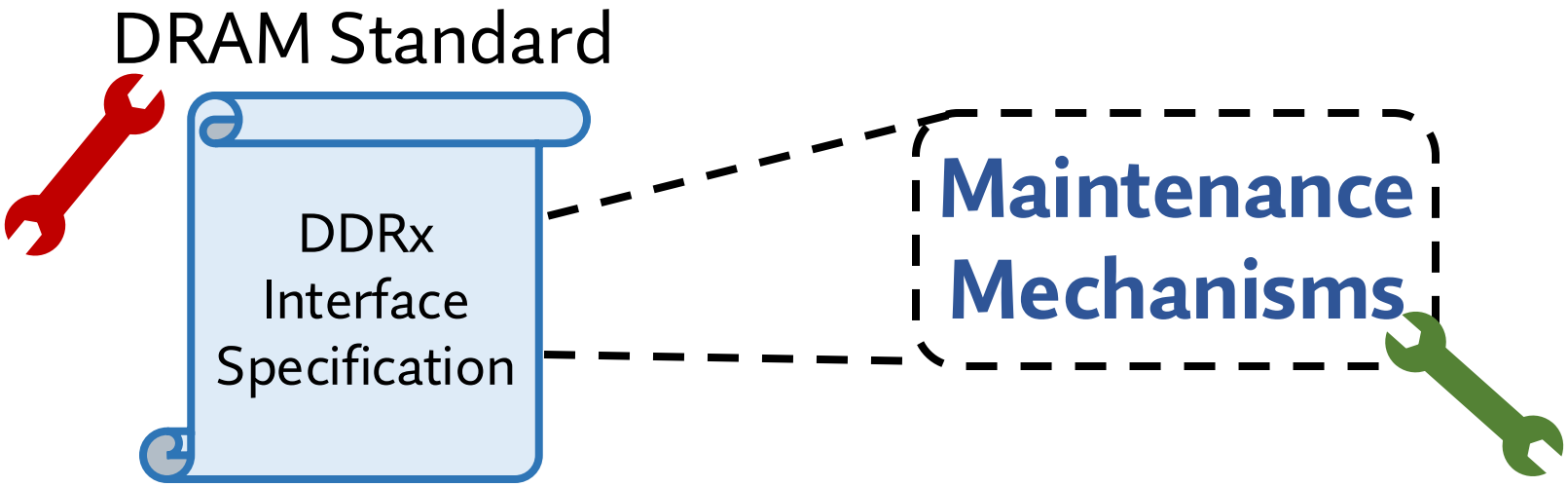


DRAM Standard Body – JEDEC*



*Joint Electron Device Engineering Council

Barrier to New Maintenance Mechanisms

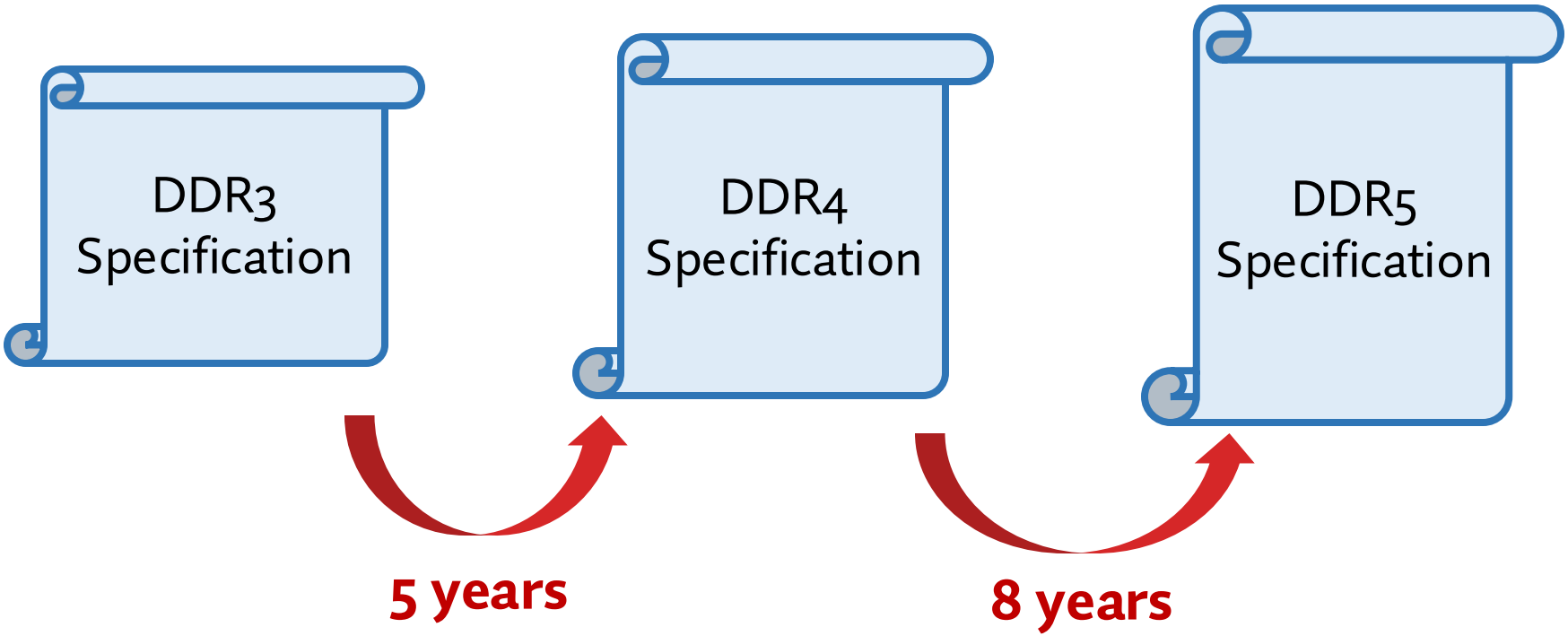


- Adding new or modifying existing maintenance mechanisms requires lengthy modifications to

1. **DRAM specifications** and
2. other system components that obey the specifications

DRAM interface is rigid

DRAM Specifications Evolve Slowly



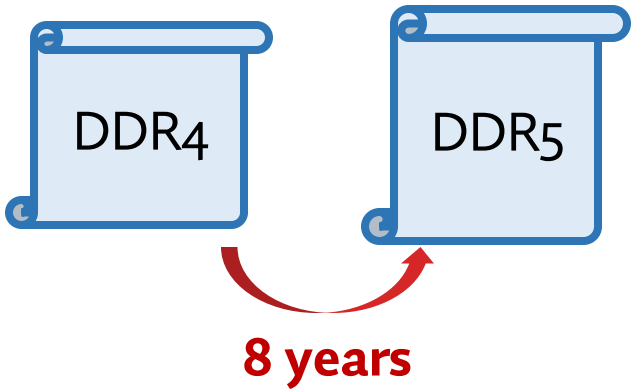
Multi-year effort by the JEDEC committee

Introducing new maintenance operations takes a long time

Recently-Introduced Maintenance Mechanisms

- DDR5 introduces three maintenance techniques

1. **Same Bank Refresh**
   - improve bank-level parallelism
2. **Refresh Management (RFM)**
   - improve robustness
3. **In-DRAM ECC scrubbing**
   - improve error tolerance



These improvements could have been **released earlier**

Problem and Our Goal

Problem

Introducing new maintenance operations
takes a long time

Our Goal

Ease and **accelerate** the process of implementing
new efficient in-DRAM maintenance operations

DRAM Access and Maintenance

- Categorize DRAM operations into **two** classes:

1 Access

- Performed to serve memory requests
- Uses information available *only* to the memory controller
  - e.g., load *address*, store *data*

2 Maintenance

- Performed to maintain DRAM data integrity
- Uses information available *only* to the DRAM chip
  - e.g., in-DRAM row activation counter

DRAM Access and Maintenance

- Categorize DRAM operations into **two** classes:

Key observation:
A DRAM chip could **"maintain" itself**

2 **Maintenance**

- Performed to maintain DRAM data integrity
- Uses information available *only* to the DRAM chip
  - e.g., in-DRAM row activation counter

A DRAM Chip Should Maintain Itself

- Two benefits of DRAM chip "autonomously" performing maintenance operations

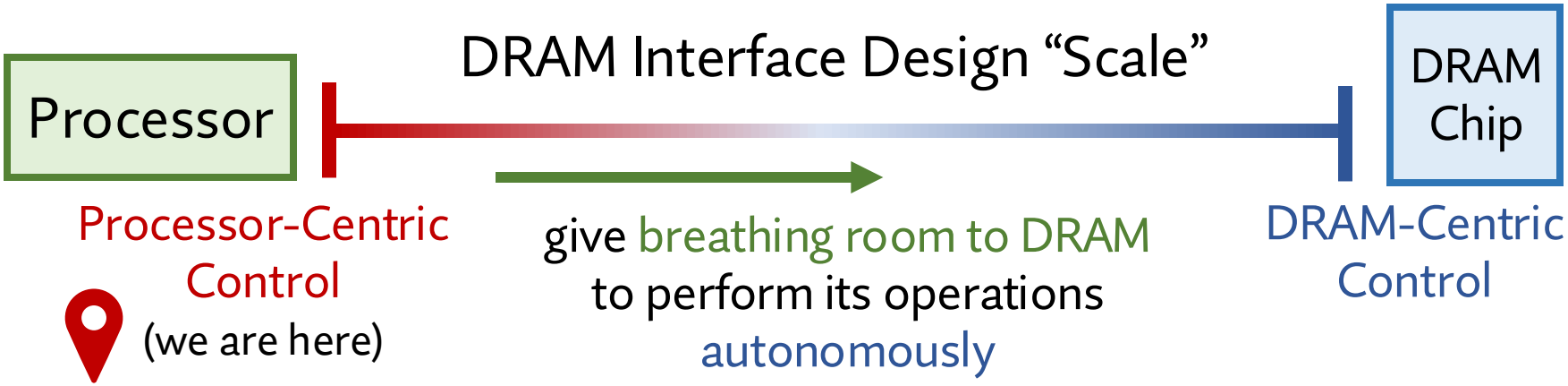
1. Maintenance mechanisms can be implemented more easily and rapidly
   - DRAM interface modifications are not required

2. Enable DRAM manufacturers with breathing room to perform architectural optimizations *without* exposing DRAM-internal proprietary information

Solution Approach

Enable autonomous maintenance operations

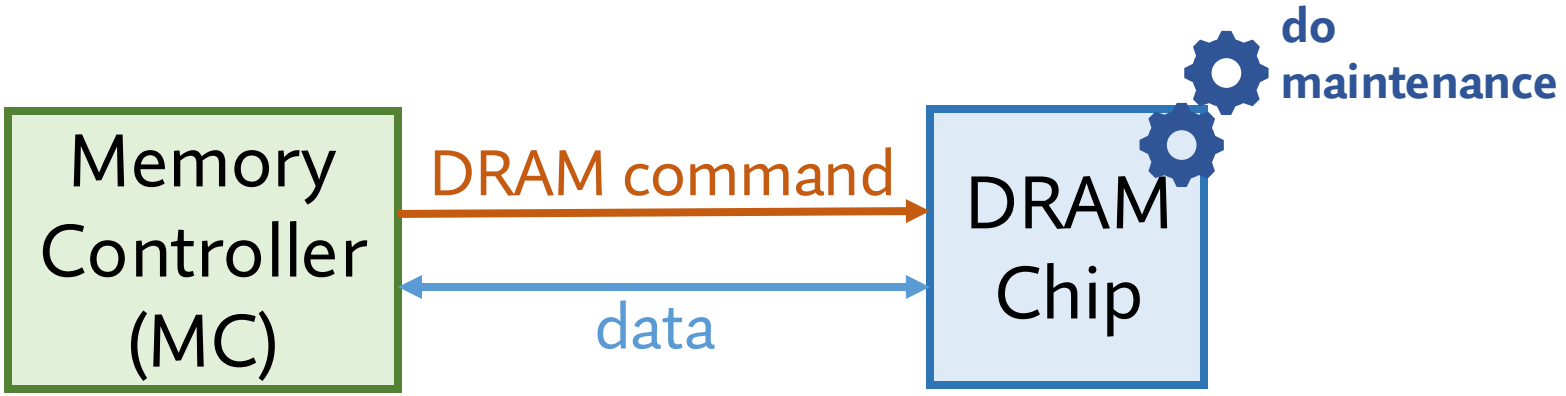
- **Key Challenge:** DRAM interface is too rigid to accommodate autonomous in-DRAM maintenance operations



- **Goal:** Make a simple, one-time change to the DRAM interface that enables autonomous maintenance operations

SMD Key Idea: Autonomous Maintenance

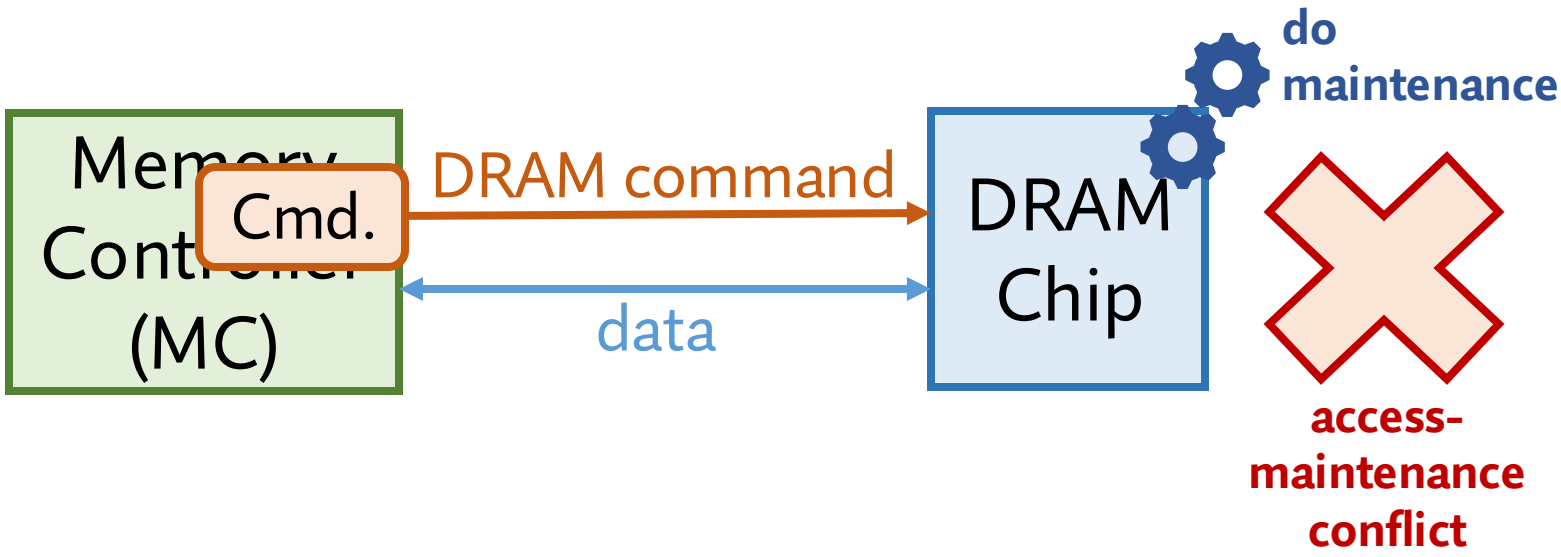
DRAM chip controls in-DRAM maintenance operations



Enable implementing **new maintenance mechanisms** **without** modifying the standard and exposing **DRAM-internal proprietary** information

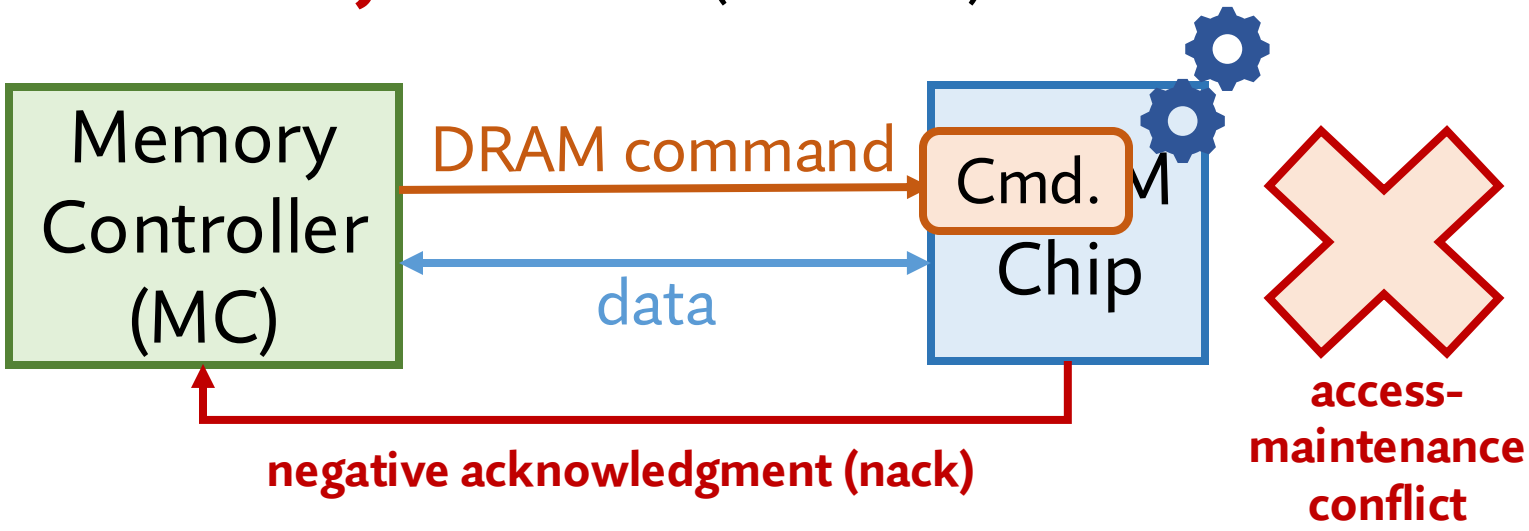
Access-Maintenance Conflicts

- **Problem:** Access-maintenance conflict



SMD Key Mechanism

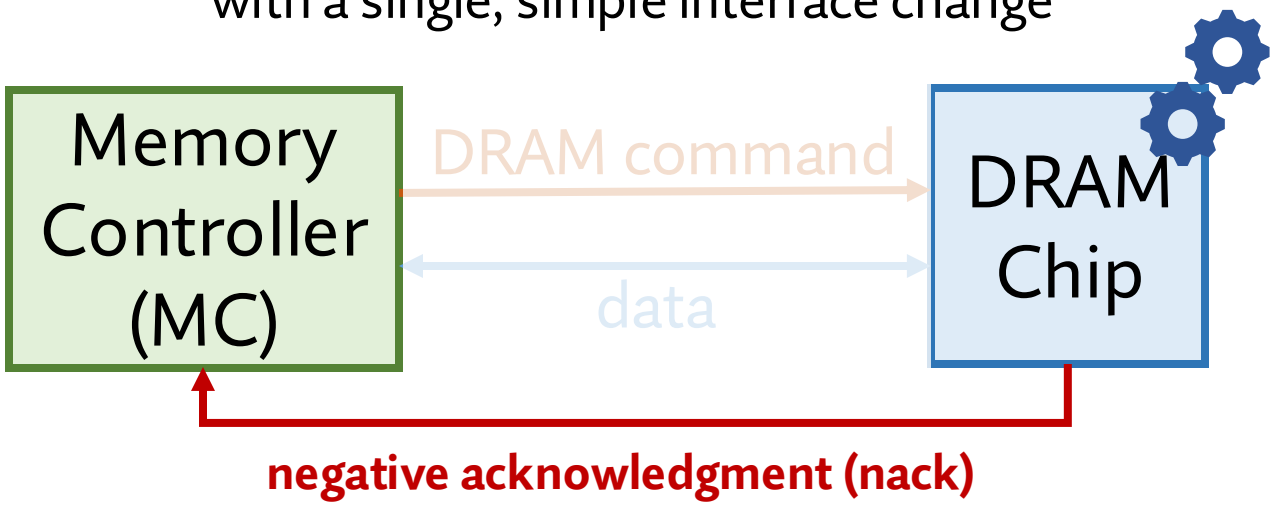
- **Problem:** Access-maintenance conflict
- **Key mechanism: Reject** access (activate) commands



SMD Key Contribution

DRAM chip controls in-DRAM maintenance operations

with a single, simple interface change



orchestrates all access operations

can now perform its own maintenance autonomously

Partition the work nicely between the memory controller and the DRAM chip

Performance and Energy Summary

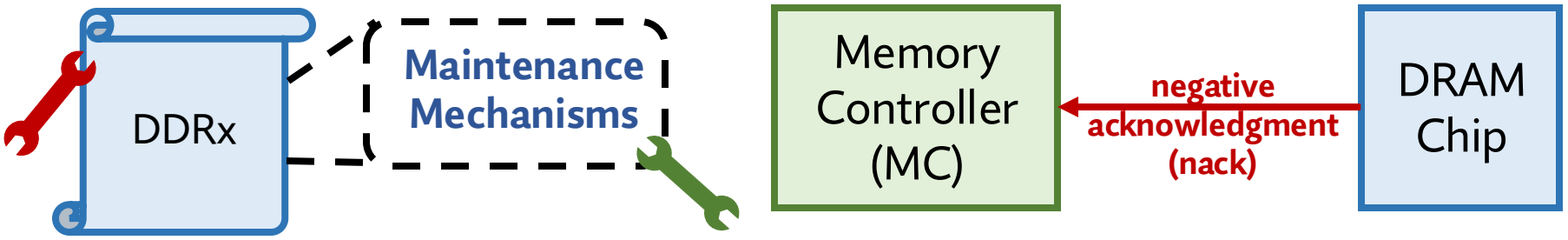
SMD provides performance and energy benefits comparable to a hypothetical system without maintenance while improving system robustness

- Benefits over the baseline system attributed to:

1. Overlapping the latency of maintenance operations with useful access operations

2. Reduced command interference and energy use: MC does not issue maintenance commands

Self-Managing DRAM Conclusion



New maintenance mechanisms require changes to DRAM standards

With a simple, single modification to the DRAM interface, SMD enables implementing new in-DRAM maintenance mechanisms with no further changes to the DRAM interface and other components

We showcase three high-performance and energy-efficient SMD-based in-DRAM maintenance mechanisms

Our Hope

SMD enables practical adoption of innovative ideas in DRAM design and inspires better ways of partitioning work between processor and DRAM

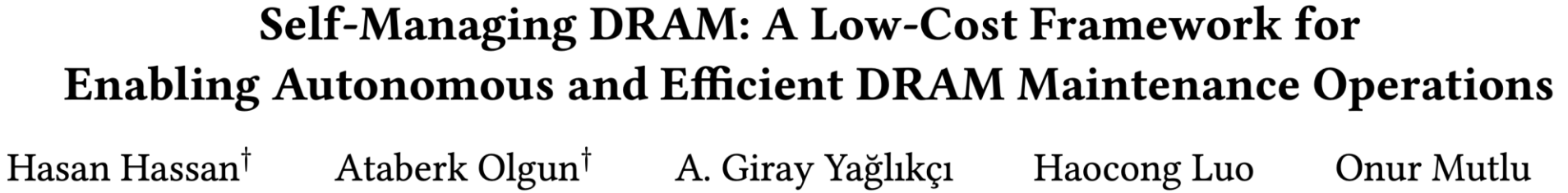
SMD: A Single Change to DRAM Interface

4. Self-Managing DRAM (SMD)

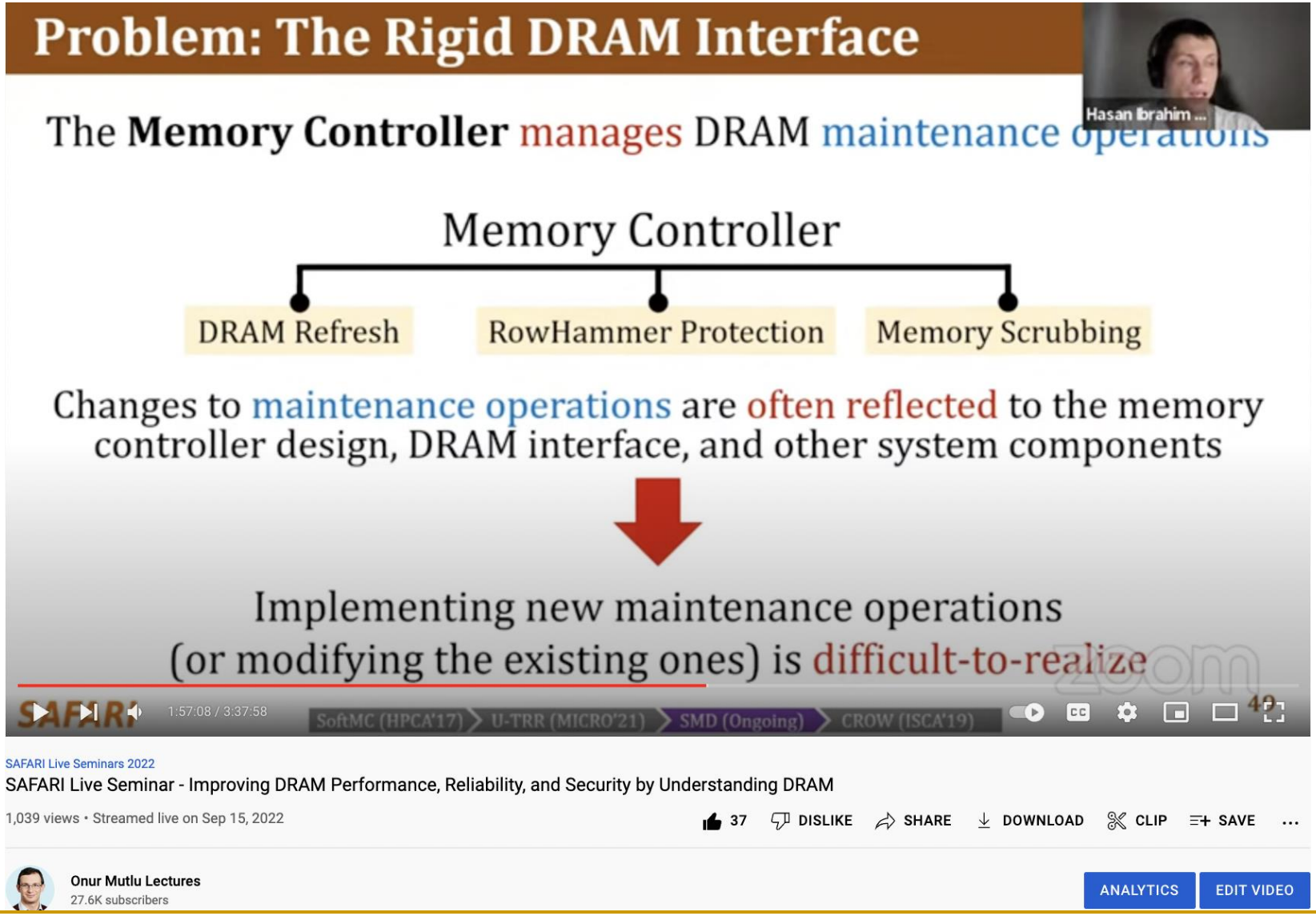
We propose the Self-Managing DRAM (SMD) framework that decouples the responsibilities of the DRAM chip and the memory controller with a single change to the DRAM interface. Giving a DRAM chip the ability (i.e., the *breathing room*) to autonomously perform maintenance operations renders DRAM interface modifications for implementing future maintenance operations unnecessary. SMD is part of a continuum between fully master-slave (completely controlled by one side) and fully request-reply (equal partner) interfaces. Today's commodity DRAM interface (e.g., DDRx [119, 140, 141], HBMx [143, 144]) is completely on the fully master-slave end of the continuum as the memory controller dictates everything the DRAM chip does. SMD, in contrast, tries to achieve a balance that enables faster innovation and easier adoption of new ideas by giving the DRAM chip some freedom to autonomously perform maintenance operations. We believe that doing so would ultimately more quickly enable better (more performant, efficient, robust) computing systems.

More on Self Managing DRAM

- Hasan Hassan, Ataberk Olgun, A. Giray Yaglikci, Haocong Luo, and Onur Mutlu,
  "Self-Managing DRAM: A Low-Cost Framework for Enabling Autonomous and Efficient DRAM Maintenance Operations"
  *Proceedings of the 57th International Symposium on Microarchitecture (**MICRO**)*, Austin, TX, USA, November 2024.



Talk on Self-Managing DRAM



https://www.youtube.com/watch?v=mGa6-vpExbE

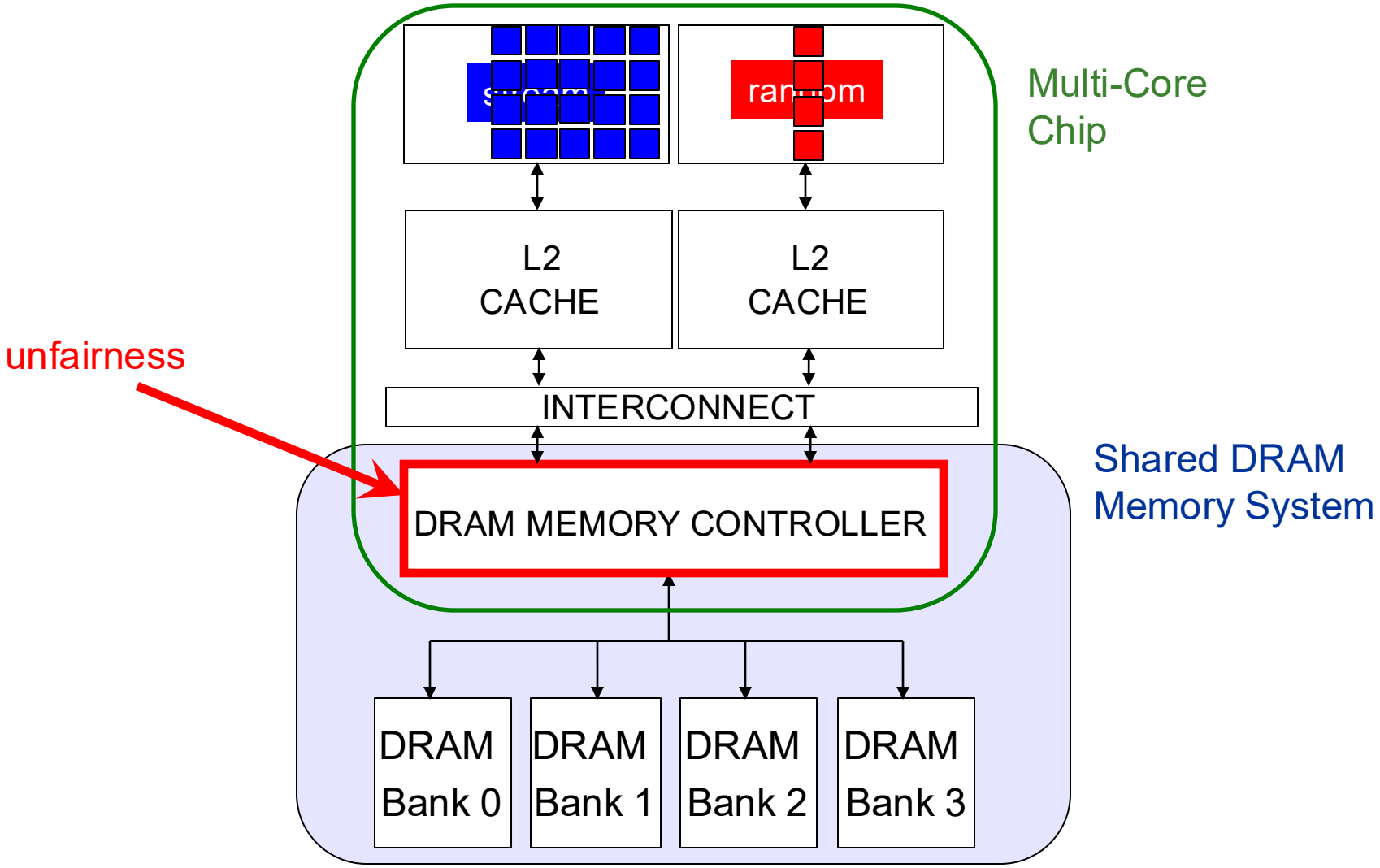
Memory Interference

Inter-Thread/Application Interference

- Problem: Threads share the memory system, but memory system does not distinguish between threads' requests

- Existing memory systems
  - Free-for-all, shared based on demand
  - Control algorithms thread-unaware and thread-unfair
  - Aggressive threads can deny service to others
  - Do not try to reduce or control inter-thread interference

Uncontrolled Interference: An Example



A Memory Performance Hog

```
// initialize large arrays A, B

for (j=0; j<N; j++) {
    index = j*linesize;   streaming
    A[index] = B[index];
    …
}
```

```
// initialize large arrays A, B

for (j=0; j<N; j++) {
    index = rand();   random
    A[index] = B[index];
    …
}
```

STREAM

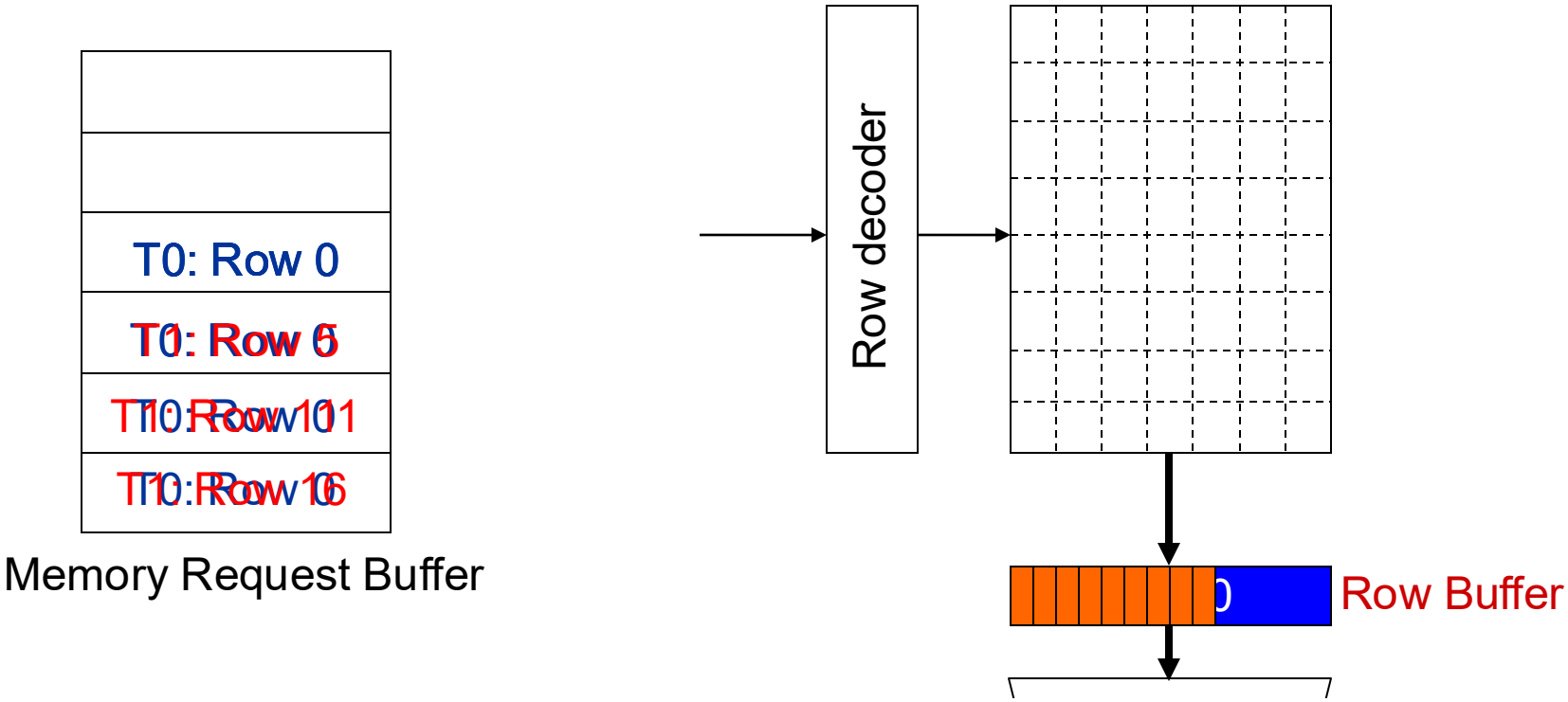
- Sequential memory access
- Very high row buffer locality (96% hit rate)
- Memory intensive

RANDOM

- Random memory access
- Very low row buffer locality (3% hit rate)
- Similarly memory intensive

Moscibroda and Mutlu, "Memory Performance Attacks," USENIX Security 2007.

What Does the Memory Hog Do?

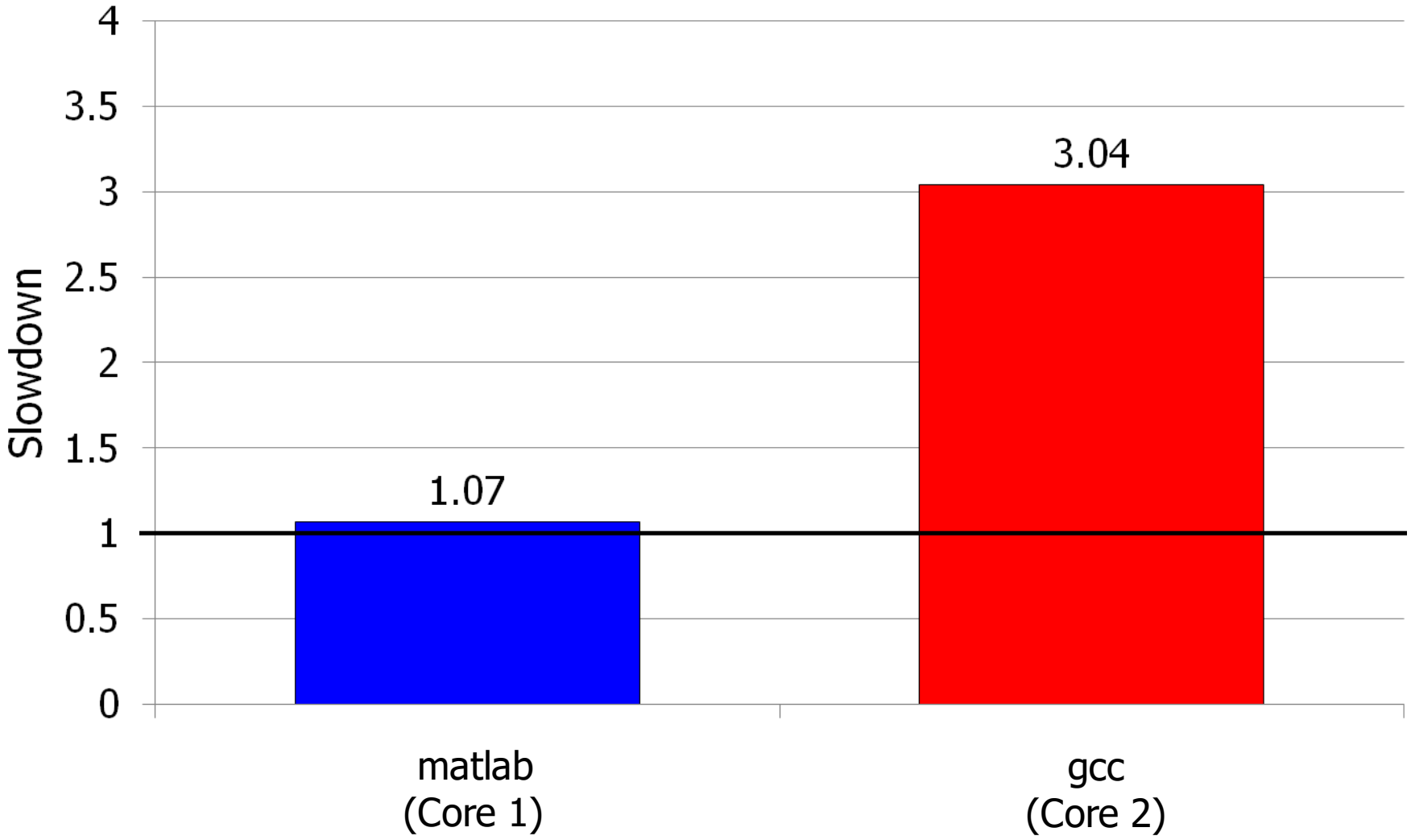


Row size: 8KB, cache block size: 64B

128 (8KB/64B) requests of T0 serviced before T1

Moscibroda and Mutlu, "Memory Performance Attacks," USENIX Security 2007.

Unfair Slowdowns due to Interference



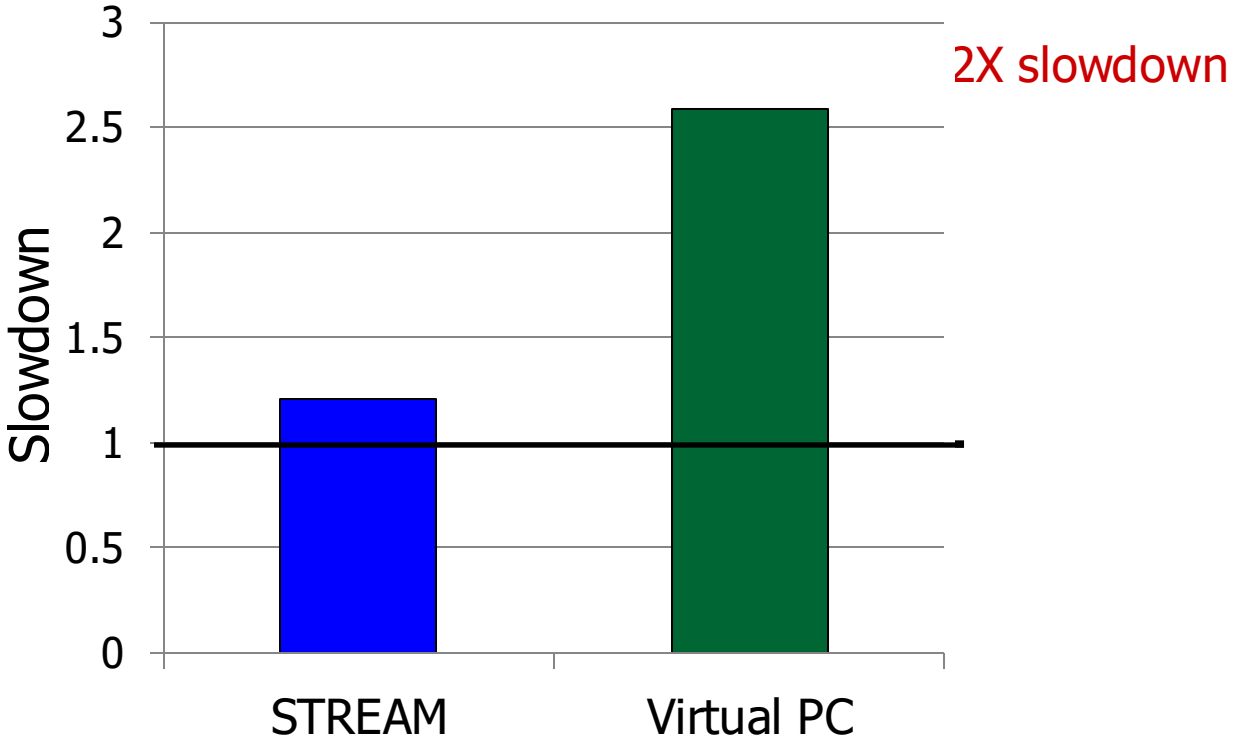
Moscibroda and Mutlu, "Memory performance attacks: Denial of memory service in multi-core systems," USENIX Security 2007.

DRAM Controllers

- A row-conflict memory access takes significantly longer than a row-hit access
- Current controllers take advantage of the row buffer
- Commonly used scheduling policy (FR-FCFS) [Rixner 2000]*
  (1) Row-hit first: Service row-hit memory accesses first
  (2) Oldest-first: Then service older accesses first
- This scheduling policy aims to maximize DRAM throughput
  - But, it is unfair when multiple threads share the DRAM system

*Rixner et al., "Memory Access Scheduling," ISCA 2000.
*Zuravleff and Robinson, "Controller for a synchronous DRAM …," US Patent 5,630,096, May 1997.

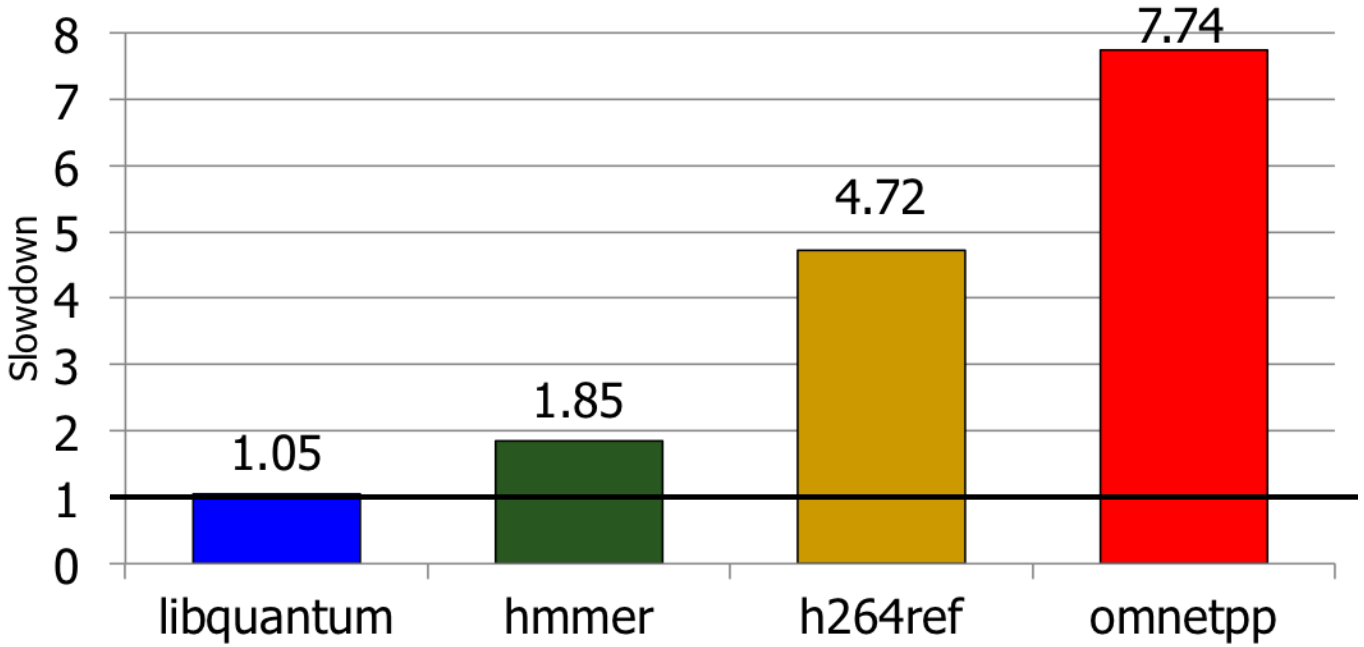
Effect of the Memory Performance Hog



Results on Intel Pentium D running Windows XP
(Similar results for Intel Core Duo and AMD Turion, and on Fedora Linux)

Moscibroda and Mutlu, "Memory Performance Attacks," USENIX Security 2007.

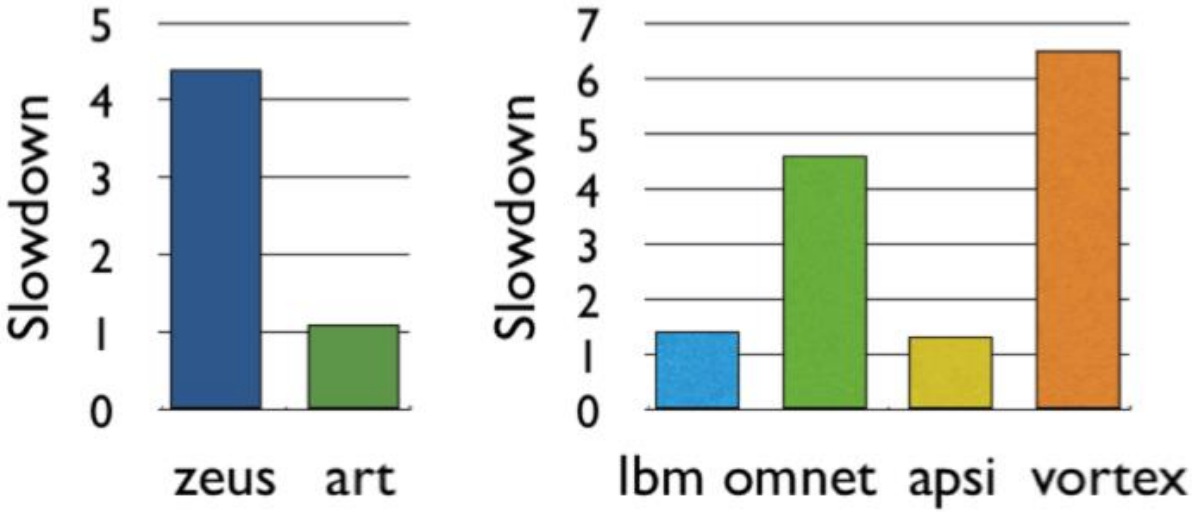
Greater Problem with More Cores



- Vulnerable to denial of service (DoS)
- Unable to enforce priorities or SLAs
- Low system performance

Uncontrollable, unpredictable system

Greater Problem with More Cores

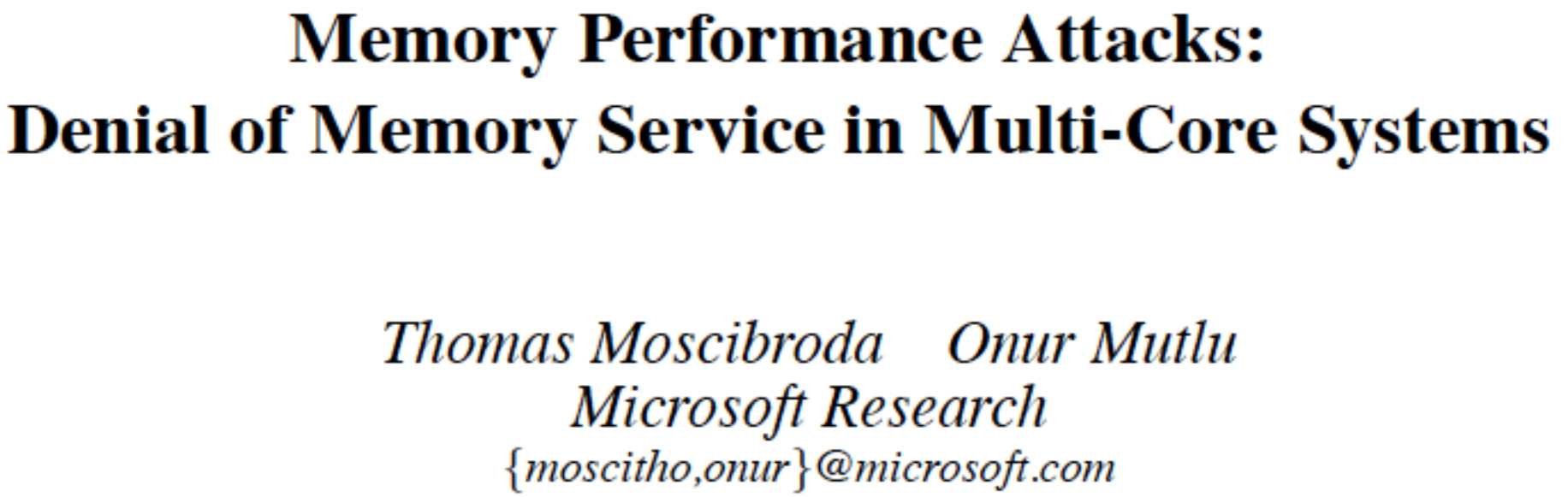


- Vulnerable to denial of service (DoS)
- Unable to enforce priorities or SLAs
- Low system performance

Uncontrollable, unpredictable system

More on Memory Performance Attacks

- Thomas Moscibroda and Onur Mutlu,
  "Memory Performance Attacks: Denial of Memory Service in Multi-Core Systems"
  *Proceedings of the 16th USENIX Security Symposium (**USENIX SECURITY**)*, pages 257-274, Boston, MA, August 2007. Slides (ppt)

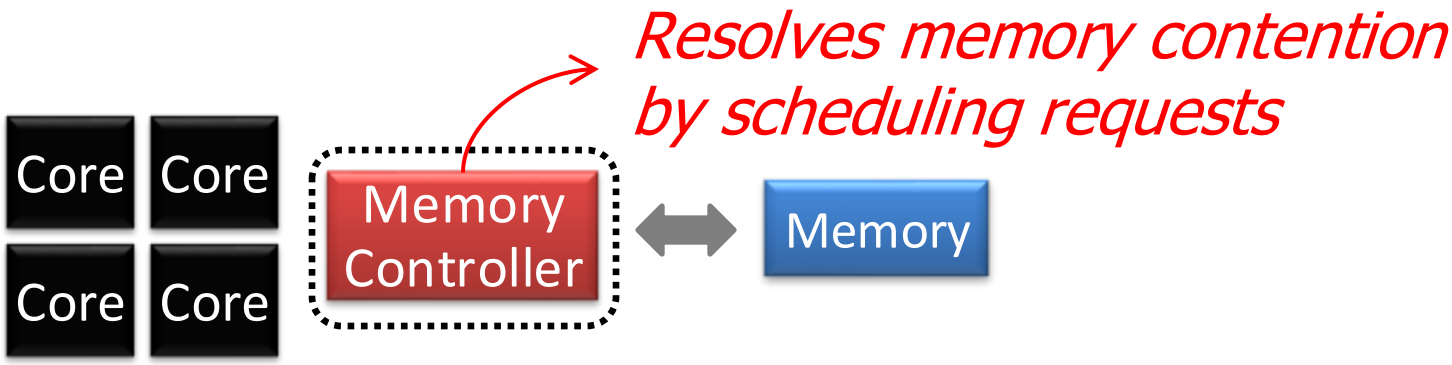


How Do We Solve The Problem?

- Inter-thread interference is uncontrolled in all memory resources
  - Memory controller
  - Interconnect
  - Caches

- We need to control it
  - i.e., design an interference-aware (QoS-aware) memory system

QoS-Aware Memory Scheduling



- How to schedule requests to provide
  - High system performance
  - High fairness to applications
  - Configurability to system software

- Memory controller needs to be aware of threads

QoS-Aware Memory: Readings (I)

- Onur Mutlu and Thomas Moscibroda,
  "Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors"
  *Proceedings of the 40th International Symposium on Microarchitecture (**MICRO**)*, pages 146-158, Chicago, IL, December 2007. [Summary] [Slides (ppt)]

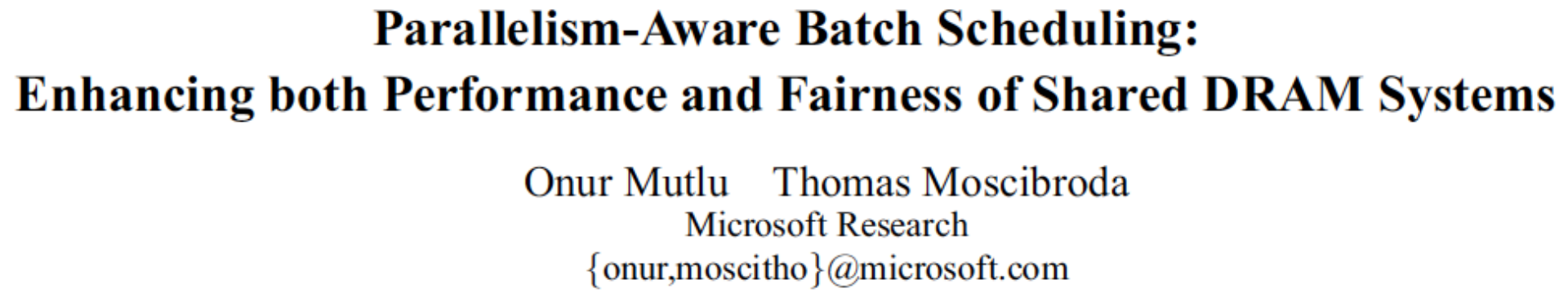
Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors

Onur Mutlu Thomas Moscibroda

Microsoft Research
{onur,moscitho}@microsoft.com

QoS-Aware Memory: Readings (II)

- Onur Mutlu and Thomas Moscibroda,
  "Parallelism-Aware Batch Scheduling: Enhancing both Performance and Fairness of Shared DRAM Systems"
  *Proceedings of the 35th International Symposium on Computer Architecture (**ISCA**)*, pages 63-74, Beijing, China, June 2008.
  [Summary] [Slides (ppt)]
  [Lecture Slides (pptx) (pdf)]
  [Lecture Video (1 hr 16 mins), 8 October 2020]
  One of the 12 computer architecture papers of 2008 selected as Top Picks by IEEE Micro.



On PAR-BS

- Variants implemented in Samsung SoC memory controllers

Effective platform level approach and DRAM accesses are crucial to system performance. This paper touches this topics and suggest a superior approach to current known techniques.

Review from ISCA 2008

QoS-Aware Memory: Readings (III)

- Yoongu Kim, Dongsu Han, Onur Mutlu, and Mor Harchol-Balter, **"ATLAS: A Scalable and High-Performance Scheduling Algorithm for Multiple Memory Controllers"**
  *Proceedings of the 16th International Symposium on High-Performance Computer Architecture (**HPCA**)*, Bangalore, India, January 2010. Slides (pptx)
  Best paper session. One of the four papers nominated for the Best Paper Award by the Program Committee.

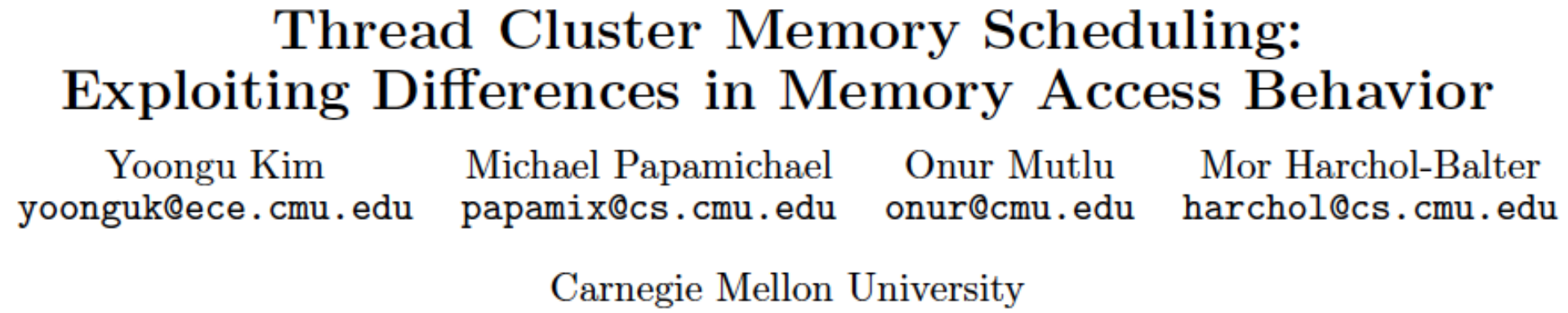
ATLAS: A Scalable and High-Performance Scheduling Algorithm for Multiple Memory Controllers

Yoongu Kim Dongsu Han Onur Mutlu Mor Harchol-Balter

Carnegie Mellon University

QoS-Aware Memory: Readings (IV)

- Yoongu Kim, Michael Papamichael, Onur Mutlu, and Mor Harchol-Balter, **"Thread Cluster Memory Scheduling: Exploiting Differences in Memory Access Behavior"**
  *Proceedings of the 43rd International Symposium on Microarchitecture (**MICRO**)*, pages 65-76, Atlanta, GA, December 2010. Slides (pptx) (pdf)
  One of the 11 computer architecture papers of 2010 selected as Top Picks by IEEE Micro.



Thread Cluster Memory Scheduling:
Exploiting Differences in Memory Access Behavior

Yoongu Kim yoonguk@ece.cmu.edu
Michael Papamichael papamix@cs.cmu.edu
Onur Mutlu onur@cmu.edu
Mor Harchol-Balter harchol@cs.cmu.edu

Carnegie Mellon University

QoS-Aware Memory: Readings (V)

- Lavanya Subramanian, Donghyuk Lee, Vivek Seshadri, Harsha Rastogi, and Onur Mutlu,
  "The Blacklisting Memory Scheduler: Achieving High Performance and Fairness at Low Cost"
  *Proceedings of the 32nd IEEE International Conference on Computer Design (**ICCD**)*, Seoul, South Korea, October 2014.
  [Slides (pptx) (pdf)]



The Blacklisting Memory Scheduler:
Achieving High Performance and Fairness at Low Cost

Lavanya Subramanian, Donghyuk Lee, Vivek Seshadri, Harsha Rastogi, Onur Mutlu
Carnegie Mellon University
{lsubrama,donghyu1,visesh,harshar,onur}@cmu.edu

QoS-Aware Memory: Readings (VI)

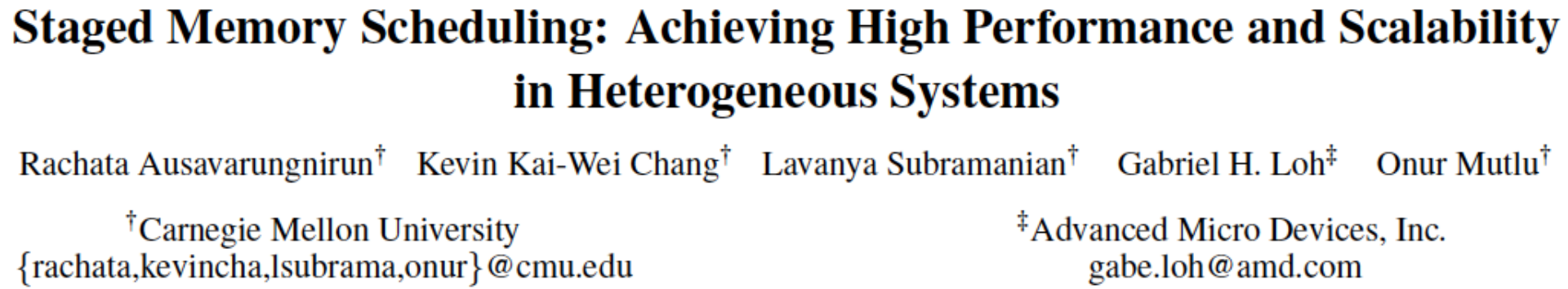
- Lavanya Subramanian, Donghyuk Lee, Vivek Seshadri, Harsha Rastogi, and Onur Mutlu,
  "BLISS: Balancing Performance, Fairness and Complexity in Memory Access Scheduling"
  IEEE Transactions on Parallel and Distributed Systems (***TPDS***), to appear in 2016. arXiv.org version, April 2015.
  An earlier version as *SAFARI Technical Report*, TR-SAFARI-2015-004, Carnegie Mellon University, March 2015.
  [Source Code]

BLISS: Balancing Performance, Fairness and Complexity in Memory Access Scheduling

Lavanya Subramanian, Donghyuk Lee, Vivek Seshadri, Harsha Rastogi, and Onur Mutlu

QoS-Aware Memory: Readings (VII)

- Rachata Ausavarungnirun, Kevin Chang, Lavanya Subramanian, Gabriel Loh, and Onur Mutlu,
  "Staged Memory Scheduling: Achieving High Performance and Scalability in Heterogeneous Systems"
  *Proceedings of the 39th International Symposium on Computer Architecture (**ISCA**)*, Portland, OR, June 2012. Slides (pptx)



Staged Memory Scheduling: Achieving High Performance and Scalability in Heterogeneous Systems

Rachata Ausavarungnirun[†] Kevin Kai-Wei Chang[†] Lavanya Subramanian[†] Gabriel H. Loh[‡] Onur Mutlu[†]

[†]Carnegie Mellon University
{rachata,kevincha,lsubrama,onur}@cmu.edu

[‡]Advanced Micro Devices, Inc.
gabe.loh@amd.com

QoS-Aware Memory: Readings (VIII)

- Hiroyuki Usui, Lavanya Subramanian, Kevin Kai-Wei Chang, and Onur Mutlu,
  "DASH: Deadline-Aware High-Performance Memory Scheduler for Heterogeneous Systems with Hardware Accelerators"
  ACM Transactions on Architecture and Code Optimization (***TACO***),
  Vol. 12, January 2016.
  Presented at the 11th HiPEAC Conference, Prague, Czech Republic, January 2016.
  [Slides (pptx) (pdf)]
  [Source Code]

DASH: Deadline-Aware High-Performance Memory Scheduler for Heterogeneous Systems with Hardware Accelerators

HIROYUKI USUI, LAVANYA SUBRAMANIAN, KEVIN KAI-WEI CHANG, and ONUR MUTLU, Carnegie Mellon University

QoS-Aware Memory: Readings (IX)

- Lavanya Subramanian, Vivek Seshadri, Yoongu Kim, Ben Jaiyen, and Onur Mutlu,
  "MISE: Providing Performance Predictability and Improving Fairness in Shared Main Memory Systems"
  *Proceedings of the 19th International Symposium on High-Performance Computer Architecture (**HPCA**)*, Shenzhen, China, February 2013. Slides (pptx)

MISE: Providing Performance Predictability and Improving Fairness in Shared Main Memory Systems

Lavanya Subramanian Vivek Seshadri Yoongu Kim Ben Jaiyen Onur Mutlu
Carnegie Mellon University

QoS-Aware Memory: Readings (X)

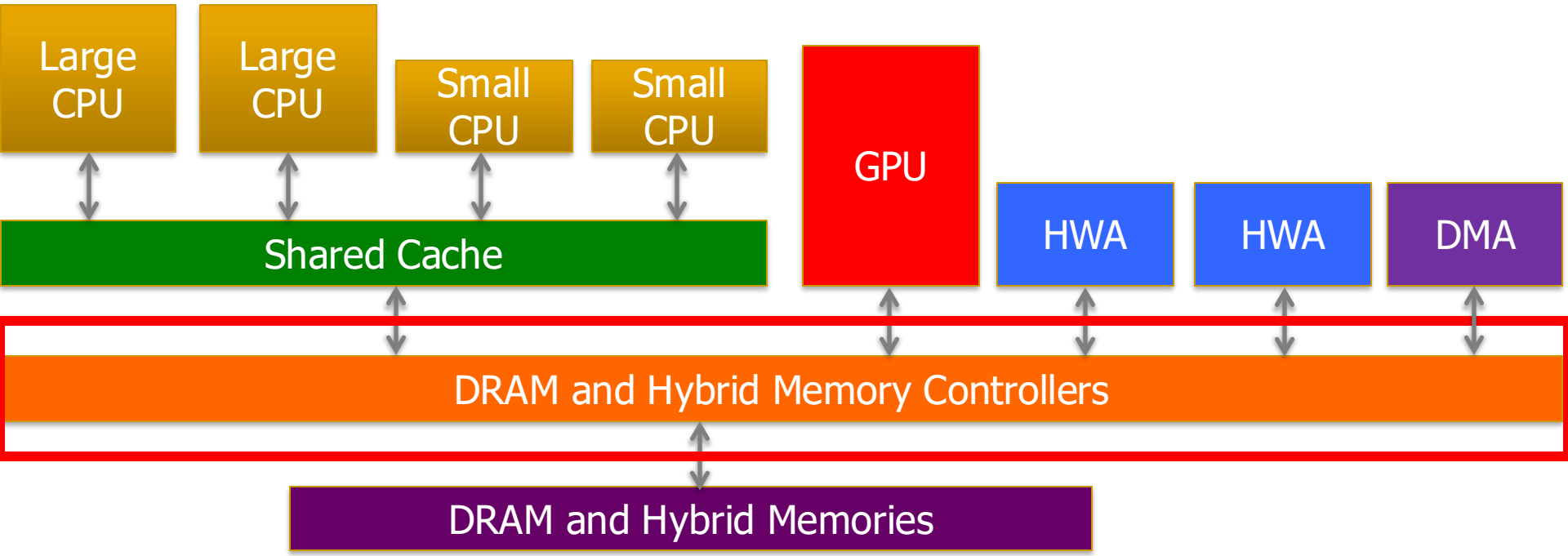
- Lavanya Subramanian, Vivek Seshadri, Arnab Ghosh, Samira Khan, and Onur Mutlu,
  "The Application Slowdown Model: Quantifying and Controlling the Impact of Inter-Application Interference at Shared Caches and Main Memory"
  *Proceedings of the 48th International Symposium on Microarchitecture (**MICRO**)*, Waikiki, Hawaii, USA, December 2015.
  [Slides (pptx) (pdf)] [Lightning Session Slides (pptx) (pdf)] [Poster (pptx) (pdf)]
  [Source Code]

The Application Slowdown Model: Quantifying and Controlling the Impact of Inter-Application Interference at Shared Caches and Main Memory

Lavanya Subramanian*§ Vivek Seshadri* Arnab Ghosh*†
Samira Khan*‡ Onur Mutlu*

*Carnegie Mellon University §Intel Labs †IIT Kanpur ‡University of Virginia

Memory Control is Getting More Complex



- Heterogeneous agents: CPUs, GPUs, HWAs, DMA engine, …
- Main memory interference between CPUs, GPUs, HWAs, DMA …

Many goals, many constraints, many metrics …

The Future

Memory Controllers are critical to research

They will become even more important

Computer Architecture

Lecture 10a: Memory Controllers II

Dr. Mohammad Sadrosadati
Prof. Onur Mutlu

ETH Zürich

Fall 2025

24 October 2025

Backup Slides

Data-Aware Architectures

Corollaries: Architectures Today …

- Architectures are terrible at dealing with data
  - Designed to mainly store and move data vs. to compute
  - They are processor-centric as opposed to **data-centric**

- Architectures are terrible at taking advantage of vast amounts of data (and metadata) available to them
  - Designed to make simple decisions, ignoring lots of data
  - They make human-driven decisions vs. **data-driven** decisions

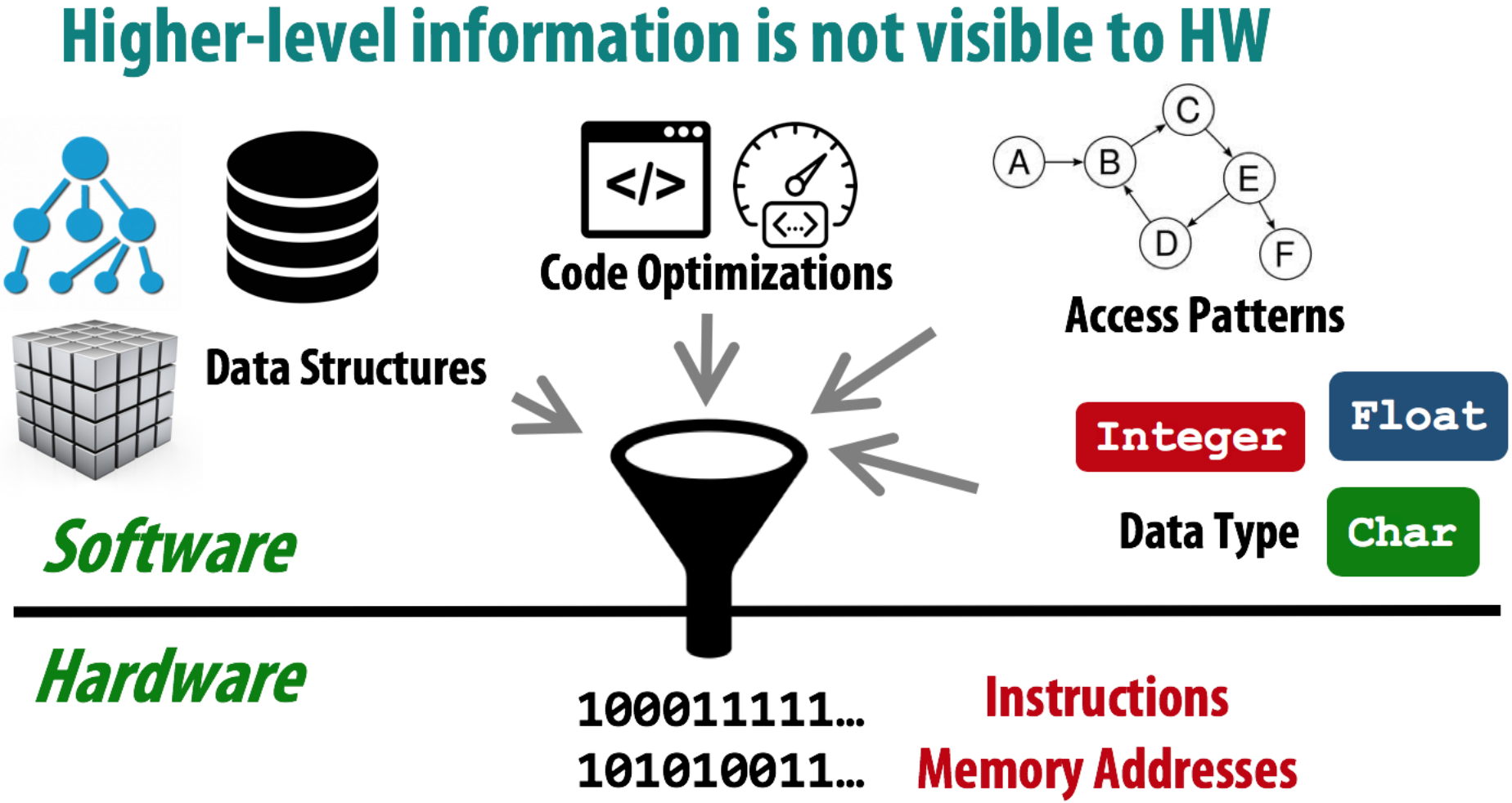
- Architectures are terrible at knowing and exploiting different properties of application data
  - Designed to treat all data as the same
  - They make component-aware decisions vs. **data-aware**

Data-Aware Architectures

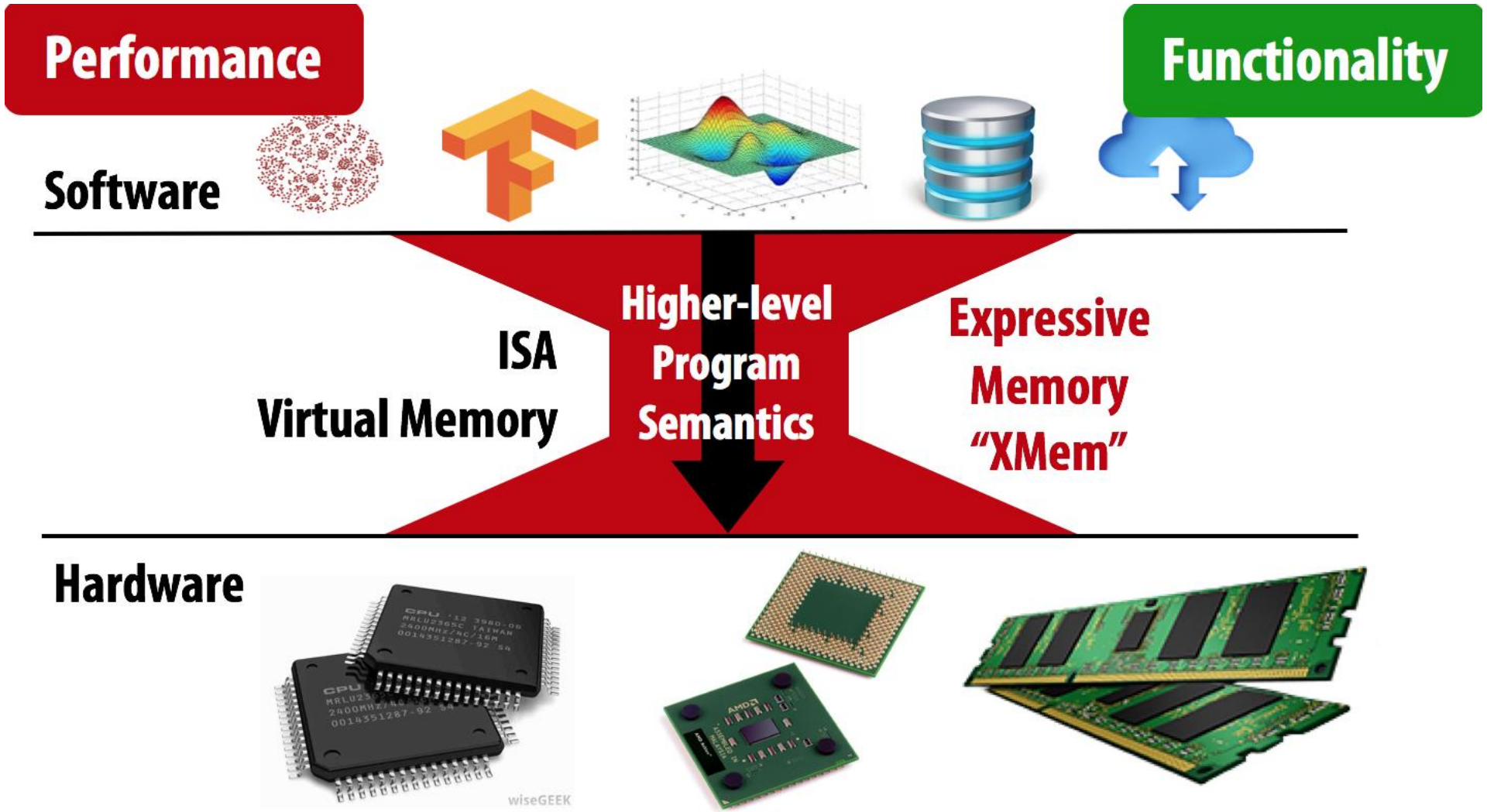
- A data-aware architecture understands what it can do with and to each piece of data

- It makes use of different properties of data to improve performance, efficiency and other metrics
  - Compressibility
  - Approximability
  - Locality
  - Sparsity
  - Criticality for Computation X
  - Access Semantics
  - …

One Problem: Limited Expressiveness



A Solution: More Expressive Interfaces



Expressive (Memory) Interfaces

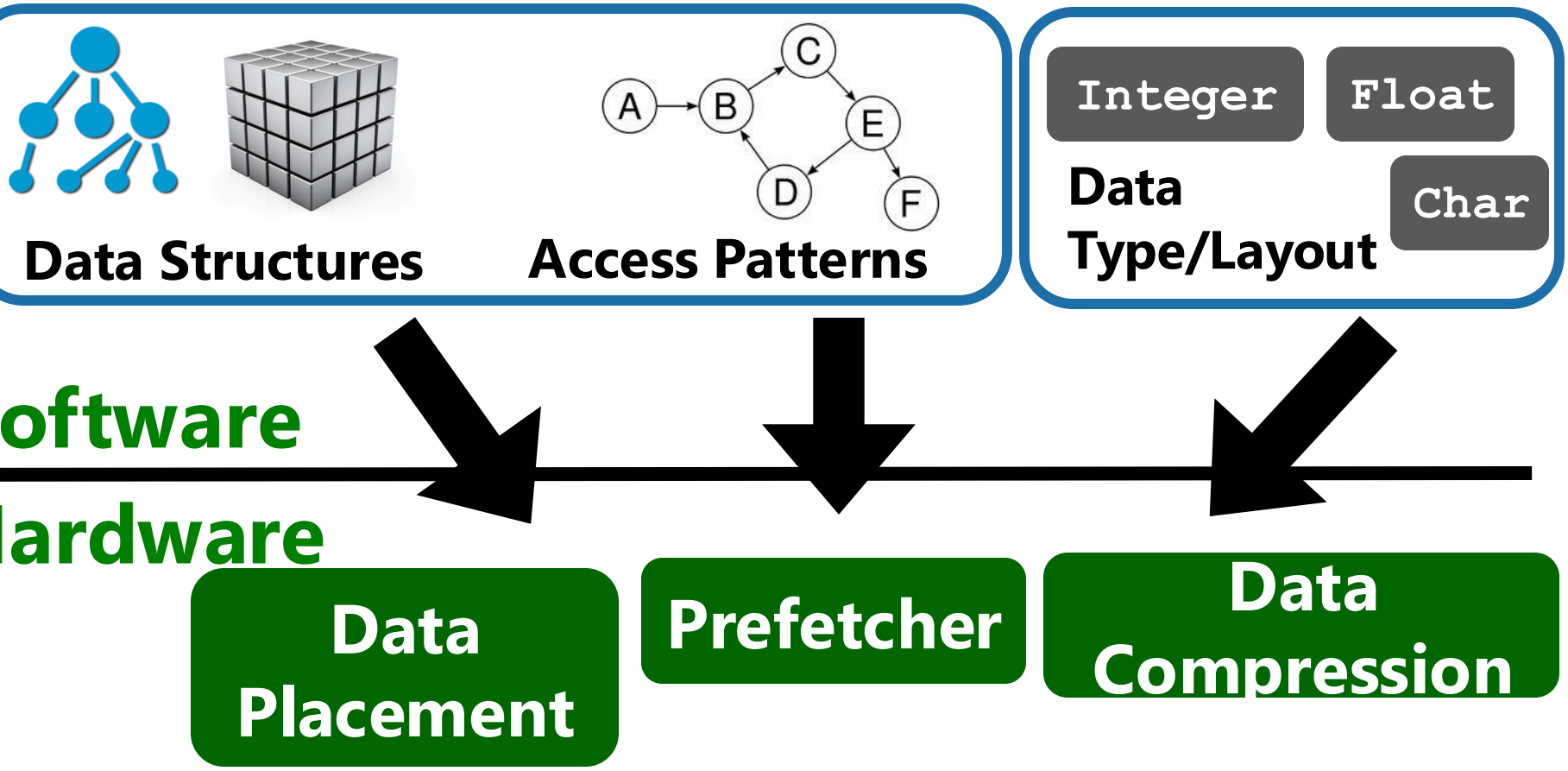
- Nandita Vijaykumar, Abhilasha Jain, Diptesh Majumdar, Kevin Hsieh, Gennady Pekhimenko, Eiman Ebrahimi, Nastaran Hajinazar, Phillip B. Gibbons and Onur Mutlu, **"A Case for Richer Cross-layer Abstractions: Bridging the Semantic Gap with Expressive Memory"**
  Proceedings of the 45th International Symposium on Computer Architecture (***ISCA***), Los Angeles, CA, USA, June 2018.
  [Slides (pptx) (pdf)] [Lightning Talk Slides (pptx) (pdf)]
  [Lightning Talk Video]

A Case for Richer Cross-layer Abstractions: Bridging the Semantic Gap with Expressive Memory

Nandita Vijaykumar[†§] Abhilasha Jain[†] Diptesh Majumdar[†] Kevin Hsieh[†] Gennady Pekhimenko[‡]
Eiman Ebrahimi[⁂] Nastaran Hajinazar[┼] Phillip B. Gibbons[†] Onur Mutlu[§†]

[†]Carnegie Mellon University [‡]University of Toronto [⁂]NVIDIA
[┼]Simon Fraser University [§]ETH Zürich

SW provides key program information to HW



Broader goal: Enable many cross-layer optimizations

Express:

Data structures

Access semantics

Data types

Working set

Reuse

Access frequency

…

Optimizations:

Cache Management

Data Placement in DRAM

Data Compression

Approximation

DRAM Cache Management

NVM Management

NUCA/NUMA Optimizations

…

Benefits:

More efficient HW:

✓Performance

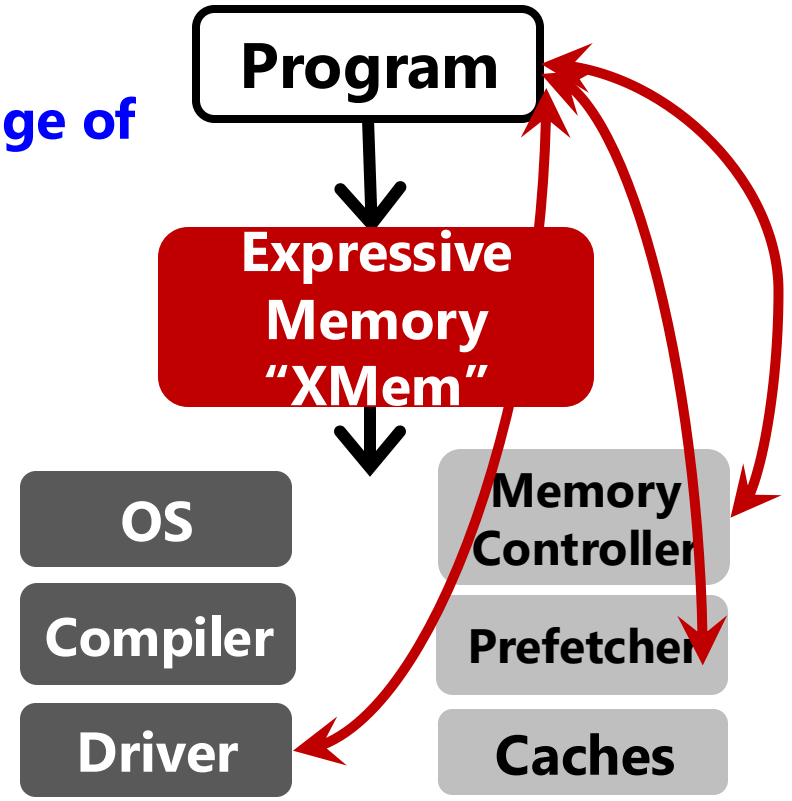
Reduced SW burden:

✓Programmability

✓Portability

Our approach: Rich cross-layer abstractions

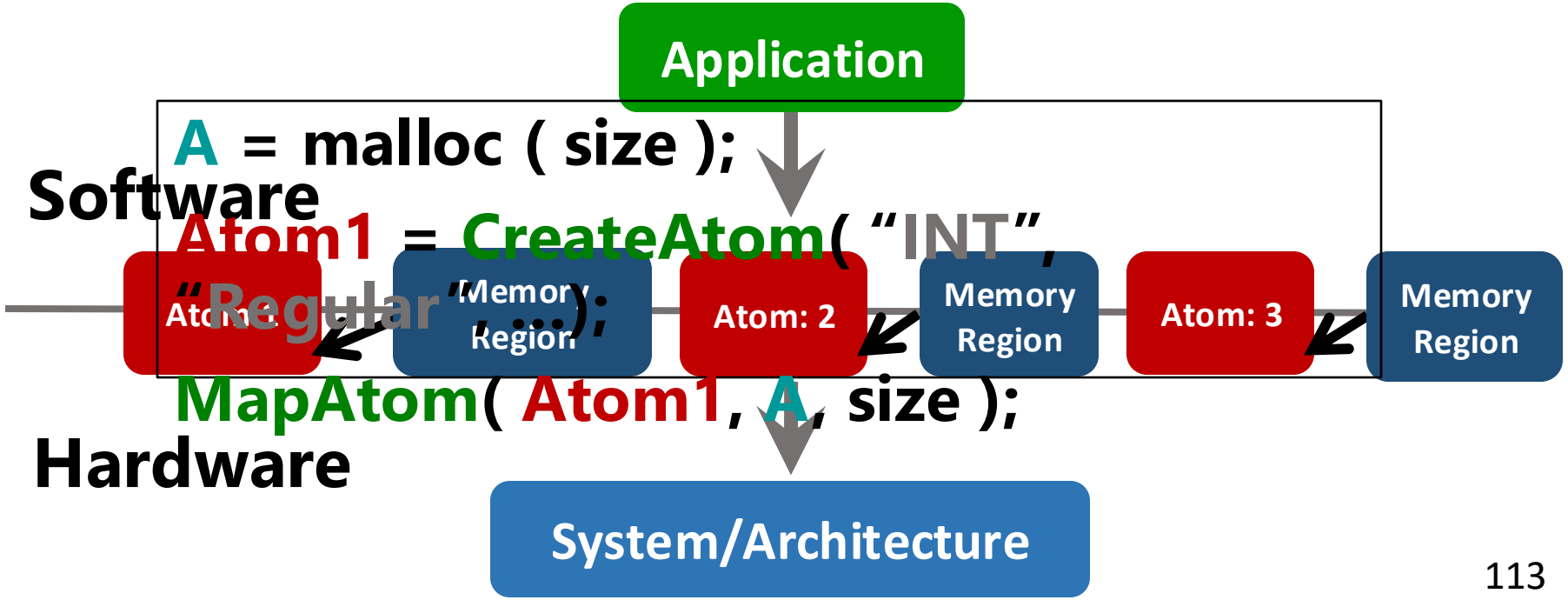
1. **Generality: Enable a wide range of cross-layer approaches**
2. **Minimize programmer effort**
3. **Overhead**



<u>Approach</u>: Flexibly associate specific semantic information with any data & code

Example: XMem

- Goal: convey data semantics to the hardware enables more intelligent management of resources.
- XMem: introduces a new HW/SW abstraction, called *Atom,* for conveying data semantics



Vijaykumar+, "A Case for Richer Cross-layer Abstractions: Bridging the Semantic Gap with Expressive Memory", ISCA 2018

XMem Aids/Enables Many Optimizations

Table 1: Summary of the example memory optimizations that XMem aids.

Memory optimization	Example semantics provided by XMem (described in §3.3)	Example Benefits of XMem
Cache management	*(i)* Distinguishing between data structures or pools of similar data; *(ii)* Working set size; *(iii)* Data reuse	Enables: *(i)* applying different caching policies to different data structures or pools of data; *(ii)* avoiding cache thrashing by *knowing* the active working set size; *(iii)* bypassing/prioritizing data that has no/high reuse. (§5)
Page placement in DRAM e.g., [23, 24]	*(i)* Distinguishing between data structures; *(ii)* Access pattern; *(iii)* Access intensity	Enables page placement at the *data structure* granularity to *(i)* isolate data structures that have high row buffer locality and *(ii)* spread out concurrently-accessed irregular data structures across banks and channels to improve parallelism. (§6)
Cache/memory compression e.g., [25–32]	*(i)* Data type: integer, float, char; *(ii)* Data properties: sparse, pointer, data index	Enables using a *different compression algorithm* for each data structure based on data type and data properties, e.g., sparse data encodings, FP-specific compression, delta-based compression for pointers [27].
Data prefetching e.g., [33–36]	*(i)* Access pattern: strided, irregular, irregular but repeated (e.g., graphs), access stride; *(ii)* Data type: index, pointer	Enables *(i) highly accurate* software-driven prefetching while leveraging the benefits of hardware prefetching (e.g., by being memory bandwidth-aware, avoiding cache thrashing); *(ii)* using different prefetcher *types* for different data structures: e.g., stride [33], tile-based [20], pattern-based [34–37], data-based for indices/pointers [38, 39], etc.
DRAM cache management e.g., [40–46]	*(i)* Access intensity; *(ii)* Data reuse; *(iii)* Working set size	*(i)* Helps avoid cache thrashing by knowing working set size [44]; *(ii)* Better DRAM cache management via reuse behavior and access intensity information.
Approximation in memory e.g., [47–53]	*(i)* Distinguishing between pools of similar data; *(ii)* Data properties: tolerance towards approximation	Enables *(i)* each memory component to track how approximable data is (at a fine granularity) to inform approximation techniques; *(ii)* data placement in heterogeneous reliability memories [54].
Data placement: NUMA systems e.g., [55, 56]	*(i)* Data partitioning across threads (i.e., relating data to threads that access it); *(ii)* Read-Write properties	Reduces the need for profiling or data migration *(i)* to co-locate data with threads that access it and *(ii)* to identify Read-Only data, thereby enabling techniques such as replication.
Data placement: hybrid memories e.g., [16, 57, 58]	*(i)* Read-Write properties (Read-Only/Read-Write); *(ii)* Access intensity; *(iii)* Data structure size; *(iv)* Access pattern	Avoids the need for profiling/migration of data in hybrid memories to *(i)* effectively manage the asymmetric read-write properties in NVM (e.g., placing Read-Only data in the NVM) [16, 57]; *(ii)* make tradeoffs between data structure "hotness" and size to allocate fast/high bandwidth memory [14]; and *(iii)* leverage row-buffer locality in placement based on access pattern [45].
Managing NUCA systems e.g., [15, 59]	*(i)* Distinguishing pools of similar data; *(ii)* Access intensity; *(iii)* Read-Write or Private-Shared properties	*(i)* Enables using different cache policies for different data pools (similar to [15]); *(ii)* Reduces the need for reactive mechanisms that detect sharing and read-write characteristics to inform cache policies.

Expressive (Memory) Interfaces

- Nandita Vijaykumar, Abhilasha Jain, Diptesh Majumdar, Kevin Hsieh, Gennady Pekhimenko, Eiman Ebrahimi, Nastaran Hajinazar, Phillip B. Gibbons and Onur Mutlu, **"A Case for Richer Cross-layer Abstractions: Bridging the Semantic Gap with Expressive Memory"**
  Proceedings of the 45th International Symposium on Computer Architecture (***ISCA***), Los Angeles, CA, USA, June 2018.
  [Slides (pptx) (pdf)] [Lightning Talk Slides (pptx) (pdf)]
  [Lightning Talk Video]

A Case for Richer Cross-layer Abstractions: Bridging the Semantic Gap with Expressive Memory

Nandita Vijaykumar[†§] Abhilasha Jain[†] Diptesh Majumdar[†] Kevin Hsieh[†] Gennady Pekhimenko[‡]
Eiman Ebrahimi[⁂] Nastaran Hajinazar[┼] Phillip B. Gibbons[†] Onur Mutlu[§†]

[†]Carnegie Mellon University [‡]University of Toronto [⁂]NVIDIA
[┼]Simon Fraser University [§]ETH Zürich

Expressive (Memory) Interfaces for GPUs

- Nandita Vijaykumar, Eiman Ebrahimi, Kevin Hsieh, Phillip B. Gibbons and Onur Mutlu, **"The Locality Descriptor: A Holistic Cross-Layer Abstraction to Express Data Locality in GPUs"**
  Proceedings of the 45th International Symposium on Computer Architecture (***ISCA***), Los Angeles, CA, USA, June 2018.
  [Slides (pptx) (pdf)] [Lightning Talk Slides (pptx) (pdf)]
  [Lightning Talk Video]

The Locality Descriptor:

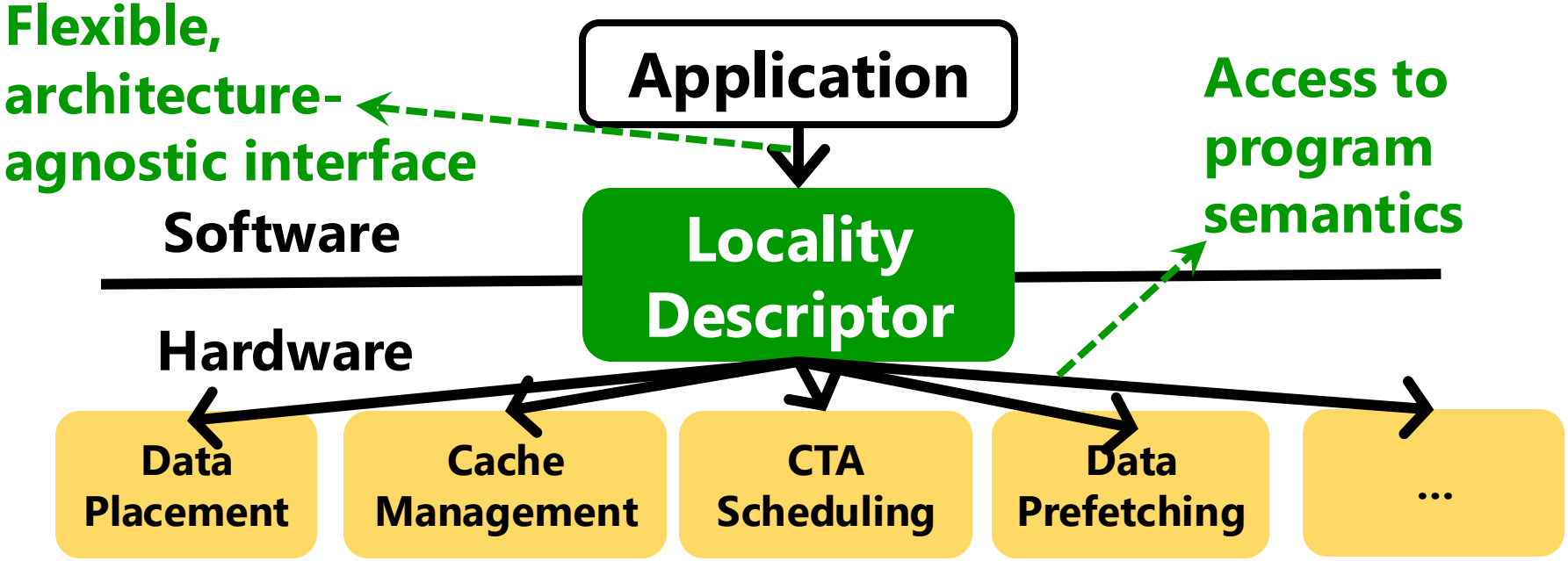
A Holistic Cross-Layer Abstraction to Express Data Locality in GPUs

Nandita Vijaykumar[†§] Eiman Ebrahimi[‡] Kevin Hsieh[†]
Phillip B. Gibbons[†] Onur Mutlu[§†]

[†]Carnegie Mellon University [‡]NVIDIA [§]ETH Zürich

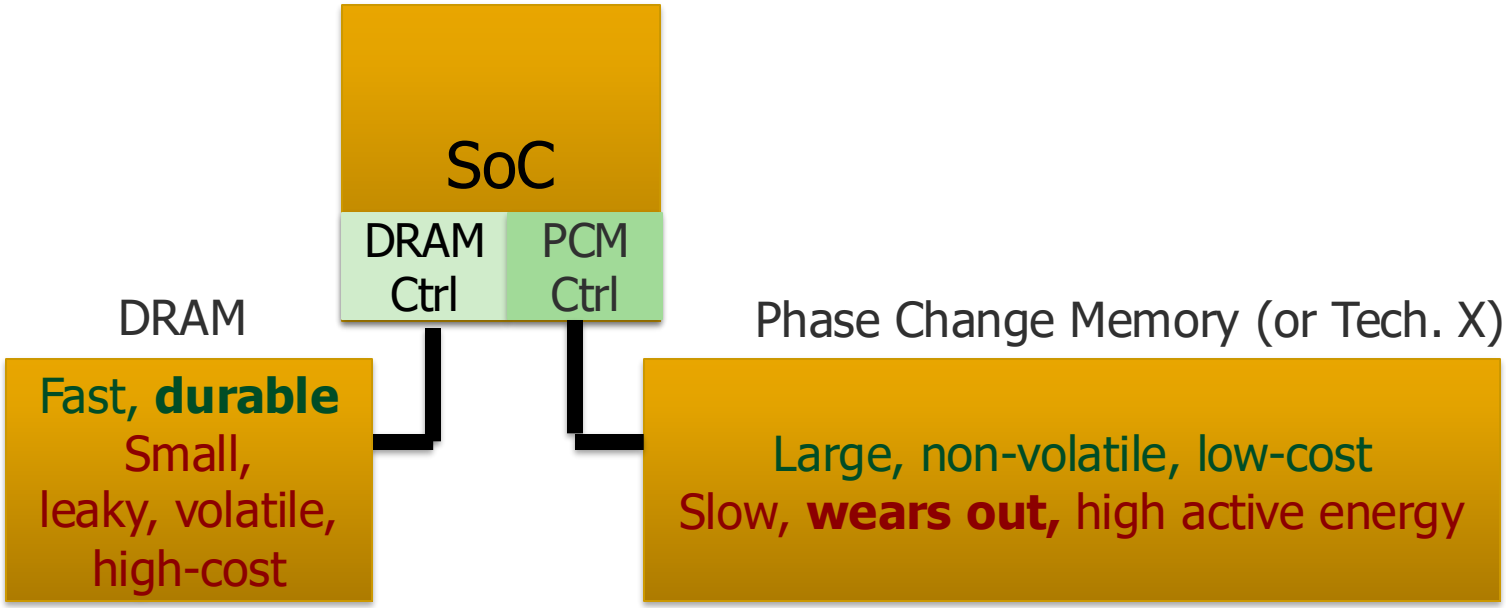
Locality Descriptor: Executive Summary

Exploiting data locality in GPUs is a challenging task



Performance Benefits:
26.6% (up to 46.6%) from cache locality
53.7% (up to 2.8x) from NUMA locality

An Example: Hybrid Memory Management



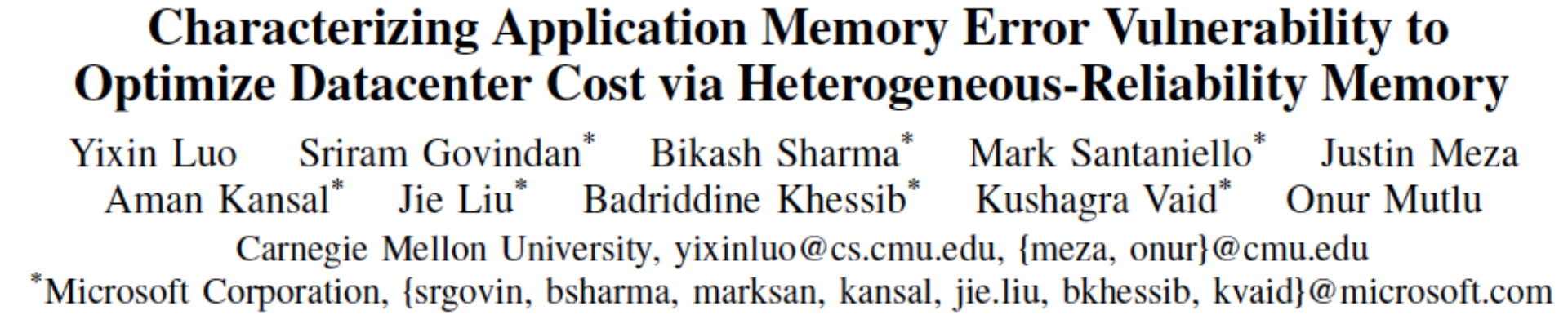
Hardware/software manage data allocation and movement
to achieve the best of multiple technologies

Meza+, "Enabling Efficient and Scalable Hybrid Memories," IEEE Comp. Arch. Letters, 2012.

Yoon+, "Row Buffer Locality Aware Caching Policies for Hybrid Memories," ICCD 2012 Best Paper Award.

An Example: Heterogeneous-Reliability Memory

- Yixin Luo, Sriram Govindan, Bikash Sharma, Mark Santaniello, Justin Meza, Aman Kansal, Jie Liu, Badriddine Khessib, Kushagra Vaid, and Onur Mutlu, **"Characterizing Application Memory Error Vulnerability to Optimize Data Center Cost via Heterogeneous-Reliability Memory"** *Proceedings of the 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (**DSN**)*, Atlanta, GA, June 2014. [Summary] [Slides (pptx) (pdf)] [Coverage on ZDNet]

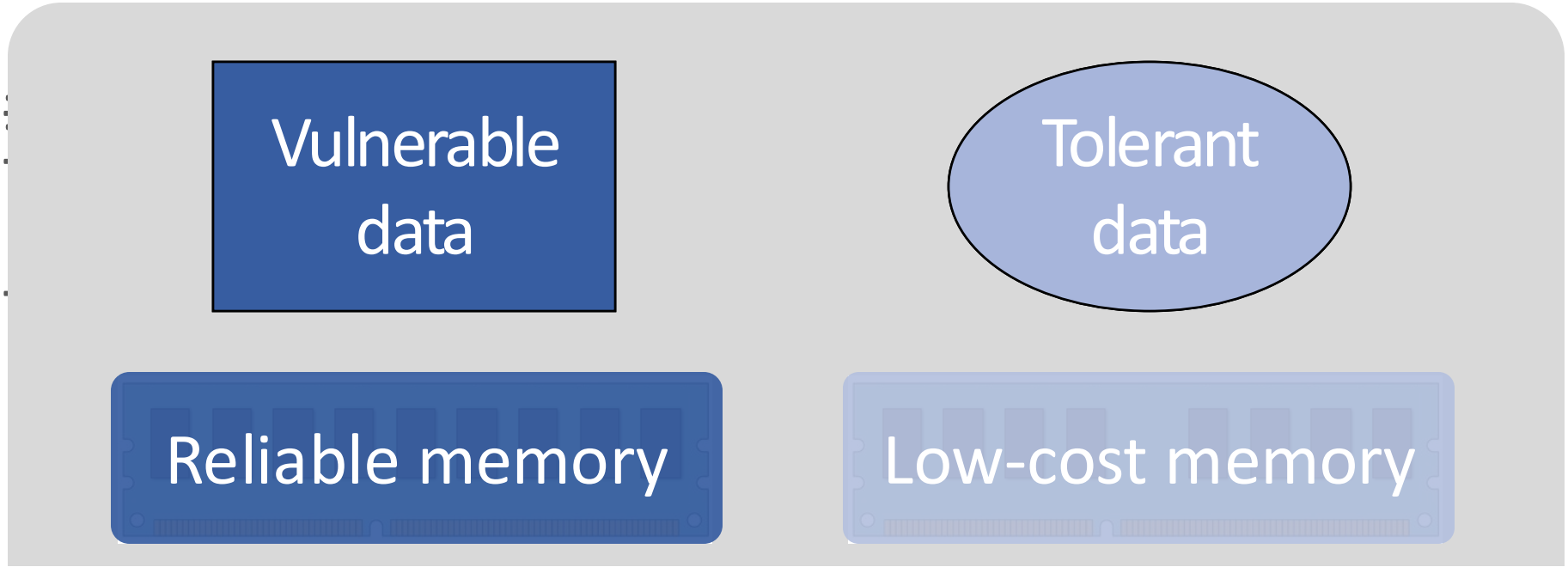


Characterizing Application Memory Error Vulnerability to Optimize Datacenter Cost via Heterogeneous-Reliability Memory

Yixin Luo Sriram Govindan* Bikash Sharma* Mark Santaniello* Justin Meza
Aman Kansal* Jie Liu* Badriddine Khessib* Kushagra Vaid* Onur Mutlu

Carnegie Mellon University, yixinluo@cs.cmu.edu, {meza, onur}@cmu.edu
*Microsoft Corporation, {srgovin, bsharma, marksan, kansal, jie.liu, bkhessib, kvaid}@microsoft.com

Exploiting Memory Error Tolerance with Hybrid Memory Systems



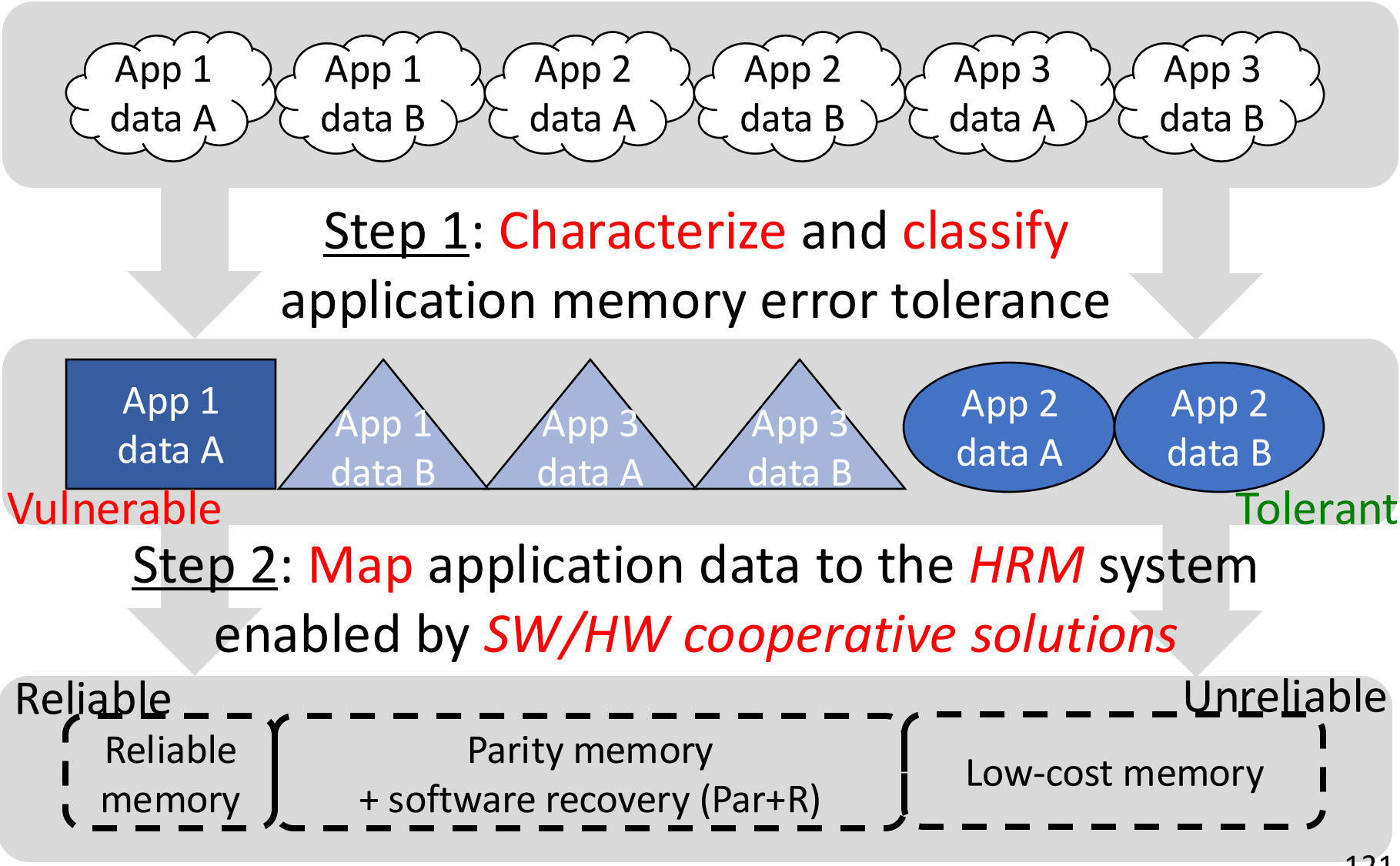
On Microsoft's Web Search workload

Reduces server hardware cost by 4.7 %

Achieves single server availability target of 99.90 %

Heterogeneous-**R**eliability **M**emory [DSN 2014]

Heterogeneous-Reliability Memory



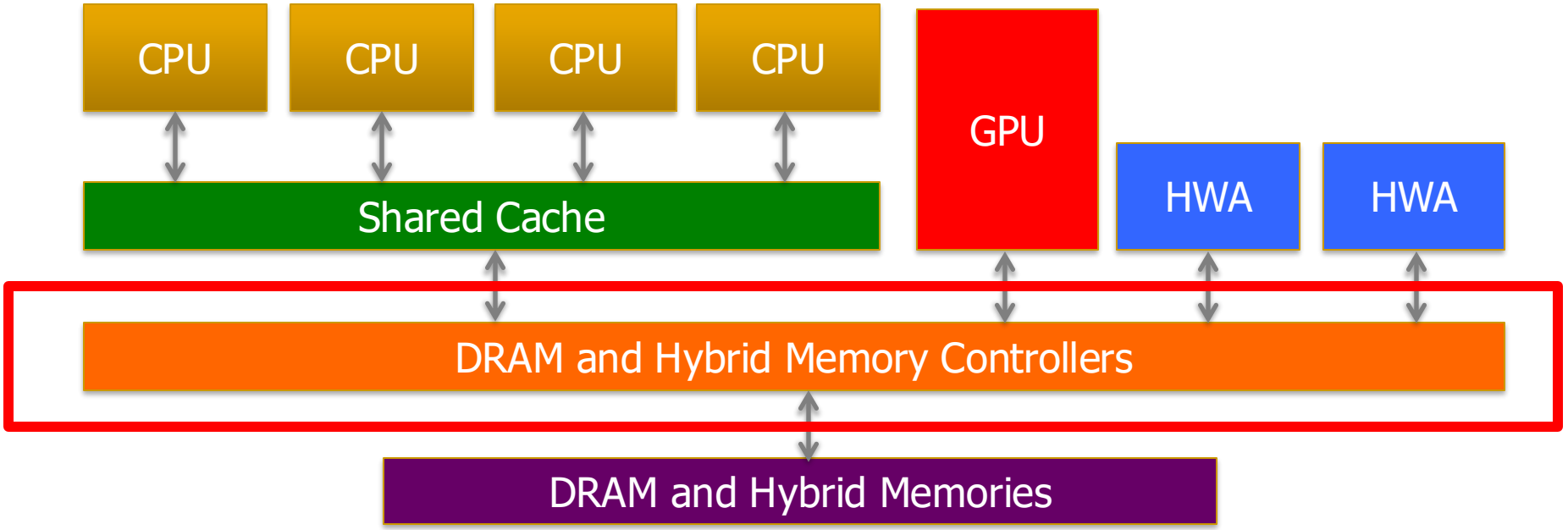
More on Heterogeneous-Reliability Memory

- Yixin Luo, Sriram Govindan, Bikash Sharma, Mark Santaniello, Justin Meza, Aman Kansal, Jie Liu, Badriddine Khessib, Kushagra Vaid, and Onur Mutlu, **"Characterizing Application Memory Error Vulnerability to Optimize Data Center Cost via Heterogeneous-Reliability Memory"** *Proceedings of the 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (**DSN**)*, Atlanta, GA, June 2014. [Summary] [Slides (pptx) (pdf)] [Coverage on ZDNet]

Characterizing Application Memory Error Vulnerability to Optimize Datacenter Cost via Heterogeneous-Reliability Memory

Yixin Luo Sriram Govindan* Bikash Sharma* Mark Santaniello* Justin Meza
Aman Kansal* Jie Liu* Badriddine Khessib* Kushagra Vaid* Onur Mutlu
Carnegie Mellon University, yixinluo@cs.cmu.edu, {meza, onur}@cmu.edu
*Microsoft Corporation, {srgovin, bsharma, marksan, kansal, jie.liu, bkhessib, kvaid}@microsoft.com

Data-Aware Cross-Layer Hybrid System Management



- Heterogeneous agents: CPUs, GPUs, and HWAs
- Main memory interference between CPUs, GPUs, HWAs
- Many timing constraints for various memory types
- Many goals at the same time: performance, fairness, QoS, energy efficiency, …

Another Example: EDEN for DNNs

- Deep Neural Network evaluation is very DRAM-intensive (especially for large networks)

1. Some data and layers in DNNs are very tolerant to errors

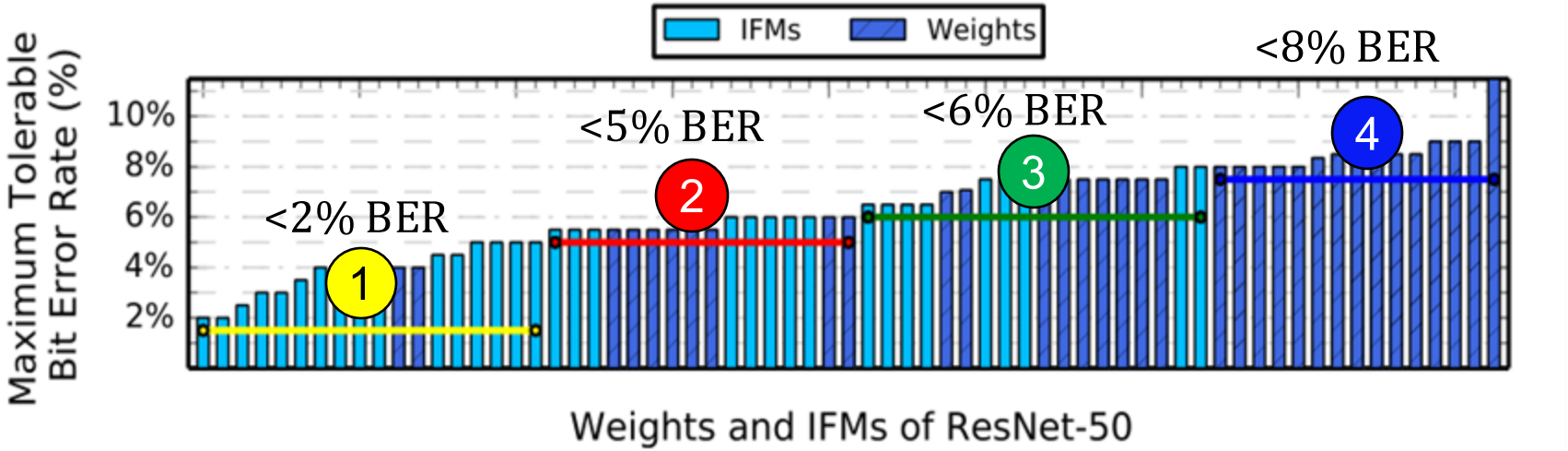
2. Reduce DRAM latency and voltage on such data and layers

3. While still achieving a user-specified DNN accuracy target by making training DRAM-error-aware

Data-aware management of DRAM latency and voltage for Deep Neural Network Inference

Example DNN Data Type to DRAM Mapping

Mapping example of ResNet-50:



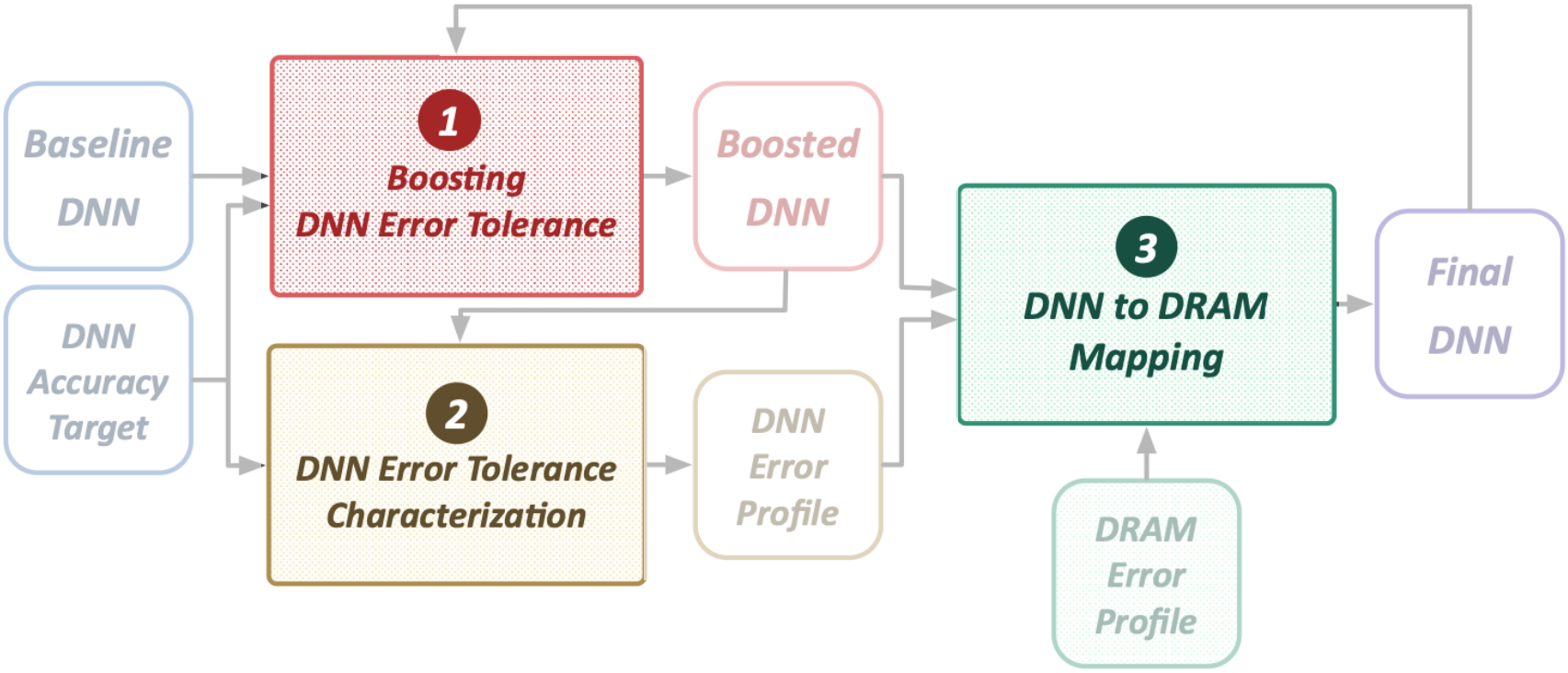
Map more error-tolerant DNN layers
to DRAM partitions **with lower voltage/latency**

4 DRAM partitions with different error rates

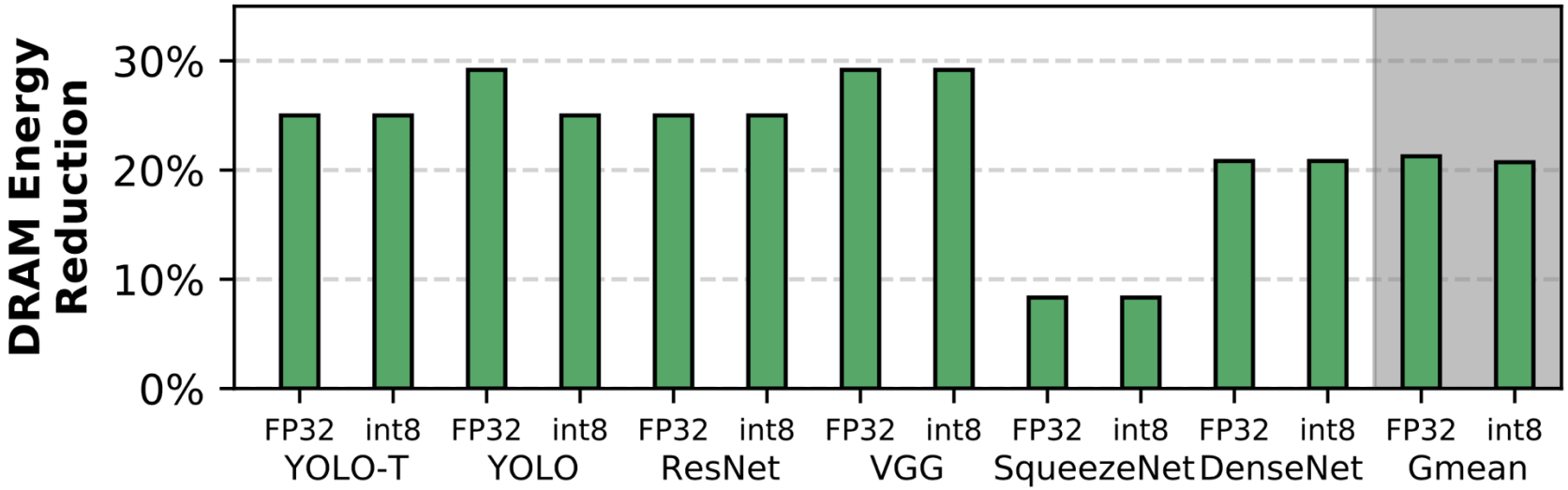
EDEN: Overview

Key idea: Enable **accurate, efficient** DNN inference using **approximate DRAM**

EDEN is an **iterative** process that has **3 key steps**



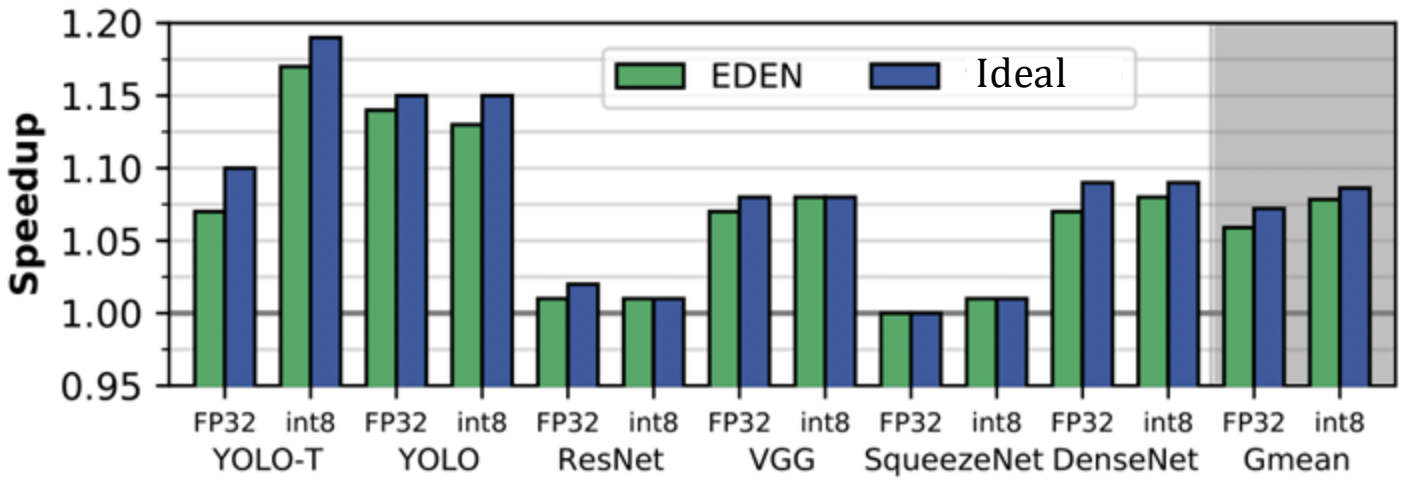
CPU: DRAM Energy Evaluation



Average 21% DRAM energy reduction

maintaining accuracy within 1% of original

CPU: Performance Evaluation



Average 8% system speedup
Some workloads achieve **17% speedup**

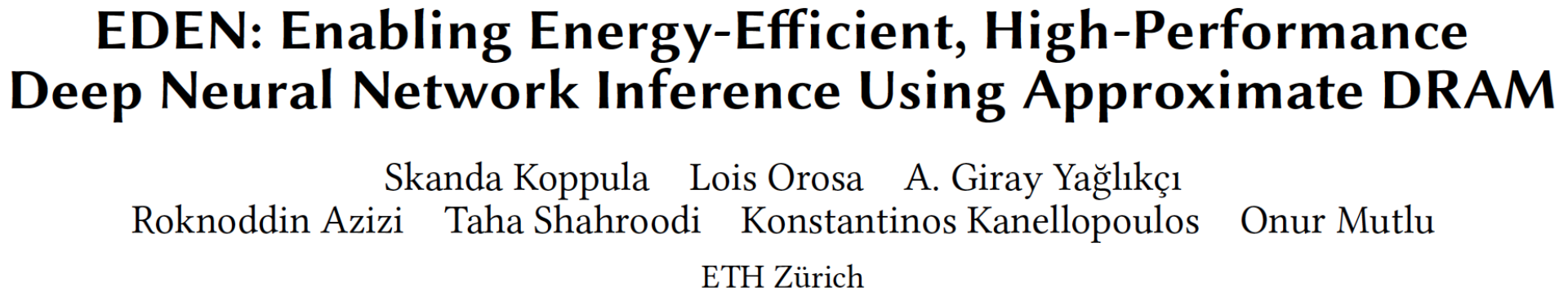
EDEN achieves **close to the ideal** speedup possible via tRCD scaling

GPU, Eyeriss, and TPU: Energy Evaluation

- **GPU**: average **37% energy reduction**
- **Eyeriss**: average **31% energy reduction**
- **TPU**: average **32% energy reduction**

EDEN: Data-Aware Efficient DNN Inference

- Skanda Koppula, Lois Orosa, A. Giray Yaglikci, Roknoddin Azizi, Taha Shahroodi, Konstantinos Kanellopoulos, and Onur Mutlu,
  "EDEN: Enabling Energy-Efficient, High-Performance Deep Neural Network Inference Using Approximate DRAM"
  *Proceedings of the 52nd International Symposium on Microarchitecture (**MICRO**)*, Columbus, OH, USA, October 2019.
  [Lightning Talk Slides (pptx) (pdf)]
  [Lightning Talk Video (90 seconds)]



SMASH: SW/HW Indexing Acceleration

- Konstantinos Kanellopoulos, Nandita Vijaykumar, Christina Giannoula, Roknoddin Azizi, Skanda Koppula, Nika Mansouri Ghiasi, Taha Shahroodi, Juan Gomez-Luna, and Onur Mutlu,
  "SMASH: Co-designing Software Compression and Hardware-Accelerated Indexing for Efficient Sparse Matrix Operations"
  *Proceedings of the 52nd International Symposium on Microarchitecture (**MICRO**)*, Columbus, OH, USA, October 2019.
  [Slides (pptx) (pdf)]
  [Lightning Talk Slides (pptx) (pdf)]
  [Poster (pptx) (pdf)]
  [Lightning Talk Video (90 seconds)]
  [Full Talk Lecture (30 minutes)]

SMASH: Co-designing Software Compression and Hardware-Accelerated Indexing for Efficient Sparse Matrix Operations

Konstantinos Kanellopoulos[1] Nandita Vijaykumar[2,1] Christina Giannoula[1,3] Roknoddin Azizi[1]
Skanda Koppula[1] Nika Mansouri Ghiasi[1] Taha Shahroodi[1] Juan Gomez Luna[1] Onur Mutlu[1,2]
[1]ETH Zürich [2]Carnegie Mellon University [3]National Technical University of Athens

Data-Aware Virtual Memory Framework

Nastaran Hajinazar, Pratyush Patel, Minesh Patel, Konstantinos Kanellopoulos, Saugata Ghose, Rachata Ausavarungnirun, Geraldo Francisco de Oliveira Jr., Jonathan Appavoo, Vivek Seshadri, and Onur Mutlu,
"The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework"
*Proceedings of the 47th International Symposium on Computer Architecture (**ISCA**)*, Virtual, June 2020.
[Slides (pptx) (pdf)]
[Lightning Talk Slides (pptx) (pdf)]
[ARM Research Summit Poster (pptx) (pdf)]
[Talk Video (26 minutes)]
[Lightning Talk Video (3 minutes)]
[Lecture Video (43 minutes)]

The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework

Nastaran Hajinazar[⋆†] Pratyush Patel[⋈] Minesh Patel[⋆] Konstantinos Kanellopoulos[⋆] Saugata Ghose[‡]
Rachata Ausavarungnirun[⊙] Geraldo F. Oliveira[⋆] Jonathan Appavoo[◇] Vivek Seshadri[▽] Onur Mutlu[⋆‡]

[⋆]*ETH Zürich* [†]*Simon Fraser University* [⋈]*University of Washington* [‡]*Carnegie Mellon University*
[⊙]*King Mongkut's University of Technology North Bangkok* [◇]*Boston University* [▽]*Microsoft Research India*

SW/HW Climate Modeling Accelerator

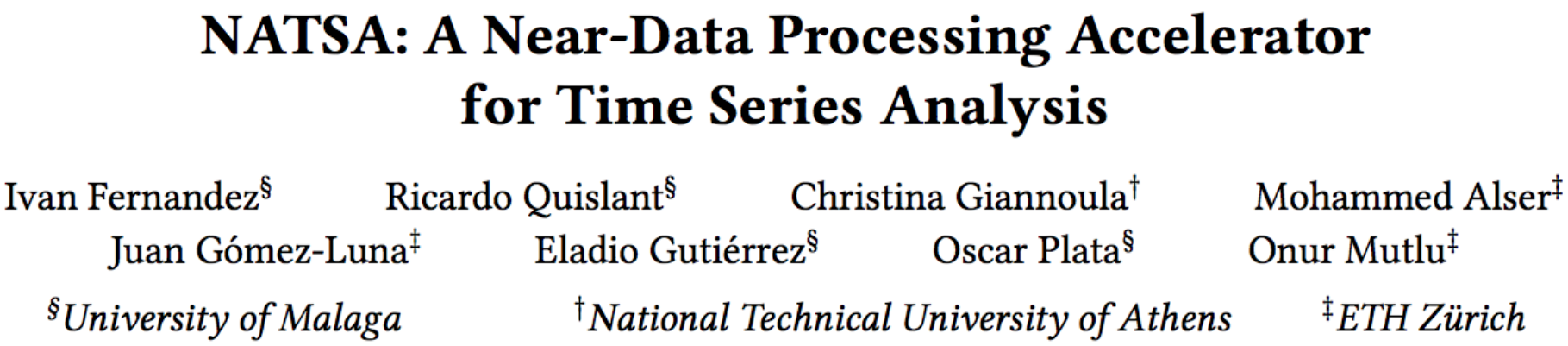
- Gagandeep Singh, Dionysios Diamantopoulos, Christoph Hagleitner, Juan Gómez-Luna, Sander Stuijk, Onur Mutlu, and Henk Corporaal,
  "NERO: A Near High-Bandwidth Memory Stencil Accelerator for Weather Prediction Modeling"
  *Proceedings of the 30th International Conference on Field-Programmable Logic and Applications (**FPL**)*, Gothenburg, Sweden, September 2020.
  [Slides (pptx) (pdf)]
  [Lightning Talk Slides (pptx) (pdf)]
  [Talk Video (23 minutes)]
  Nominated for the Stamatis Vassiliadis Memorial Award.

NERO: A Near High-Bandwidth Memory Stencil Accelerator for Weather Prediction Modeling

Gagandeep Singh[a,b,c] Dionysios Diamantopoulos[c] Christoph Hagleitner[c] Juan Gómez-Luna[b]
Sander Stuijk[a] Onur Mutlu[b] Henk Corporaal[a]
[a]Eindhoven University of Technology [b]ETH Zürich [c]IBM Research Europe, Zurich

HW/SW Time Series Analysis Accelerator

- Ivan Fernandez, Ricardo Quislant, Christina Giannoula, Mohammed Alser, Juan Gómez-Luna, Eladio Gutiérrez, Oscar Plata, and Onur Mutlu,
  "NATSA: A Near-Data Processing Accelerator for Time Series Analysis"
  *Proceedings of the 38th IEEE International Conference on Computer Design (**ICCD**)*, Virtual, October 2020.
  [Slides (pptx) (pdf)]
  [Talk Video (10 minutes)]
  [Source Code]



NATSA: A Near-Data Processing Accelerator for Time Series Analysis

Ivan Fernandez[§] Ricardo Quislant[§] Christina Giannoula[†] Mohammed Alser[‡]
Juan Gómez-Luna[‡] Eladio Gutiérrez[§] Oscar Plata[§] Onur Mutlu[‡]

[§]*University of Malaga* [†]*National Technical University of Athens* [‡]*ETH Zürich*

FPGA-based Processing Near Memory

- Gagandeep Singh, Mohammed Alser, Damla Senol Cali, Dionysios Diamantopoulos, Juan Gómez-Luna, Henk Corporaal, and Onur Mutlu, **"FPGA-based Near-Memory Acceleration of Modern Data-Intensive Applications"**
  IEEE Micro (***IEEE MICRO***), 2021.

FPGA-based Near-Memory Acceleration of Modern Data-Intensive Applications

Gagandeep Singh[◇] **Mohammed Alser**[◇] **Damla Senol Cali**[⋈]
Dionysios Diamantopoulos[▽] **Juan Gómez-Luna**[◇]
Henk Corporaal[⋆] **Onur Mutlu**[◇⋈]

[◇]*ETH Zürich* [⋈]*Carnegie Mellon University*
[⋆]*Eindhoven University of Technology* [▽]*IBM Research Europe*

Accelerating Linked Data Structures

- Kevin Hsieh, Samira Khan, Nandita Vijaykumar, Kevin K. Chang, Amirali Boroumand, Saugata Ghose, and Onur Mutlu,
  "Accelerating Pointer Chasing in 3D-Stacked Memory: Challenges, Mechanisms, Evaluation"
  *Proceedings of the 34th IEEE International Conference on Computer Design (**ICCD**)*, Phoenix, AZ, USA, October 2016.

Accelerating Pointer Chasing in 3D-Stacked Memory: Challenges, Mechanisms, Evaluation

Kevin Hsieh[†] Samira Khan[‡] Nandita Vijaykumar[†]
Kevin K. Chang[†] Amirali Boroumand[†] Saugata Ghose[†] Onur Mutlu[§†]
[†]*Carnegie Mellon University* [‡]*University of Virginia* [§]*ETH Zürich*

Accelerating Approximate String Matching

- Damla Senol Cali, Gurpreet S. Kalsi, Zulal Bingol, Can Firtina, Lavanya Subramanian, Jeremie S. Kim, Rachata Ausavarungnirun, Mohammed Alser, Juan Gomez-Luna, Amirali Boroumand, Anant Nori, Allison Scibisz, Sreenivas Subramoney, Can Alkan, Saugata Ghose, and Onur Mutlu, **"GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis"**
  *Proceedings of the 53rd International Symposium on Microarchitecture (**MICRO**)*, Virtual, October 2020.
  [Lighting Talk Video (1.5 minutes)]
  [Lightning Talk Slides (pptx) (pdf)]
  [Talk Video (18 minutes)]
  [Slides (pptx) (pdf)]

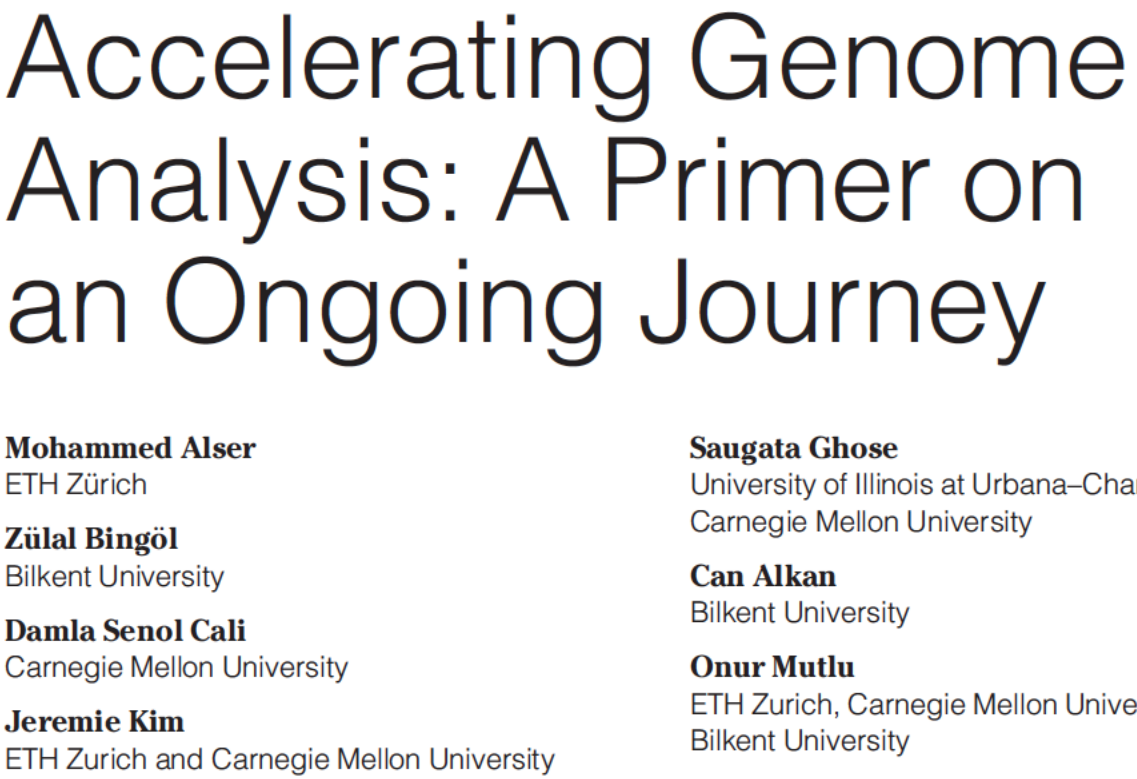
GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis

Damla Senol Cali[†⋈] Gurpreet S. Kalsi[⋈] Zülal Bingöl[▽] Can Firtina[◇] Lavanya Subramanian[‡] Jeremie S. Kim[◇†]
Rachata Ausavarungnirun[⊙] Mohammed Alser[◇] Juan Gomez-Luna[◇] Amirali Boroumand[†] Anant Nori[⋈]
Allison Scibisz[†] Sreenivas Subramoney[⋈] Can Alkan[▽] Saugata Ghose[⋆†] Onur Mutlu[◇†▽]

[†]*Carnegie Mellon University* [⋈]*Processor Architecture Research Lab, Intel Labs* [▽]*Bilkent University* [◇]*ETH Zürich*
[‡]*Facebook* [⊙]*King Mongkut's University of Technology North Bangkok* [⋆]*University of Illinois at Urbana–Champaign*

Accelerating Genome Analysis [IEEE MICRO 2020]

- Mohammed Alser, Zulal Bingol, Damla Senol Cali, Jeremie Kim, Saugata Ghose, Can Alkan, and Onur Mutlu,
  "Accelerating Genome Analysis: A Primer on an Ongoing Journey"
  IEEE Micro (***IEEE MICRO***), Vol. 40, No. 5, pages 65-75, September/October 2020.
  [Slides (pptx)(pdf)]
  [Talk Video (1 hour 2 minutes)]



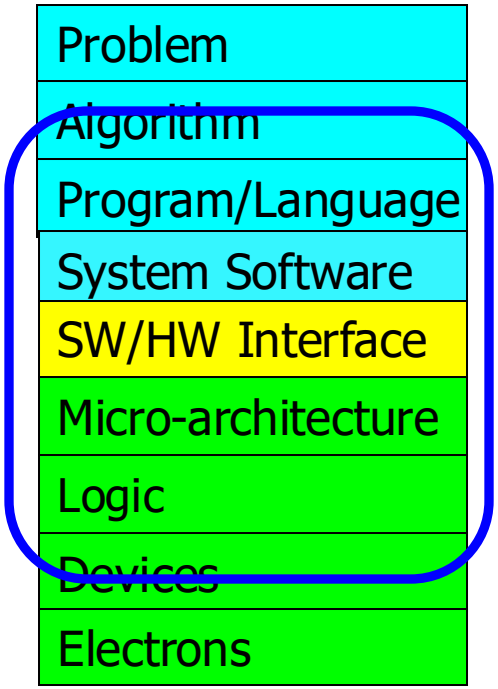
Challenge and Opportunity for Future

Data-Aware

(Expressive)

Computing Architectures

We Need to **Rethink** the Entire Stack



We can get there case by case